

AI Agent Harness 中文教程

Build Your Own AI Coding Agent Harness · Vercel Academy 社区中文译本

原作者 Joel Hooks · 非官方中文翻译 · 11 模块 / 39 页 · 个人离线归档 2026-09-05

目录

来源与版权说明	11
课程结构	12
课程介绍	13
自己动手构建 AI 编码 Agent Harness	13
你将构建什么	13
前置要求	13
课程如何进行	13
课程模块	14
模块 1：Agent 循环	14
模块 2：工具设计	14
模块 3：系统提示词	14
模块 4：沙箱抽象	14
模块 5：上下文管理	14
模块 6：子 Agent 委派	15
模块 7：沙箱生命周期	15
模块 8：人机协作	15
模块 9：规划与验证	15
模块 10：交互界面	15
模块 11：可扩展性	15
综合项目	16
技术栈	16
从聊天机器人到 Agent	17
从聊天机器人到 Agent	17
学习成果	17
快速路径	17
动手练习 1.1	17
聊天机器人	18
一个工具改变一切	19
为什么限制为 500 行	21
动手试试	21
提交	22
完成标准	22
参考实现	22
你的第一批工具	26

你的第一批工具	26
学习成果	26
快速路径	26
动手练习 1.2	26
看错工具赢了	27
描述提示词	27
为什么要有 50 场比赛上限	30
动手试试	30
提交	31
完成标准	31
参考实现	31
补齐工具箱	37
补齐工具箱	37
学习成果	37
快速路径	37
动手练习 1.3	37
为什么不 needsApproval ?	37
执行层门	38
动手试试	40
三利一 Agent	41
提交	41
完成标准	41
参考实现	42
有效的工具描述	49
有效的工具描述	49
学习成果	49
快速路径	49
动手练习 2.1	49
为什么要用双重负片	50
完整契约	50
为什么每个部分都值得被认可	53
动手试试	53
提交	54
完成标准	54
参考实现	54
安全执行 Shell	58
安全执行 Shell	58
学习成果	58
快速路径	58
动手练习 2.2	58
接缝的走向	59
建工厂	59
构建本地后端	61
动手试试	62
提交	63
完成标准	63
参考实现	63

审批门	67
审批门	67
学习成果	67
快速路径	67
动手练习 2.3	67
第一阶段：布尔值	68
第二阶段：功能	68
第三阶段：可辨识联合类型	68
这会解锁什么	71
动手试试	71
提交	71
完成标准	72
参考实现	72
组织 Agent 指令	75
组织 Agent 指令	75
学习成果	75
快速路径	75
动手练习 3.1	75
之前	75
之后	76
具体发生了什么变化	77
动手试试	77
提交	77
完成标准	78
参考实现	78
动态构建系统提示词	80
动态构建系统提示词	80
学习成果	80
快速路径	80
动手练习 3.2	80
语境的形态	80
构建器	81
接入主流程	83
动手试试	84
提交	84
完成标准	84
参考实现	85
验证门	88
验证门	88
学习成果	88
快速路径	88
动手练习 3.3	88
契约	88
范围化的索赔，并排呈现	89
这有什么作用，什么不做	90
动手试试	90
提交	90

完成标准	91
参考实现	91
项目上下文	93
项目上下文	93
学习成果	93
快速路径	93
动手练习 3.4	93
发现经过	93
AGENTS.md 里会有什么	94
为什么现在一个文件就足够了	96
动手试试	96
提交	96
完成标准	97
参考实现	97
设计接口	99
设计沙箱接口	99
学习成果	99
快速路径	99
动手练习 4.1	99
接口	100
每种方法的收益	100
重构工具	101
动手试试	101
提交	102
完成标准	102
参考实现	102
本地实现	105
本地实现	105
学习成果	105
快速路径	105
动手练习 4.2	105
实现	106
完成接线	107
动手试试	108
提交	108
完成标准	109
参考实现	109
内存实现	111
内存实现	111
学习成果	111
快速路径	111
动手练习 4.3	111
just-bash API	112
实现	112
接线环境变换开关	114
写时复制，一句话	114
动手试试	114

提交	115
完成标准	115
参考实现	116
云端实现	118
云端实现	118
学习成果	118
云层沙箱长什么样	118
权衡取舍	120
何时选择每种	120
expiresAt 和 snapshot 给你买什么	121
动手试试	121
提交	121
完成标准	121
生命周期钩子	122
生命周期钩子	122
学习成果	122
快速路径	122
动手练习 4.5	122
接口	123
每个钩子的作用	123
接到 Agent 循环	124
动手试试	125
提交	126
完成标准	126
参考实现	126
问题所在	128
问题所在	128
学习成果	128
快速路径	128
动手练习 5.1	128
添加日志记录	129
执行一项痛苦的任务	129
为什么输入 token 增长	130
什么不行	130
动手试试	130
提交	131
完成标准	131
参考实现	131
修剪旧结果	133
修剪旧结果	133
学习成果	133
快速路径	133
动手练习 5.2	133
修复方案	134
到底发生了什么	134
为什么要三条消息	135
动手试试	135

提交	136
完成标准	136
参考实现	137
工具输出设计	139
工具输出设计	139
学习成果	139
快速路径	139
动手练习 5.3	139
帽表	139
带电容的 Bash 输出	140
结构化回报, 而非原始数据倾销	140
截断契约	141
动手试试	141
提交	142
完成标准	142
参考实现	143
缓存控制	145
缓存控制	145
学习成果	145
快速路径	145
动手练习 5.4	145
辅助函数	146
用修剪来作曲	147
节省的具体情况	147
为什么即使在不支持缓存的情况下, 模式也很重要	148
动手试试	148
提交	149
完成标准	149
参考实现	149
为什么要委派	152
为什么要委派	152
学习成果	152
失效模式	152
环境污染	152
失去了专注	152
过宽能力	152
模式	152
何时委托	153
动手试试	154
提交	154
完成标准	154
探索型子 Agent	155
探索型子 Agent	155
学习成果	155
快速路径	155
动手练习 6.2	155
task 工具	156

接入父 Agent	158
动手试试	158
提交	159
完成标准	159
参考实现	160
执行型子 Agent	162
执行型子 Agent	162
学习成果	162
快速路径	162
动手练习 6.3	162
执行器分支	163
探索器与执行器一览	166
教学质量对执行器更重要	166
动手试试	167
提交	167
完成标准	168
参考实现	168
任务工具	169
任务工具	169
学习成果	169
快速路径	169
动手练习 6.4	169
路由器结构	169
生成权限的去向	173
按角色模型，而非按会话模型	173
动手试试	174
提交	174
完成标准	174
参考实现	175
状态机	176
状态机	176
学习成果	176
四种状态	176
两类超时	176
硬到期	177
无活动窗口	177
哪些事件算作活动	177
沿时间线分析	177
动手试试	178
提交	178
完成标准	178
快照与恢复	179
快照与恢复	179
学习成果	179
API	179
快照保存了什么	179
三大幂等性险	179

1. 快照正在进行中	179
2. 沙箱已经在还原中运行	180
3. 双音	181
恢复解决不了的问题	181
动手试试	181
提交	182
完成标准	182
持久化工作流	183
持久化工作流	183
学习成果	183
setTimeout 的问题	183
工作流程模式	183
阶梯边界	185
成本数学	185
演示版戛然而止	186
动手试试	186
提交	186
完成标准	186
实战经验	187
实战经验	187
学习成果	187
重新连接后句柄陈旧	187
过期到期数据	187
民调重置不活跃	188
自动恢复循环	188
状态分歧	189
动手试试	189
提交	190
完成标准	190
结构化提问	191
结构化提问	191
学习成果	191
快速路径	191
动手练习 8.1	191
工具	192
系统提示词的增建	193
接入工具	194
动手试试	194
提交	195
完成标准	195
参考实现	195
审批配置	197
审批配置	197
学习成果	197
两个模型	197
何时使用哪种	198
它们如何结合	199

随行教学缺少什么	199
动手试试	200
提交	200
完成标准	200
Todo 工具	201
Todo 工具	201
学习成果	201
快速路径	201
动手练习 9.1	201
工具	201
接入主流程	204
什么时候该计划，什么时候不该	205
动手试试	205
提交	206
完成标准	206
参考实现	206
快速理解上下文	207
快速理解上下文	207
学习成果	207
快速路径	207
动手练习 9.2	207
提示词的增建	208
实际模式	208
并行阅读，适用时	209
动手试试	209
提交	210
完成标准	210
验证契约	211
验证契约	211
学习成果	211
快速路径	211
动手练习 9.3	211
探索 package.json 的门	212
传到提示词	213
接入主流程	214
范围化的索赔，并排呈现	215
动手试试	215
提交	216
完成标准	216
参考实现	216
CLI 入口	218
CLI 入口	218
学习成果	218
快速路径	218
动手练习 10.1	218
CLI	219
为什么这 finally 重要	223

动手试试	224
提交	224
完成标准	224
参考实现	225
流式输出与工具渲染	226
流式输出与工具渲染	226
学习成果	226
快速路径	226
动手练习 10.2	226
流式输出循环	227
实际操作中的样子	228
各工具渲染选择	228
动手试试	229
提交	229
完成标准	229
Web 界面	231
Web 界面	231
学习成果	231
两个表面并排	231
Web 界面新增内容	231
持久性	231
流式输出 HTTP	232
工具结果作为组成部分	233
可恢复流	233
随行教学缺少什么	233
动手试试	233
提交	234
完成标准	234
技能系统	235
技能系统	235
学习成果	235
快速路径	235
动手练习 11.1	235
技能形态	236
他们在提示词中浮出水面	238
装载工具	239
接入主流程	240
为什么要“名字进，内容出”	241
动手试试	241
提交	242
完成标准	242
参考实现	243
自定义工具	244
自定义工具	244
学习成果	244
快速路径	244
动手练习 11.2	244

注册表	245
内置辅助器	246
包装器	247
接入 CLI	248
为项目包装 <code>bash</code>	249
动手试试	250
提交	251
完成标准	251
参考实现	251
扩展点	252
扩展点	252
学习成果	252
事件表面	252
四个例子	253
每当记工具调用	253
块写入受保护文件	253
在压实前注入安全防提示词	254
关机时自动提交	254
链式规则	255
事件如何与你已有的构建结合	255
动手试试	256
提交	256
完成标准	256

来源与版权说明

本册是个人离线学习归档，供本人事后阅读与跟学使用，请勿再分发。

项	内容
原课程	<i>Build Your Own AI Coding Agent Harness</i>
原作者	Joel Hooks
原课程出处	Vercel Academy , https://vercel.com/academy/build-ai-agent-harness
中文翻译	社区非官方译本, https://agentharnesszh.vercel.app/
抓取范围	导言 + 11 个模块 / 共 39 页 (站点侧边栏全量)
抓取时间	2026-09-05

译站自带的声明：

原作者： Joel Hooks · 原课程： Vercel Academy 。本页为非官方中文翻译。代码、命令与 API 名称保持原样；遇到歧义请以英文原文为准。

跟学提醒： 这是非官方译本，术语偶有生硬（如把 `harness` 直译保留、个别句子机翻痕迹明显）。每页顶部都保留了对应的原文 URL，读着别扭时直接对照英文原页。技术栈为 TypeScript + AI SDK + Vercel Sandbox + just-bash + Zod v3。

课程结构

模块	主题	课数
导言	课程介绍	1
模块 1	Agent 循环	3
模块 2	工具设计	3
模块 3	系统提示词	4
模块 4	沙箱抽象	5
模块 5	上下文管理	4
模块 6	子 Agent 委派	4
模块 7	沙箱生命周期	4
模块 8	人机协作	2
模块 9	规划与验证	3
模块 10	交互界面	3
模块 11	可扩展性	3

课程是「因果序列」设计——每一课解决上一课暴露出来的问题，建议按顺序读，别跳。

课程介绍

导言 · 第 1/39 页 · 原文: <https://agentharnesszh.vercel.app>

自己动手构建 AI 编码 Agent Harness

使用 AI SDK 、 Vercel Sandbox 和 just-bash 从零构建 AI 编码 Agent Harness 。涵盖工具循环、工具设计、系统提示词、沙箱抽象、上下文修剪、子 Agent 委派、生命周期管理和可扩展性。

只有一个工具循环和三个工具，还只能算演示。真正拿它干活时，问题才会出现：读取一个 5000 行的文件后，内容会永远留在上下文里；给它 `bash`，它可能直接运行 `rm -rf`；让它重构模块，它却只解释应该怎样重构；一个长任务就能填满上下文窗口，让 Agent 忘掉自己的指令；云沙箱按分钟计费，超时后代码还可能消失。

Harness 指的就是围绕 Agent 搭建、专门处理这些问题的整套系统。本课程将从零开始构建 **TeensyCode**。

你将构建什么

TeensyCode 是一套可实际运行的 AI 编码 Agent Harness 。它拥有紧凑的 TypeScript 核心、完整工具集和多种沙箱后端。因为每个部分都由你亲手实现，所以它能够彻底理解它：

- **工具循环**：基于 **ToolLoopAgent**，提供 `read`、`grep`、`write`、`edit`、`bash`、`task` 和 `askUser` 工具
- **安全门**：从安全命令允许列表开始，在执行层拦截危险操作，并逐步演进为支持交互、后台和委派场景的可配置审批
- **行为提示词**：使用 Agency 、 Guardrails 和 Handling Ambiguity 等结构化章节，并注入 **AGENTS.md** 实现项目级配置
- **沙箱抽象**：使用统一的 **Sandbox** 接口，提供本地实现（Node fs 与 child_process）和内存实现（just-bash 与写时复制虚拟文件系统）；切换后端时无需修改工具
- **上下文管理**：结合 `pruneMessages`、有界工具输出和缓存控制，让长会话保持可用并控制成本
- **子 Agent 委派**：为探索器和执行器提供隔离上下文、受限工具，并按任务角色选择模型
- **人机协作**：使用带多选项的 `askUser`，并遵循“先搜索、再提问、后行动”的歧义处理协议
- **沙箱生命周期**：使用状态机理解生命周期，并实现快照、恢复和持久化工作流
- **可扩展性**：通过事件总线、渐进式披露的技能系统和自定义工具注册扩展 Harness

前置要求

- 熟悉 TypeScript 、 async/await 和基本终端操作
- 已设置 **AI_GATEWAY_API_KEY** 环境变量
- Node.js 20+ 或 Bun 运行时
- 建议先学习 [构建文件系统 Agent](#) 课程

课程如何进行

按因果顺序推进。每一步都源于上一步暴露的问题。第一步加入 `read`，因为聊天机器人看不到文件；第二步加入 `grep`，因为 Agent 无法搜索；第三步加入 `bash`，因为它不能执行命令，但这也带来了运行 `rm -rf` 的风险。每一步只聚焦一个概念，同时让其余部分始终可以运行。

模块 1 到 6 以边学边做为主：编写代码、运行并验证。模块 7 侧重概念和分析，因为沙箱生命周期涉及不适合在本地随意演示的持久化工作流与状态机。模块 8 到 11 则结合实现与分析。

课程模块

模块 1：Agent 循环

从一个不带工具的 `ToolLoopAgent`（聊天机器人）开始，加入 `read` 和 `grep` 使其成为 Agent，再为 `bash` 加上安全门。

- **从聊天机器人到 Agent**：一个工具如何让聊天机器人变成 Agent
- **你的第一批工具**：为什么工具描述就是模型的选择接口
- **补齐工具箱**：如何在执行层拦截危险工具

模块 2：工具设计

把描述扩展为五段式契约，提取工具工厂模式，并实现可配置审批。

- **有效的工具描述**：WHEN TO USE、WHEN NOT TO USE、DO NOT USE FOR 与 EXAMPLES
- **安全执行 Shell**：用工厂和操作接口分离契约与执行
- **审批门**：从布尔值演进到函数和可辨识联合类型

模块 3：系统提示词

通过结构化指令、动态组合、验证门和 `AGENTS.md` 塑造 Agent 行为。

- **组织 Agent 指令**：Agency 加 Guardrails，让 Agent 行动而不是只做解释
- **动态构建系统提示词**：让 `buildSystemPrompt()` 根据运行时上下文调整提示词
- **验证门**：建立 typecheck、lint、test 和 build 契约
- **项目上下文**：放入一个 `AGENTS.md` 即可改变 Agent 行为

模块 4：沙箱抽象

一个接口，多种实现。工具调用 `sandbox.exec()`，而不是直接调用 `child_process.exec()`。

- **设计沙箱接口**：定义包含 `readFile`、`exec` 和 `stop` 的 `Sandbox` 类型
- **本地实现**：封装 Node fs 与 `child_process`
- **内存实现**：使用 just-bash 和写时复制覆盖层
- **云端实现**：理解远程 VM 的概念与取舍
- **生命周期钩子**：实现 `afterStart`、`beforeStop` 和 `onTimeout`

模块 5：上下文管理

每次工具调用都会一直留在上下文中。通过修剪、有界输出和缓存控制解决这个问题。

- **问题所在**：通过 token 日志观察线性增长
- **修剪旧结果**：使用 `pruneMessages` 避免旧工具输出不断堆积
- **工具输出设计**：预防优于清理，为每个工具设置明确上限

- **缓存控制**：使用服务商请求头降低重复上下文成本

模块 6：子 Agent 委派

父 Agent 负责规划，子 Agent 负责执行；各自使用隔离上下文、受限工具和按角色选择的模型。

- **为什么要委派**：理解单 Agent 的失效模式
- **探索型子 Agent**：只读、低成本、受约束的探索
- **执行型子 Agent**：完整工具、更强模型和委派式信任
- **任务工具**：处理路由、权限和角色模型选择

模块 7：沙箱生命周期

云沙箱既有成本又会超时。本模块侧重概念与分析。

- **状态机**：状态转换、超时和活动跟踪
- **快照与恢复**：冻结、恢复和幂等性风险
- **持久化工作流**：结合 Vercel Workflow 与 `sleep()`
- **实战经验**：生命周期系统中的生产环境陷阱

模块 8：人机协作

猜错方向的 Agent，往往比主动提问的 Agent 浪费更多时间。

- **结构化提问**：使用带多选项和歧义处理协议的 `askUser`
- **审批配置**：用配置定义模式，用事件承载策略

模块 9：规划与验证

行动前规划，行动后验证。

- **Todo 工具**：带状态跟踪的任务分解
- **快速理解上下文**：先 `grep`，只读取准备修改的内容
- **验证契约**：按顺序执行验证门，并限定结论范围

模块 10：交互界面

Agent 本身没有界面。CLI、TUI 和 Web 只是不同的渲染策略。

- **CLI 入口**：参数、沙箱工厂与可靠退出
- **流式输出与工具渲染**：实时文本和工具调用展示
- **Web 界面**：同一个 Agent，不同的渲染器

模块 11：可扩展性

依靠事件而不是继承扩展系统；技能采用渐进式披露，工具采用注册机制。

- **技能系统**：提示词只列名称，需要时再加载完整内容
- **自定义工具**：无需 `fork` 即可注册，并能组合已有工具
- **扩展点**：通过生命周期事件订阅、阻止或修改行为

综合项目

让你的 Harness 在真实项目中完成任务。不要只做“添加 Hello World 接口”，而应尝试“为认证路由添加速率限制”。观察上下文在哪里溢出、Agent 在哪里选错工具、子 Agent 在哪里收到含糊指令，然后逐一修复实际暴露的问题。

技术栈	
组件	作用
AI SDK	ToolLoopAgent、tool()、stepCountIs、pruneMessages 与流式输出
AI Gateway	模型路由；可直接使用 "anthropic/claude-haiku-4-5" 字符串，无需额外包装器
Vercel Sandbox	提供带隔离文件系统、git 和 npm 的远程 VM
just-bash	提供内存虚拟文件系统和模拟 bash
Vercel Workflow	为沙箱生命周期提供持久化工作流
Zod v3	定义工具输入 schema；v4 会破坏 AI SDK v6 的类型兼容性

从聊天机器人到 Agent

模块 1 · Agent 循环 · 第 2/39 页 · 原文: <https://agentharnesszh.vercel.app/from-chat-to-agent>

从聊天机器人到 Agent

你的 Agent 就像世界上最自信的实习生。让它查看 `tsconfig.json`，它会兴致勃勃地描述文件里大概有什么；让它找出第 42 行的错误，它也能给出一个听起来非常可信的修复方案。

但它根本没有打开文件，因为它做不到。聊天机器人没有工具，只能根据常见代码模式进行猜测，再把猜测当成分析结果。

一个工具就能改变这种情况。我们将添加 `read`，让这个只会自信解释的实习生真正打开文件。

学习成果

你将得到一个 `ToolLoopAgent`：收到提示词后，它会调用 `read` 工具检查指定文件并汇报发现。

快速路径

1. 在 `index.ts` 中创建 `ToolLoopAgent`，配置 `instructions`、`model` 和 `stopWhen: stepCountIs(10)`
2. 先不提供任何工具，运行它并观察聊天机器人如何只解释自己“会做什么”
3. 添加带 Zod `inputSchema` 和 500 行上限的 `read` 工具，然后再次运行

动手练习 1.1

先在 `index.ts` 中构建最小可用的 Agent，再添加一个工具。

要求：

1. 从 `ai` 导入 `ToolLoopAgent`、`stepCountIs` 和 `tool`，并从 `zod` 导入 `z`
2. 创建 Agent，设置 `model: "anthropic/claude-haiku-4-5"`、简短指令和 `stopWhen: stepCountIs(10)`
3. 添加 `read` 工具，接受 `path`、可选的 `offset` 和可选的 `limit`
4. 将输出限制为 500 行，并在每行前加上行号

实现提示：

- `ToolLoopAgent` 接受 `instructions`、`model`、`tools` 和 `stopWhen`。这里应使用 `instructions`，而不是 `system`
- 使用 `agent.generate({ prompt })` 调用 Agent，而不是 `agent.generate(prompt)`
- `tool()` 的 `description` 字段是提供给模型的提示词，不是普通 `docstring`；模型根据它判断何时调用工具
- 基于工作目录解析路径，避免 Agent 意外读取项目外的文件

聊天机器人

从最小的 **Agent** 开始：没有工具，只有指令。

```
import { ToolLoopAgent, stepCountIs } from "ai";

const cwd = process.argv[2] || process.cwd();

const agent = new ToolLoopAgent({
  model: "anthropic/claude-haiku-4-5",
  instructions: `You are a coding agent.\nWorking directory: ${cwd}`,
  tools: {},
  stopWhen: stepCountIs(10),
});

const prompt = process.argv.slice(3).join(" ") || "Hello!";
const { text, steps } = await agent.generate({ prompt });
console.log(text);
console.log(`\n(${steps.length} steps)`);
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

运行它：

```
bun run index.ts . "What files are in this project?"
```

1

你会得到一段礼貌、热心但完全虚构的回复，例如“我很乐意帮你探索项目文件！”，随后列出它打算查看的内容。问题是，它其实什么也看不到。这就是聊天机器人。

(1 steps)

1

一步。没有工具调用。模型会说话，这就是它能做的全部。

**** 注意： AI SDK v6 命名 ****

请使用 `instructions`（不是 `system`）、`stopWhen`（不是 `stopCondition`）以及 `agent.generate({ prompt })`（不是 `agent.generate(prompt)`）。这些名称在不同 AI SDK 版本中发生过变化。错误名称可能仍能通过编译，但 Agent 不会按预期运行。

一个工具改变一切

现在添加 `read` 工具。这一步会让聊天机器人真正变成 Agent：

```
import { ToolLoopAgent, stepCountIs, tool } from "ai";
import { z } from "zod";
import { readFileSync } from "node:fs";
import { resolve } from "node:path";

const cwd = resolve(process.argv[2] || process.cwd());

const read = tool({
  description: `Read a file from the project. Returns numbered lines.
  WHEN TO USE: viewing file contents, checking configs, reading source code.
  WHEN NOT TO USE: searching across files (use grep instead).`,
  inputSchema: z.object({
    path: z.string().describe("File path relative to working directory"),
    offset: z.number().optional().describe("Start line (1-indexed)"),
    limit: z.number().optional().describe("Max lines to return"),
  }),
  execute: async ({ path: filePath, offset, limit }) => {
    const abs = resolve(cwd, filePath);
    const content = readFileSync(abs, "utf-8");
    let lines = content.split("\n");

    if (offset) lines = lines.slice(offset - 1);
    if (limit) lines = lines.slice(0, limit);

    const MAX_LINES = 500;
    const truncated = lines.length > MAX_LINES;
    if (truncated) lines = lines.slice(0, MAX_LINES);

    const numbered = lines.map((l, i) => `${(offset || 1) + i}: ${l}`);
    return truncated
      ? numbered.join("\n") + `\n... (truncated at ${MAX_LINES} lines)`
```

```
        : numbered.join("\n");
    },
  });

const agent = new ToolLoopAgent({
  model: "anthropic/claude-haiku-4-5",
  instructions: `You are a coding agent.\nWorking directory: ${cwd}`,
  tools: { read },
  stopWhen: stepCountIs(10),
});
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32

33
34
35
36
37
38
39
40
41

`description` 字段承担的工作比看起来更多。模型会先阅读所有工具描述，再决定下一步做什么。WHEN TO USE 和 WHEN NOT TO USE 不是写给开发者看的注释，而是帮助模型在多个工具之间做选择的提示词。

为什么限制为 500 行

注意 `MAX_LINES = 500`。如果没有它，对一个 10000 行文件执行无界 `read` 会把全部内容塞进上下文窗口，而且 Agent 会在余下会话中一直携带这份结果。一次不谨慎的读取就可能消耗 10% 的可用上下文。

本课程会对每个工具贯彻这种限制。后面有专门的上下文管理模块，但良好习惯应当从工具设计本身开始。

动手试试

使用与 `read` 实际能力匹配的提示词运行 Agent：

```
bun run index.ts . "Read the tsconfig.json"
```

1

你应该会看到模型先调用 `read`，再总结文件内容。整个过程需要两步，而不是一步：

```
Here's the tsconfig.json:  
- target: ESNEXT  
- moduleResolution: bundler  
- strict: true
```

(2 steps)

1
2
3
4
5
6

转变就是这么简单：一次工具调用，再生成一次回答。模型之所以选择 `read`，是因为工具描述明确说明了它的适用场景。

**** 注意：选择与工具能力匹配的提示词 ****

`read` 可以检查已知文件，但不能列出目录。在后面两课加入 `grep` 和 `bash` 之前，请使用 `Read the tsconfig.json` 或 `Read package.json` 这类提示词。

快速检查类型：

```
npx tsc --noEmit
```

1

提交

```
git add index.ts
git commit -m "feat(agent): add ToolLoopAgent with read tool"
```

1

2

完成标准

- ☐ `index.ts` 中的 `ToolLoopAgent` 配置了 `read` 工具
- ☐ 聊天机器人版本（无工具）在一步后返回
- ☐ `Agent` 版本（带 `read`）会调用工具并报告文件内容
- ☐ `read` 返回带编号的行，并支持可选偏移量和行数限制
- ☐ 输出在 500 行处截断，并附带清晰说明
- ☐ `npx tsc --noEmit`

**** 注意：移除上限再试一次 ****

用 `seq 1 1000 > /tmp/big.txt` 创建一个 1000 行文件，然后让 `Agent` 读取它。移除 `MAX_LINES = 500` 限制后再运行一次，观察响应规模的变化。再想象一个持续 30 步的任务：先出问题的会是模型推理能力，还是 token 上限？

参考实现

```
import { ToolLoopAgent, stepCountIs, tool } from "ai";
import { z } from "zod";
import { readFileSync } from "node:fs";
import { resolve } from "node:path";
```

```
const cwd = resolve(process.argv[2] || process.cwd());

const read = tool({
  description: `Read a file from the project. Returns numbered lines.
  WHEN TO USE: viewing file contents, checking configs, reading source code.
  WHEN NOT TO USE: searching across files (use grep instead).`,
  inputSchema: z.object({
    path: z.string().describe("File path relative to working directory"),
    offset: z.number().optional().describe("Start line (1-indexed)"),
    limit: z.number().optional().describe("Max lines to return"),
  }),
  execute: async ({ path: filePath, offset, limit }) => {
    const abs = resolve(cwd, filePath);
    const content = readFileSync(abs, "utf-8");
    let lines = content.split("\n");

    if (offset) lines = lines.slice(offset - 1);
    if (limit) lines = lines.slice(0, limit);

    const MAX_LINES = 500;
    const truncated = lines.length > MAX_LINES;
    if (truncated) lines = lines.slice(0, MAX_LINES);

    const numbered = lines.map((l, i) => `${(offset || 1) + i}: ${l}`);
    return truncated
      ? numbered.join("\n") + `\n... (truncated at ${MAX_LINES} lines)`
      : numbered.join("\n");
  },
});

const agent = new ToolLoopAgent({
  model: "anthropic/claude-haiku-4-5",
  instructions: `You are a coding agent.\nWorking directory: ${cwd}`,
  tools: { read },
  stopWhen: stepCountIs(10),
});

const prompt = process.argv.slice(3).join(" ") || "Hello!";
const { text, steps } = await agent.generate({ prompt });
console.log(text);
console.log(`\n(${steps.length} steps)`);
```

1
2
3
4

5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46

你的第一批工具

模块 1 · Agent 循环 · 第 3/39 页 · 原文: <https://agentharnesszh.vercel.app/your-first-tools>

你的第一批工具

上一节课，你把 Agent `read` 交给了聊天机器人，变成了有用的东西。有用，但有限。你的 Agent 只能打开它已经知道名字的文件。让它“查找所有 TODO 注释”，它开始猜哪些文件可能有注释，然后逐条阅读。

那不是搜索。那是礼貌地挥舞。

加上 `grep`，Agent 就有了真正的搜索工具。但现在你有了新的问题。模型每次接手任务时都要在 `read` 和 `grep` 之间做选择。除非你告诉它怎么做，否则它就会选错。

学习成果

你拥有一个 `grep` 工具，描述丰富且能塑造行为。模型使用 `grep` 进行搜索，`read` 用于文件检查，路径完全由描述决定。

快速路径

1. 添加一个带有正则表达式模式、可选的球状过滤器和 50 匹配上限的 `grep` 工具
2. 用 WHEN TO USE、WHEN NOT TO USE、DO NOT USE FOR 和 EXAMPLES 来写描述
3. 更新 `read` 的描述以匹配相同的契约

动手练习 1.2

先构建 `grep` 工具，然后重写两个描述，直到模型正确路由。

要求：

1. 添加一个带有 `pattern`、可选 `path` 和可选 `glob` Zod schema 的 `grep` 工具
2. 使用 `execSync` 实现 `execute`grep -rn``，排除 `node_modules` 和 `.git`
3. 输出上限为 50 匹配并报告总数量
4. 使用四部分的契约描述来 `read` 和 `grep`：WHEN TO USE，WHEN NOT TO USE，DO NOT USE FOR，EXAMPLES

实现提示：

- 从 `node:child_process` 进口 `execSync`
- 引用输入 `shell` 命令以避免在特殊字符上中断
- 将 `grep` 的非零退出（未找到匹配）视为成功，而非错误
- 描述是模型选择工具的用 API。写给模型看，而不是读者

看错工具赢了

先做个简短的描述，看看情况有多糟：

```
const grep = tool({
  description: "Search files.",
  inputSchema: z.object({
    pattern: z.string(),
    glob: z.string().optional(),
  }),
  // ... execute with execSync grep
});
```

1
2
3
4
5
6
7
8

现在问问 Agent：

```
bun run index.ts . "Find all TODO comments in this project"
```

1

模型无视 **grep**，伸手拿起 **read**，打开随机文件，希望有个 **TODO**。如果你已经添加了 **bash**，它会尝试那个。两个词的描述无法给模型任何可用的信息，所以它只能猜测。

这是第一次工具选择重要。这也是它第一次断裂。

描述提示词

修复并不是更好的实现。这样描述更准确：

```
const grep = tool({
  description: `Search file contents using regex. Returns matching lines with file
  ↩ paths.
  WHEN TO USE: finding patterns across multiple files, locating function definitions,
  searching for imports, finding TODOs or error messages.
  WHEN NOT TO USE: reading a known file (use read instead).
  DO NOT USE FOR: running commands, listing directories.
  EXAMPLES:
  - Find all TODO comments: pattern "TODO" glob "*.ts"
```

```

- Find function definitions: pattern "function \\\w+" glob "*.ts",
inputSchema: z.object({
  pattern: z.string().describe("Regex pattern to search for"),
  path: z.string().optional().describe("Directory to search (default: working
↪ dir)"),
  glob: z.string().optional().describe("File glob filter, e.g. '*.ts'"),
}),
execute: async ({ pattern, path: searchPath, glob: globFilter }) => {
  const dir = resolve(cwd, searchPath || ".");
  const escapedPattern = pattern.replace(/'/g, `\\'`);
  const escapedGlob = (globFilter || "*").replace(/'/g, `\\'`);
  const cmd = `grep -rn --exclude-dir=node_modules --exclude-dir=.git
↪ --include='${escapedGlob}' -E '${escapedPattern}' '${dir}' 2>/dev/null`;

  try {
    const stdout = execSync(cmd, { encoding: "utf-8", timeout: 10_000 });
    const lines = stdout.trim().split("\n").filter(Boolean);

    const MAX_MATCHES = 50;
    const truncated = lines.length > MAX_MATCHES;
    const result = truncated ? lines.slice(0, MAX_MATCHES) : lines;

    return truncated
      ? result.join("\n") + `\\n... (${lines.length} total, showing first $
↪ {MAX_MATCHES})`
      : result.join("\n") || "No matches found.";
  } catch (error: any) {
    const stdout = String(error?.stdout || "").trim();
    if (stdout) {
      const lines = stdout.split("\n").filter(Boolean);
      const MAX_MATCHES = 50;
      const truncated = lines.length > MAX_MATCHES;
      const result = truncated ? lines.slice(0, MAX_MATCHES) : lines;
      return truncated
        ? result.join("\n") + `\\n... (${lines.length} total, showing first $
↪ {MAX_MATCHES})`
        : result.join("\n");
    }
    return "No matches found.";
  }
},
});

```

1

2

3

4

5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46

这描述中发生了两件事。WHEN TO USE 使工具的工作变得清晰可读。WHEN NOT 要 USE 并且确实 NOT USE FOR 引导模型避免默认使用它最喜欢的工具，而（根据我们的经验）通常是 `bash`。

**** 注意：巴什引力 ****

我们测试过的每个模型（Haiku、Sonnet、Opus）在工具描述薄弱时默认为 `bash`。生产线增强了负转向，每个工具都 WHEN NOT TO 和 DO 的“USE”和 DO 的 NOT USE FOR，因为说一次还不够。本课的模式源自真实的 Harness，观察 Agent 不断错误拨弦。

现在对 `read` 做同样的治疗。`read` 上的描述同样强烈反驳 `grep`：

```
const read = tool({
  description: `Read a file from the project. Returns numbered lines.
  WHEN TO USE: viewing file contents, checking configs, reading source code.
  WHEN NOT TO USE: searching across files (use grep instead).
  DO NOT USE FOR: running commands, listing directories.` ,
  // ... rest unchanged
});
```

1
2
3
4
5
6
7

为什么要有 50 场比赛上限

`grep` 和 `read` 一样，得到了同样有上下文意识的处理。没有上限，在大型代码库中搜索 `import` 会淹没数百行导入，Agent 不需要。50 匹配就足够回答这个问题了。500 是污染。

动手试试

用重写的描述运行搜索提示词：

```
bun run index.ts . "Find all TODO comments in this project"
```

1

你应该直接看到模型调用 `grep`，有像 `TODO` 这样的模式和像 `*.ts` 那样的团块。在一个小文件里放几条 `// TODO:` 评论，结果就很明显了。排除 `node_modules` 能让输出聚焦在你的代码上，而不是依赖关系。

现在运行文件检查提示词：

```
bun run index.ts . "Read the tsconfig.json"
```

1

这仍然使用 `read`，而不是 `grep`。这些描述引导模型向两个方向发展。搜索提示词向 `grep` 拉。已知档案提示词向 `read` 拉。

```
npx tsc --noEmit
```

1

**** 注意：故意让验证变得无聊 ****

真实的代码库匹配度太多，难以用眼法判断。在你控制的小文件里放两个 `// TODO:` 评论，然后运行搜索。关键是验证路由，而不是发现漏洞。让测试变得明显。

提交

```
git add index.ts
git commit -m "feat(tools): add grep with behavioral description contract"
```

1

2

完成标准

- ☐ "Find all TODO comments" 调用 `grep`，不是 `read` 也不是 `bash`
- ☐ "Read the tsconfig.json" 调用 `read`，不是 `grep`
- ☐ `grep` 最多匹配 50 场，截断后报告总数
- ☐ `read` 和 `grep` 都使用 WHEN TO USE，WHEN NOT to USE，并且确实使用 NOT USE FOR
- ☐ `npx tsc --noEmit`

**** 注意：把描述推到破碎为止 ****

开始逐节削弱 `grep` 的描述。先把 EXAMPLES 放下。那就 NOT USE FOR。然后 WHEN NOT TO USE。模型在什么时候会切换回 `bash` 或 `read`？Haiku、Sonnet 和 Opus 之间的阈值会变化吗？

参考实现

```
import { ToolLoopAgent, stepCountIs, tool } from "ai";
import { z } from "zod";
```

```

import { readFileSync } from "node:fs";
import { resolve } from "node:path";
import { execSync } from "node:child_process";

const cwd = resolve(process.argv[2] || process.cwd());

const read = tool({
  description: `Read a file from the project. Returns numbered lines.
  WHEN TO USE: viewing file contents, checking configs, reading source code.
  WHEN NOT TO USE: searching across files (use grep instead).
  DO NOT USE FOR: running commands, listing directories.`,
  inputSchema: z.object({
    path: z.string().describe("File path relative to working directory"),
    offset: z.number().optional().describe("Start line (1-indexed)"),
    limit: z.number().optional().describe("Max lines to return"),
  }),
  execute: async ({ path: filePath, offset, limit }) => {
    const abs = resolve(cwd, filePath);
    const content = readFileSync(abs, "utf-8");
    let lines = content.split("\n");

    if (offset) lines = lines.slice(offset - 1);
    if (limit) lines = lines.slice(0, limit);

    const MAX_LINES = 500;
    const truncated = lines.length > MAX_LINES;
    if (truncated) lines = lines.slice(0, MAX_LINES);

    const numbered = lines.map((l, i) => `${(offset || 1) + i}: ${l}`);
    return truncated
      ? numbered.join("\n") + `\n... (truncated at ${MAX_LINES} lines)`
      : numbered.join("\n");
  },
});

const grep = tool({
  description: `Search file contents using regex. Returns matching lines with file
  ↪ paths.
  WHEN TO USE: finding patterns across multiple files, locating function definitions,
  searching for imports, finding TODOs or error messages.
  WHEN NOT TO USE: reading a known file (use read instead).
  DO NOT USE FOR: running commands, listing directories.
  EXAMPLES:
  - Find all TODO comments: pattern "TODO" glob "*.ts"
  - Find function definitions: pattern "function \\\w+" glob "*.ts",
  inputSchema: z.object({
    pattern: z.string().describe("Regex pattern to search for"),

```

```

    path: z.string().optional().describe("Directory to search (default: working
    ↪ dir)"),
    glob: z.string().optional().describe("File glob filter, e.g. '*.ts'"),
  }),
  execute: async ({ pattern, path: searchPath, glob: globFilter }) => {
    const dir = resolve(cwd, searchPath || ".");
    const escapedPattern = pattern.replace(/'/g, `'\''`);
    const escapedGlob = (globFilter || "*").replace(/'/g, `'\''`);
    const cmd = `grep -rn --exclude-dir=node_modules --exclude-dir=.git
    ↪ --include='${escapedGlob}' -E '${escapedPattern}' '${dir}' 2>/dev/null`;

    try {
      const stdout = execSync(cmd, { encoding: "utf-8", timeout: 10_000 });
      const lines = stdout.trim().split("\n").filter(Boolean);

      const MAX_MATCHES = 50;
      const truncated = lines.length > MAX_MATCHES;
      const result = truncated ? lines.slice(0, MAX_MATCHES) : lines;

      return truncated
        ? result.join("\n") + `\\n... (${lines.length} total, showing first $
        ↪ {MAX_MATCHES})`
        : result.join("\n") || "No matches found.";
    } catch (error: any) {
      const stdout = String(error?.stdout || "").trim();
      if (stdout) {
        const lines = stdout.split("\n").filter(Boolean);
        const MAX_MATCHES = 50;
        const truncated = lines.length > MAX_MATCHES;
        const result = truncated ? lines.slice(0, MAX_MATCHES) : lines;
        return truncated
          ? result.join("\n") + `\\n... (${lines.length} total, showing first $
          ↪ {MAX_MATCHES})`
          : result.join("\n");
      }
      return "No matches found.";
    }
  },
});

const agent = new ToolLoopAgent({
  model: "anthropic/claude-haiku-4-5",
  instructions: `You are a coding agent.\nWorking directory: ${cwd}`,
  tools: { read, grep },
  stopWhen: stepCountIs(10),
});

```

```
const prompt = process.argv.slice(3).join(" ") || "Hello!";
const { text, steps } = await agent.generate({ prompt });
console.log(text);
console.log(`\n(${steps.length} steps)`);
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37

38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

补齐工具箱

模块 1 · Agent 循环 · 第 4/39 页 · 原文: <https://agentharnesszh.vercel.app/completing-the-toolbox>

补齐工具箱

你的 Agent 读取文件。它会搜索代码。接下来它会想运行一个命令。也许 `npm test`。也许是 `git status`。也许，在糟糕的日子里，会有 `rm -rf /`。

猛击是你给 Agent 最有用的工具，也是最危险的。在这节课中，我们会先添加它，然后给它戴上牵引绳。

学习成果

你有一个 `bash` 工具，在工作目录中运行命令，并被允许列表限制。安全命令会自动运行。其他的都是返回一个块消息，模型会把它传回给用户，老实说。

快速路径

1. 添加一个带有 `SAFE_PREFIXES` 允许列表（`ls`、`cat`、`git status` 和好友）的 `bash` 工具。
2. 用清晰的错误字符串屏蔽不在允许列表中的内容
3. 验证模型报告的模块，而不是虚构成功

动手练习 1.3

在 Agent 上加 `bash`，并在 `execute` 层做门控。

要求：

1. 从 `node:child_process` 进口 `execSync`
2. 定义一个 `SAFE_PREFIXES` 数组，带有只读命令，如 `ls`、`cat`、`pwd`、`git status`、`git log`、`git diff`
3. 写一个 `isSafe(command)` 检定，与允许列表进行对比
4. `execute` 中，如果命令不安全，返回阻塞消息，否则执行
5. 用四部分的描述写工具契约

实现提示：

- 按前缀匹配，不是精确命令。`ls -la` 应该和她相配 `ls`
- 在 `execSync` 上设置一个 `timeout`，这样挂机的过程不会冻结 Agent
- 阻挡消息应该准确告诉模型阻挡了什么。模型会把这些信息传递给用户
- AI SDK 有个 `needsApproval` 选项。我们这里不使用它。原因请见下文

为什么不 `needsApproval`？

AI SDK 给你一个 `needsApproval` 场 `tool()` 看起来完全符合我们的需求。事实并非如此。

```
const bash = tool({
  needsApproval: () => true,
  execute: async ({ command }) => {
    // This never runs, but the model thinks it did
  },
});
```

1
2
3
4
5
6

当 `needsApproval` 返回 `true` 时，SDK 会在响应中创建 `tool-approval-request` 并跳过执行。如果你没有接上审批处理器，工具调用就会消失。模型没有得到结果，于是自己编造了一个：“完成！我删了文件。”

模型认为命令运行了。用户会看到成功提示。命令没有运行。这比运行命令还糟糕，因为用户根本不知道出了什么问题。

****Warning：** 需要审批是一个信号，而非一个 `gate`

`needsApproval` 告诉 Harness“这需要人类的审批。”Harness 的工作就是真正采取行动。没有周围的流，卡住的工具会悄无声息地消失，模型用虚构填补空白。

我们将在 **模块 8** 中建立适当的审批流程。目前，我们在 `execute` 内部进行门控，这样模型总能获得真实的字符串。

执行层门

添加允许列表的工具：

```
import { execSync } from "node:child_process";

const SAFE_PREFIXES = [
  "ls", "cat", "echo", "pwd", "which", "find",
  "head", "tail", "wc", "git log", "git status", "git diff",
];

function isSafe(command: string): boolean {
  return SAFE_PREFIXES.some((p) => command.trim().startsWith(p));
}

const bash = tool({
  description: `Execute a shell command in the working directory.
  WHEN TO USE: running build commands, installing packages, running tests,
  git operations, directory listings.
```

```
WHEN NOT TO USE: reading file contents (use read instead).
  Searching for patterns (use grep instead).
DO NOT USE FOR: reading files (use read), searching code (use grep).`,
inputSchema: z.object({
  command: z.string().describe("Shell command to execute"),
}),
execute: async ({ command }) => {
  if (!isSafe(command)) {
    return `Blocked: "${command}" requires approval. Only safe commands ($
↪ {SAFE_PREFIXES.join(", ")}) run automatically.`;
  }
  try {
    const stdout = execSync(command, {
      cwd,
      encoding: "utf-8",
      timeout: 30_000,
    });
    return stdout || "(no output)";
  } catch (e: any) {
    return `Exit ${e.status ?? 1}: ${e.stdout || e.stderr || e.message || ""}`;
  }
},
});
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20

21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37

关键的模式是：当命令被阻塞时，工具会返回字符串。这条线最终作为工具结果出现在对话中。模型会像读取其他结果一样读取，并可以将块真实地传回给用户。

动手试试

运行一个映射到安全命令的提示词：

```
bun run index.ts . "List all files in this directory"
```

1

模型应该按 `bash`，选择安全命令，比如 `ls` 或 `find`，然后返回输出。

现在试试危险的办法：

```
bun run index.ts . "Run the command: rm -rf node_modules"
```

1

模型调用 `bash`rm -rf`。大门挡住了它。块状消息以工具结果的形式返回，模型会将其传递给你：

```
The command "rm -rf node_modules" requires approval.  
Only safe commands run automatically.
```

1

2

这才是重点。没有无声的失败。没有虚构的成功。

****Warning : 关注创意 rewrites****

如果你说“删除 `node_modules`”，模型可能会尝试 `find . -name node_modules -exec rm -rf {}` + 而不是 `rm -rf`。我们的前缀检查抓住了 `rm` 但 `find -exec` 漏掉了。生产 Harness 使用正则表达式模式来表示危险命令。我们保留前缀检查是因为它清晰，而不是因为它完整。

```
npx tsc --noEmit
```

1

三利一 Agent

你现在拥有了这三种工具：

工具	作用	安全
<code>read</code>	查看文件内容	500 行电容
<code>grep</code>	跨文件搜索	50 场比赛上限
<code>bash</code>	运行 shell 命令	<code>SAFE_PREFIXES</code> 允许名单

描述引导着选择。这些大写保护上下文。门保护你的机器。Agent 是有用且可控的。

提交

```
git add index.ts
git commit -m "feat(tools): add bash with execute-level safety gate"
```

1

2

完成标准

- ☐ 安全命令如 `ls`、`find` 和 `git status`bash`
- ☐ 自然语言文件读取提示词仍然会路由到 `read`，而不是 `bash`cat`
- ☐ `rm -rf`、`sudo` 及其他未知命令返回阻断消息
- ☐ 模型报告显示，用户没有假装成功，而是阻止了指令
- ☐ `npx tsc --noEmit`

**** 注意：勾勒审批流程 ****

执行层门诚实但直白。真正的 Harness 会问使用者。试着添加一个 `needsApproval` 检查，返回 `true` 非安全命令，暂停循环，并向用户展示一个类似 `“Agent 想运行 npm install express 这样的提示词。允许吗？”` 审批简历与工具结果。否认继续，并用否认信息让模型适应。你会在模块 8 中正确构建，但现在值得先草拟一下，看看为什么当 Agent 运行 50 个命令时，交互审批会变得复杂。

参考实现

```
import { ToolLoopAgent, stepCountIs, tool } from "ai";
import { z } from "zod";
import { readFileSync } from "node:fs";
import { resolve } from "node:path";
import { execSync } from "node:child_process";

const cwd = resolve(process.argv[2] || process.cwd());

const SAFE_PREFIXES = [
  "ls", "cat", "echo", "pwd", "which", "find",
  "head", "tail", "wc", "git log", "git status", "git diff",
];

function isSafe(command: string): boolean {
  return SAFE_PREFIXES.some((p) => command.trim().startsWith(p));
}

const read = tool({
  description: `Read a file from the project. Returns numbered lines.
  WHEN TO USE: viewing file contents, checking configs, reading source code.
  WHEN NOT TO USE: searching across files (use grep instead).
  DO NOT USE FOR: running commands, listing directories.`,
  inputSchema: z.object({
    path: z.string().describe("File path relative to working directory"),
    offset: z.number().optional().describe("Start line (1-indexed)"),
    limit: z.number().optional().describe("Max lines to return"),
  }),
  execute: async ({ path: filePath, offset, limit }) => {
    const abs = resolve(cwd, filePath);
    const content = readFileSync(abs, "utf-8");
    let lines = content.split("\n");

    if (offset) lines = lines.slice(offset - 1);
    if (limit) lines = lines.slice(0, limit);

    const MAX_LINES = 500;
    const truncated = lines.length > MAX_LINES;
```

```

    if (truncated) lines = lines.slice(0, MAX_LINES);

    const numbered = lines.map((l, i) => `${(offset || 1) + i}: ${l}`);
    return truncated
      ? numbered.join("\n") + `\n... (truncated at ${MAX_LINES} lines)`
      : numbered.join("\n");
  },
});

const grep = tool({
  description: `Search file contents using regex. Returns matching lines with file
  ↪ paths.
  WHEN TO USE: finding patterns across multiple files, locating function definitions,
    searching for imports, finding TODOs or error messages.
  WHEN NOT TO USE: reading a known file (use read instead).
  DO NOT USE FOR: running commands, listing directories.
  EXAMPLES:
    - Find all TODO comments: pattern "TODO" glob "*.ts"
    - Find function definitions: pattern "function \\\w+" glob "*.ts",
    inputSchema: z.object({
      pattern: z.string().describe("Regex pattern to search for"),
      path: z.string().optional().describe("Directory to search (default: working
  ↪ dir)"),
      glob: z.string().optional().describe("File glob filter, e.g. '*.ts'"),
    }),
  execute: async ({ pattern, path: searchPath, glob: globFilter }) => {
    const dir = resolve(cwd, searchPath || ".");
    const escapedPattern = pattern.replace(/'/g, `'\''`);
    const escapedGlob = (globFilter || "*").replace(/'/g, `'\''`);
    const cmd = `grep -rn --exclude-dir=node_modules --exclude-dir=.git
  ↪ --include='${escapedGlob}' -E '${escapedPattern}' '${dir}' 2>/dev/null`;

    try {
      const stdout = execSync(cmd, { encoding: "utf-8", timeout: 10_000 });
      const lines = stdout.trim().split("\n").filter(Boolean);

      const MAX_MATCHES = 50;
      const truncated = lines.length > MAX_MATCHES;
      const result = truncated ? lines.slice(0, MAX_MATCHES) : lines;

      return truncated
        ? result.join("\n") + `\n... (${lines.length} total, showing first $
  ↪ {MAX_MATCHES})`
        : result.join("\n") || "No matches found.";
    } catch (error: any) {
      const stdout = String(error?.stdout || "").trim();
      if (stdout) {

```

```

    const lines = stdout.split("\n").filter(Boolean);
    const MAX_MATCHES = 50;
    const truncated = lines.length > MAX_MATCHES;
    const result = truncated ? lines.slice(0, MAX_MATCHES) : lines;
    return truncated
      ? result.join("\n") + `\\n... (${lines.length} total, showing first $
↪ {MAX_MATCHES})`
      : result.join("\n");
  }
  return "No matches found.";
}
},
});

const bash = tool({
  description: `Execute a shell command in the working directory.
WHEN TO USE: running build commands, installing packages, running tests,
git operations, directory listings.
WHEN NOT TO USE: reading file contents (use read instead).
Searching for patterns (use grep instead).
DO NOT USE FOR: reading files (use read), searching code (use grep).`,
  inputSchema: z.object({
    command: z.string().describe("Shell command to execute"),
  }),
  execute: async ({ command }) => {
    if (!isSafe(command)) {
      return `Blocked: "${command}" requires approval. Only safe commands ($
↪ {SAFE_PREFIXES.join(", ")}) run automatically.`;
    }
    try {
      const stdout = execSync(command, {
        cwd,
        encoding: "utf-8",
        timeout: 30_000,
      });
      return stdout || "(no output)";
    } catch (e: any) {
      return `Exit ${e.status ?? 1}: ${e.stdout || e.stderr || e.message || ""}`;
    }
  },
});

const agent = new ToolLoopAgent({
  model: "anthropic/claude-haiku-4-5",
  instructions: `You are a coding agent.\nWorking directory: ${cwd}`,
  tools: { read, grep, bash },
  stopWhen: stepCountIs(10),
});

```

```
});  
  
const prompt = process.argv.slice(3).join(" ") || "Hello!";  
const { text, steps } = await agent.generate({ prompt });  
console.log(text);  
console.log(`\n(${steps.length} steps)`);
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35

36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77

78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119

120

121

122

123

124

125

126

127

128

129

130

131

有效的工具描述

模块 2 · 工具设计 · 第 5/39 页 · 原文: <https://agentharnesszh.vercel.app/descriptions-that-work>

有效的工具描述

你在《Agent 循环》中写第一篇工具描述时，用了两个部分：WHEN TO USE 和 WHEN NOT TO USE。这足以让路由用三个工具运行起来。

再加一个工具。再加五分之一。添加子 Agent、编辑、写作和待办事项。模型又开始模糊了。它会为 read 应该句柄的内容挑选 bash，或者跳过 grep，进入一个探索性的 read 循环，打开二十个文件。

修复方法和之前一样。我们只是需要更多。

学习成果

你 Harness 中的每个工具都有五个部分的描述契约（WHEN 到 USE，WHEN NOT 到 USE，DO NOT USE FOR，USAGE，EXAMPLES），模型会正确地穿越模糊提示词。

快速路径

1. 将每个工具的描述扩展为 5 个部分
2. 添加 USAGE 用于参数指导，添加 EXAMPLES 用于具体调用
3. 保留双重负片（WHEN NOT TO USE 并做 NOT USE FOR），因为没有它，每个模型都会漏回去 bash

动手练习 2.1

在 `index.ts` 重构每个工具的描述，以充分利用完整契约。

要求：

1. 每个描述都以一行简要的工具功能和输出格式的总结开头
2. WHEN USE 列出 2-4 个具体场景，使用关键词，模型将在提示词中看到
3. WHEN NOT TO USE 按名称重定向到其他工具
4. DO NOT USE FOR 会重复负面引导作为硬边界
5. USAGE 解释了参数约束和默认值
6. EXAMPLES 显示了 2-3 个具体调用，输入

实现提示：

- WHEN NOT TO USE 是软的（“prefer X”）。DO NOT USE FOR 很难（“永远不要用这个做 Y”）。你想要两者都想要
- 当描述薄弱时，模型默认会对所有内容 bash。重复的负方是反力
- USAGE 在参数具有模型无法从 schema 推断的约束（如大写、默认值、编码）时才值得使用。

为什么要用双重负片

你可能会把 WHEN NOT TO USE 和 DO NOT USE FOR 并排对比，觉得它们说的是重复的。他们确实如此。这正是重点。

在我们的测试中：

- Haiku 读 WHEN NOT 是“TO USE”，但在歧义下忽略了它
- Sonnet 会遵守 WHEN NOT TO USE，但加入 DO NOT USE FOR 的强化约束后效果更好
- Opus 句柄都很好，重复也无害

我们测试过的每个模型都倾向于描述薄弱时 **bash**。我们调用这种撞击引力，那种普遍的吸引力，指向最通用的工具。说一次“不要用这个搜索”并不总是足够。说两遍几乎总是有问题。

完整契约

以下是每个部分在 **grep** 上的应用：

```
const grep = tool({
  description: `Search file contents using regex. Returns matching lines with file
  ↩ paths.
```

WHEN TO USE: finding patterns across multiple files, locating function definitions, searching for imports, finding TODOs or error messages.

WHEN NOT TO USE: reading a known file (use read instead).
Running commands (use bash instead).

DO NOT USE FOR: reading files (use read), listing directories (use bash),
modifying files (use edit).

USAGE: pattern is a regex string. glob filters by file extension.
Results are capped at 50 matches.

EXAMPLES:

- Find all TODO comments: pattern "TODO" glob "*.ts"
- Find function definitions: pattern "function \\w+" glob "*.ts"
- Find imports of a package: pattern "from 'express'" glob "*.ts`",
// ... inputSchema and execute unchanged

```
});
```

1
2
3
4
5
6
7

8
9
10
11
12
13
14
15
16
17
18
19
20
21

以下是 `read` 的同样处理：

```
const read = tool({
  description: `Read a file from the project. Returns numbered lines.

  WHEN TO USE: viewing file contents, checking configurations, reading source code,
  examining specific lines with offset/limit.

  WHEN NOT TO USE: searching for patterns across files (use grep instead).
  Running commands (use bash instead).

  DO NOT USE FOR: searching code (use grep), executing commands (use bash),
  modifying files (use edit or write).

  USAGE: path is relative to working directory. offset and limit are optional.
  Output is capped at 500 lines.`
  // ... rest unchanged
});
```

1
2
3
4
5
6
7
8
9
10

11
12
13
14
15
16

bash:

```
const bash = tool({
  description: `Execute a shell command in the working directory.

  WHEN TO USE: running build commands, installing packages, running tests,
  git operations, directory listings.

  WHEN NOT TO USE: reading file contents (use read instead).
  Searching for patterns (use grep instead).

  DO NOT USE FOR: reading files (use read), searching code (use grep).

  USAGE: command is a single shell string. Commands not in the safe-prefix
  allowlist are blocked and return a clear error message.

  EXAMPLES:
  - List files: command "ls -la"
  - Check git status: command "git status"
  - Run a test suite: command "npm test"`,
  // ... rest unchanged
});
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14

15
16
17
18
19
20

为什么每个部分都值得被认可

部分	作用
首行	工具的功能，以及它返回的
WHEN 到 USE	提示词会用到的具体场景和关键词
WHEN NOT 到 USE	柔性引导到正确工具
一定要 NOT USE FOR	再次强调的硬边界
USAGE	schema 无法捕捉的约束（大写、默认值、编码）
EXAMPLES	模型模式匹配的具体调用

描述会越来越长。没关系。工具描述存放在系统提示词中， SDK 在回合间缓存。你已经付了一次 token 。

动手试试

每个工具形状运行一个提示词，并验证路由：

```
bun run index.ts . "Find all TODO comments in this project"
bun run index.ts . "Read the package.json"
bun run index.ts . "List all files in this directory"
```

1
2
3

你应该看看：

- TODO 提示词调用 `grep`
- package.json 提示词调用 `read`
- 列表文件 `ls` 提示词调用 `bash`

```
npx tsc --noEmit
```

1

**** 注意：每个形状用一个提示词来验证 ****

选一个提示词能恰好指向一个工具。搜索形状表示 `grep`，文件形状表示 `read`，壳形状表示 `bash`。混合提示词（比如“显示我 `package.json` 内容”）有时会用 `cat` 路由到 `read` 或 `bash`，这不是路由 bug，而是模糊的提示词。

提交

```
git add index.ts
git commit -m "feat(tools): expand descriptions to full 5-section contract"
```

1

2

完成标准

- ☐ 这三种工具都有包含全部 5 个部分的描述
- ☐ TODO 搜索提示词 `grep`
- ☐ 文件读取后提示词路由到 `read`
- ☐ 壳牌列表提示词 `bash` 的路由
- ☐ `npx tsc --noEmit`

**** 注意：找到最薄弱的环节 ****

选你写的描述开始脱衣。放开 `EXAMPLES`。NOT USE FOR 掉落 DO。放下 `USAGE`。每试一次后，进行三次测试提示词。路由在什么时候会中断？它最先断在哪里？第一个被翻转的模型就是你找到的那个地板。

参考实现

```
const read = tool({
  description: `Read a file from the project. Returns numbered lines.

  WHEN TO USE: viewing file contents, checking configurations, reading source code,
  examining specific lines with offset/limit.

  WHEN NOT TO USE: searching for patterns across files (use grep instead).
  Running commands (use bash instead).

  DO NOT USE FOR: searching code (use grep), executing commands (use bash),
  modifying files (use edit or write).

  USAGE: path is relative to working directory. offset and limit are optional.
  Output is capped at 500 lines.`
})
// ... inputSchema and execute from Module 1
```

```
});

const grep = tool({
  description: `Search file contents using regex. Returns matching lines with file
↪ paths.

WHEN TO USE: finding patterns across multiple files, locating function definitions,
searching for imports, finding TODOs or error messages.

WHEN NOT TO USE: reading a known file (use read instead).
Running commands (use bash instead).

DO NOT USE FOR: reading files (use read), listing directories (use bash),
modifying files (use edit).

USAGE: pattern is a regex string. glob filters by file extension.
Results are capped at 50 matches.

EXAMPLES:
- Find all TODO comments: pattern "TODO" glob "*.ts"
- Find function definitions: pattern "function \\w+" glob "*.ts"
- Find imports of a package: pattern "from 'express'" glob "*.ts`,
// ... inputSchema and execute from Module 1
});

const bash = tool({
  description: `Execute a shell command in the working directory.

WHEN TO USE: running build commands, installing packages, running tests,
git operations, directory listings.

WHEN NOT TO USE: reading file contents (use read instead).
Searching for patterns (use grep instead).

DO NOT USE FOR: reading files (use read), searching code (use grep).

USAGE: command is a single shell string. Commands not in the safe-prefix
allowlist are blocked and return a clear error message.

EXAMPLES:
- List files: command "ls -la"
- Check git status: command "git status"
- Run a test suite: command "npm test`,
// ... inputSchema and execute from Module 1
});
```

2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

安全执行 Shell

模块 2 · 工具设计 · 第 6/39 页 · 原文: <https://agentharnesszh.vercel.app/shell-execution-with-safety>

安全执行 Shell

你的 `bash` 工具能用，但描述、安全检查和 `execSync` 调用都放在一个大闭包里。只要只有一个 `bash` 工具，这没问题。一旦你想在这台机器以外的地方运行命令，它就不再正常了。

再过几个模块，我们会用沙箱替换本地执行。遇到这种情况时，你不想重写 `bash` 工具。你想给它换个后端，其他部分保持不动。

这就是工厂模式的用途。

学习成果

你有一个 `createBashTool(operations, safePrefixes)` 工厂，会返回一个完全配置好的 `bash` 工具。面向模型的契约（描述、`schema`、安全检查）则存放在工厂内。执行后端通过 `operations` 对象注入。

快速路径

1. 用一种方法定义一个 `BashOperations` 接口，`exec(command)`
2. 把现有的 `bash` 工具包裹起来 `createBashTool(operations, safePrefixes)`
3. 构建一个包裹 `execSync` 的 `localOps` 对象，然后用它构建工具

动手练习 2.2

把 `bash` 导出到带有可交换执行后端的工厂函数中。

要求：

1. 用 `exec(command: string): Promise<{ stdout: string; exitCode: number }>` 定义 `BashOperations`
2. 写入返回 `tool()` 的 `createBashTool(operations: BashOperations, safePrefixes: string[])`
3. 安全检查在工厂内进行，使用注入 `safePrefixes`
4. 构建一个封装 `localOps` 实现 `execSync`
5. 用 `const bash = createBashTool(localOps, SAFE_PREFIXES)` 替换你现有的 `bash` 常数

实现提示：

- 工厂在 `operations` 和 `safePrefixes` 期间关闭。`execute` 功能在工具内部调用 `operations.exec(command)`，而不是直接 `execSync`
- 本地操作员（`localops`）句柄 `stdout` 和错误都一致。无论命令成功与否，返回 `{ stdout, exitCode }`

- 现在不要重构 `read`。工厂模式确实值得，后端会有所变化，目前这只是一个 `bash`

接缝的走向

现在你的 `bash` 工具直接调用 `execSync`：

```
execute: async ({ command }) => {  
  if (!isSafe(command)) return "Blocked...";  
  const stdout = execSync(command, { cwd, encoding: "utf-8", timeout: 30_000 });  
  return stdout;  
}
```

1
2
3
4
5

当你在后面加上一个沙箱时，这就变成了 `sandbox.exec(command)`。想法一样，但后端不同。工厂在这两者之间引入了一条接缝：

```
interface BashOperations {  
  exec(command: string): Promise<{ stdout: string; exitCode: number }>;  
}
```

1
2
3

模型看到的所有东西都存在于接缝之上。所有实际执行命令的设备都存放在下面。

建工厂

```
function createBashTool(operations: BashOperations, safePrefixes: string[]) {  
  function isSafe(command: string): boolean {  
    return safePrefixes.some((p) => command.trim().startsWith(p));  
  }  
  
  return tool({  
    description: `Execute a shell command in the working directory.  
  
    WHEN TO USE: running build commands, installing packages, running tests,  
    git operations, directory listings.  
  
    WHEN NOT TO USE: reading file contents (use read instead).  
    Searching for patterns (use grep instead).`  
  });  
}
```

DO NOT USE FOR: reading files (use read), searching code (use grep).

USAGE: command is a single shell string. Commands not in the safe-prefix allowlist are blocked and return a clear error message.`,

```
inputSchema: z.object({
  command: z.string().describe("Shell command to execute"),
}),
execute: async ({ command }) => {
  if (!isSafe(command)) {
    return `Blocked: "${command}" requires approval.`;
  }
  const { stdout } = await operations.exec(command);
  return stdout || "(no output)";
},
});
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25

26
27
28
29
30

注意消失了什么：没有 `execSync`，没有 `cwd` 引用，没有知道 Node `child_process` 的错误处理。工厂只知道有个叫 `operations.exec` 的东西，然后退 `stdout`。

构建本地后端

`localOps` 对象是实际 `execSync` 调用现在居住的地方：

```
const localOps: BashOperations = {
  exec: async (command) => {
    try {
      const stdout = execSync(command, {
        cwd,
        encoding: "utf-8",
        timeout: 30_000,
      });
      return { stdout, exitCode: 0 };
    } catch (e: any) {
      return {
        stdout: e.stdout || e.stderr || e.message || "",
        exitCode: e.status ?? 1,
      };
    }
  },
};

const bash = createBashTool(localOps, SAFE_PREFIXES);
```

1
2
3
4
5
6
7
8
9
10
11
12

13
14
15
16
17
18
19

当你在模块 4 中构建沙箱抽象时，交换是一行：

```
const sandboxOps: BashOperations = {  
  exec: (command) => sandbox.exec(command),  
};  
  
const bash = createBashTool(sandboxOps, SAFE_PREFIXES);
```

1
2
3
4
5

同样的工具。不同的后端。描述、`schema` 和安全检查都没有变化。

**** 注意：为什么只抨击，不读 ****

你可以把同样的工厂模式套用到 `read` 上。我们还没开始。工厂的价值在于后台真正多样化。对 `read` 来说，这要到第四模块才会发生。`bash`，执行后端和安全策略已经在反方向拉扯。有压力时重构，而不是提前。

动手试试

运行一个安全命令，确保出厂时所有布线都正确：

```
bun run index.ts . "List all files in this directory"
```

1

输出和之前一样。同样的阻塞命令行为。模型无法察觉任何变化，这正是关键所在。

试试一个 `blocked` 命令，确保安全检查仍然有效：

```
bun run index.ts . "Run: rm -rf node_modules"
```

1

你仍然应该会收到屏蔽通知。

```
npx tsc --noEmit
```

1

提交

```
git add index.ts
git commit -m "refactor(bash): extract createBashTool with operations interface"
```

1

2

完成标准

- ☐ `BashOperations` 接口定义于 `exec(command)`
- ☐ `createBashTool(operations, safePrefixes)` 还用工具
- ☐ `localOps`execSync` 包好, 返回 `{ stdout, exitCode }`
- ☐ 安全命令依然运行, 被阻挡命令依然返回阻挡消息
- ☐ `npx tsc --noEmit`

**** 注意: 画一下沙箱交换 ****

还没搭建, 先写一个不运行任何东西的 `mockOps: BashOperations`。只要 `{ stdout: "(pretend output)", exitCode: 0 }` 任何指令都回去。换 `localOps` 换成 `mockOps`, 看看 Agent 所有内容都显示合理但假的输出。这是让模块 4 中的沙箱抽象在不重写工具的情况下发挥作用的接缝。

参考实现

```
interface BashOperations {
  exec(command: string): Promise<{ stdout: string; exitCode: number }>;
}

function createBashTool(operations: BashOperations, safePrefixes: string[]) {
  function isSafe(command: string): boolean {
    return safePrefixes.some((p) => command.trim().startsWith(p));
  }

  return tool({
```

description: `Execute a shell command in the working directory.

WHEN TO USE: running build commands, installing packages, running tests, git operations, directory listings.

WHEN NOT TO USE: reading file contents (use read instead).
Searching for patterns (use grep instead).

DO NOT USE FOR: reading files (use read), searching code (use grep).

USAGE: command is a single shell string. Commands not in the safe-prefix allowlist are blocked and return a clear error message.`,

```
inputSchema: z.object({
  command: z.string().describe("Shell command to execute"),
}),
execute: async ({ command }) => {
  if (!isSafe(command)) {
    return `Blocked: "${command}" requires approval.`;
  }
  const { stdout } = await operations.exec(command);
  return stdout || "(no output)";
},
});
}
```

```
const localOps: BashOperations = {
  exec: async (command) => {
    try {
      const stdout = execSync(command, {
        cwd,
        encoding: "utf-8",
        timeout: 30_000,
      });
      return { stdout, exitCode: 0 };
    } catch (e: any) {
      return {
        stdout: e.stdout || e.stderr || e.message || "",
        exitCode: e.status ?? 1,
      };
    }
  },
};
```

```
const bash = createBashTool(localOps, SAFE_PREFIXES);
```

1

2

3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44

45
46
47
48
49
50
51
52
53
54

审批门

模块 2 · 工具设计 · 第 7/39 页 · 原文: <https://agentharnesszh.vercel.app/approval-gates>

审批门

你在模块 1 中建立的允许列表只有一种模式：屏蔽列表中所有不在列表中的内容。这对演示来说没问题。这对真正的 Harness 来说不合适。

线人没有人可以问。子 Agent 需要继承父 Agent 的部分信任，而不是全部的授权清单。而本地 Agent 负责人可能只想审批一次 `npm install express`，三步后又被问一次。

同一个门。三种不同的操作模式。我们将通过将配置本身的形状从布尔数演进到函数，再到可辨识联合类型来实现。

学习成果

`createBashTool` 接受三种模式的 `ApprovalConfig` 可辨识联合类型：`interactive`、`background` 和 `delegated`。每种模式在 `needsApproval` 返回为真时都会形成形状。

快速路径

1. 定义 `ApprovalConfig` 为具有三种模态的可辨识联合类型
2. 写入返回 `needsApproval` 函数的 `createApproval(config)`
3. 将结果传递给 `createBashTool`，验证每种模式的表现不同

动手练习 2.3

用可配置的审批系统取代静态的安全前缀检查。

要求：

1. 定义 `ApprovalConfig` 有三个变体：`{ mode: "interactive" }`、`{ mode: "background" }`、`{ mode: "delegated"; trust: string[] }`
2. 写入返回 `(input) => boolean` 的 `createApproval(config: ApprovalConfig)`
3. 更新 `createBashTool` 以接受审批函数作为参数
4. 用同一个命令测试每个模式，并验证门的行为变化

实现提示：

- `background` 所有事情都会 `false`（审批）退货。这是针对 CI 和自动运行的
- `delegated` 会对照输入 `config.trust` 并只审批匹配
- `interactive` 会将输入与保险前缀列表对照，并审批安全前缀列表。其他任何事都需要人类的审批
- 函数返回时 `true` 需要审批时，`false` 命令可以运行时返回

第一阶段：布尔值

最简单的审批门是布尔值：

```
needsApproval: true
```

1

这会阻挡所有调用。`ls`、`pwd`、`rm -rf`，全部都没了。虽然没用，但能定下形状。`needsApproval` 问题是“我们是否应该在运行前暂停等待人类的认可？”

第二阶段：功能

函数可以让你根据输入回答这个问题：

```
needsApproval: ({ command }) => {  
  if (SAFE_PREFIXES.some(p => command.startsWith(p))) return false;  
  return true;  
}
```

1

2

3

4

好多了。`ls` 跑。`rm -rf` 块。但函数只知道一条内置规则。CI 和本地终端有相同的登机口。子 Agent 和它的父 Agent 一样拥有相同的门。你无法在不重写函数的情况下重新配置。

第三阶段：可辨识联合类型

配置里承载了模式。工厂根据以下模式构建了该功能：

```
type ApprovalConfig =  
  | { mode: "interactive" }  
  | { mode: "background" }  
  | { mode: "delegated"; trust: string[] };  
  
function createApproval(config: ApprovalConfig) {  
  return ({ command }: { command: string }) => {  
    if (config.mode === "background") return false;  
  
    if (config.mode === "delegated") {  
      return !config.trust.some((p) => command.trim().startsWith(p));  
    }  
  
    return !SAFE_PREFIXES.some((p) => command.trim().startsWith(p));  
  };  
};
```

```
}

```

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

```

三种模式，一个功能。可辨识联合类型让模式变成类型化且排他。TypeScript 将 `config.trust` 缩小到只 `string[]`delegated` 分支内部，而这正是布尔和函数无法捕捉的错误。

现在 `createBashTool` 采用了审批函数，而不是安全前缀列表：

```
function createBashTool(
  operations: BashOperations,
  needsApproval: (input: { command: string }) => boolean,
) {
  return tool({
    // ... same description and schema
    execute: async ({ command }) => {
      if (needsApproval({ command })) {
        return `Blocked: "${command}" requires approval.`;
      }
      const { stdout } = await operations.exec(command);
      return stdout || "(no output)";
    },
  });
}
```

```
1
2
3
4

```

5
6
7
8
9
10
11
12
13
14
15

调用网站上的三种模式是这样的：

```
// Interactive: human approves anything not on the safe list
const bash = createBashTool(localOps, createApproval({ mode: "interactive" }));

// Background: auto-approve everything (CI, automation)
const bash = createBashTool(localOps, createApproval({ mode: "background" }));

// Delegated: subagent inherits a trust slice from its parent
const bash = createBashTool(
  localOps,
  createApproval({ mode: "delegated", trust: ["pwd", "find .", "git status"] }),
);
```

1
2
3
4
5
6
7
8
9
10
11

**** 注意：为什么是可辨识联合类型而不是三个函数 ****

你完全可以写三个独立的函数：[interactiveApproval](#)、[backgroundApproval](#)、[delegatedApproval](#)。可辨识联合类型之所以胜出，是因为配置是数据，而不是代码。你可以从[AGENTS.md](#)加载，用 Zod ([z.discriminatedUnion\("mode", \[...\]\)](#)) 验证，跨子 Agent 边界序列化，允许用户在不触碰

Harness 代码的情况下切换模式。

这会解锁什么

background 是显而易见的。Agent 在 CI 里运行，没有人可以问，你信任提示词，愿意放手。

delegated 才是有趣的。当子 Agent 启动时，你不会给它完整的保险前缀列表。你把它需要的特定命令交给它。只读探索器的表现 **pwd**、**find**、**git status**。一个执行器进行测试，结果是有点 **npm test**，**npm run build**。父 Agent 以命令决定将哪些信托委托。

interactive 你已经在做的，只是现在用配置来表达。

动手试试

试试每个模式，用一个应该阻挡的命令和一个应该通过的命令：

```
bun run index.ts . "Run: git status"
```

1

在 **interactive** 模式下，这个问题会通过（它在安全名单上）。在 **background** 模式下，什么都过去了。在 **trust: ["git status"]`delegated** 模式下，它会通过。

```
bun run index.ts . "Run: npm install express"
```

1

在 **interactive** 模式下，这个会被阻挡。在 **background** 模式下，它能运行（可能失败是因为我们没有接线 NPM，但那是另一个问题）。在 **delegated** 模式下，除非 **npm install** 在信任列表中，否则会被屏蔽。

```
npx tsc --noEmit
```

1

**** 注意：审批结果与命令结果 ****

命令可以被审批，但因普通原因仍然失败。**npm test** 通过门，然后因为测试失败而非零退出。那是指挥部的问题，不是审批的问题。调试时保持它们分开。

提交

```
git add index.ts
git commit -m "feat(approval): add discriminated union config with three modes"
```

1

2

完成标准

- ☐ `ApprovalConfig` 是可辨识联合类型, `interactive`、`background`、`delegated`
- ☐ `createApproval(config)` 返回一个 `needsApproval` 函数
- ☐ `createBashTool` 接受审批函数作为参数
- ☐ 每种模式至少在一个安全命令和一个不安全命令下表现正确
- ☐ `npx tsc --noEmit`

** 注意：会话级信任升级 **

交互模式每次都会自动拒绝所有未知指令。这很快就会让人厌烦。试着添加一个跟踪用户在会话中审批的模式的 `Set<string>`。用户审批 `npm test` 后，添加模式。接下来的 `npm test` 调用跳过了提示词。添加一个 `trust --list` 命令来显示哪些是可信的。现在考虑细节：`npm install` 应该信任所有信息，还是只 `npm install express` 完全信任？

参考实现

```
type ApprovalConfig =
  | { mode: "interactive" }
  | { mode: "background" }
  | { mode: "delegated"; trust: string[] };

function createApproval(config: ApprovalConfig) {
  return ({ command }: { command: string }) => {
    if (config.mode === "background") return false;

    if (config.mode === "delegated") {
      return !config.trust.some((p) => command.trim().startsWith(p));
    }

    return !SAFE_PREFIXES.some((p) => command.trim().startsWith(p));
  };
}

function createBashTool(
  operations: BashOperations,
  needsApproval: (input: { command: string }) => boolean,
) {
  return tool({
    description: `Execute a shell command in the working directory.

    WHEN TO USE: running build commands, installing packages, running tests,
```

git operations, directory listings.

WHEN NOT TO USE: reading file contents (use read instead).

Searching for patterns (use grep instead).

DO NOT USE FOR: reading files (use read), searching code (use grep).

USAGE: command is a single shell string. Commands not approved by the approval policy are blocked and return a clear error message.`,

```
inputSchema: z.object({
  command: z.string().describe("Shell command to execute"),
}),
execute: async ({ command }) => {
  if (needsApproval({ command })) {
    return `Blocked: "${command}" requires approval.`;
  }
  const { stdout } = await operations.exec(command);
  return stdout || "(no output)";
},
});
}

const bash = createBashTool(localOps, createApproval({ mode: "interactive" }));
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20

21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48

组织 Agent 指令

模块 3 · 系统提示词 · 第 8/39 页 · 原文: <https://agentharnesszh.vercel.app/structuring-agent-instructions>

组织 Agent 指令

现在你的系统提示词只有一句话: `You are a coding agent.` 那不是提示词。那是名牌。

你的工具已经在起大部分转向作用了。模块 2 的描述足够丰富, Agent 可能会选择阅读任务 `read`, 搜索时会 `grep`, 几乎不用帮忙。那么, 这个系统提示词是为了什么?

就是你写下策略的地方。而不是 *Agent can* 做什么 (那是工具)。 *Agent should* 做什么, 按什么顺序, 用什么样的克制。名牌不能带这个。一个被分割的遗嘱提示词。

学习成果

`instructions` 是一个分段的提示词, 明确设有 Agent 区和护栏区块。Agent 对任务进行操作 (使用工具、报告结果), 而不是解释假设性的功能。

快速路径

1. 用切段的 `You are a coding agent.` 替换提示词
2. 添加 Agent 部门: 使用工具, 不解释, 偏好专用工具胜过 `bash`
3. 添加护栏部分: 改动最小, 创建前搜索, 未经允许新增 `Deps`

动手练习 3.1

在 `index.ts` 中重写 `instructions` 字段, 使用结构化的多部分提示词。

要求:

1. 以一句话介绍角色和工作目录
2. 添加一个 `# Agency` 部分, 告诉 Agent 采取行动, 而不是解释
3. 添加一个 `# Guardrails` 部分, 包含约束 (最小改动、重复使用、不新增 `DEPS`)
4. 继续使用模板文字, 这样工作目录就能插值进去

实现提示:

- 分段标题 (`# Agency`、`# Guardrails`) 让提示词对你和模型都能扫描
- 明确地表达负面指示 (“NOT 解释一下你会怎么做”)。模型默认是解释
- 请用工具名称来称呼。模型已经有描述, 但提示词中指向它们会强化路线

之前

```
instructions: `You are a coding agent.\nWorking directory: ${cwd}`,
```

1

这方法有效。Agent 在被要求时读取文件，在被要求时搜索，在被要求时执行命令。但没有什么策略。让它“找到 TODO 所有评论并修正”，回复完全取决于模型当下的心情。也许它会起作用。也许它会生成一个计划，等待你审批。

这种模糊正是提示词存在的意义。

之后

```
instructions: `You are a coding agent working in: ${cwd}

# Agency
- USE your tools. Read files, search code, run commands, then answer.
- Do NOT explain what you WOULD do. Actually do it.
- Prefer grep for searching, read for viewing files.
- Use bash only for commands that aren't covered by other tools.

# Guardrails
- Prefer simple, minimal changes
- Search before creating, and reuse existing patterns
- No new dependencies without asking`,
```

1

2

3

4

5

6

7

8

9

10

11

12

现在策略已经写下来了。Agent 部门给予 Agent 行动许可和指示。护栏部分设定了界限。

有几点需要注意：

- “完成任务”的指令是明确的。没有它，Agent 就会陷入解释。大声说“真的去做”是令人烦躁的必要
- 工具偏好和描述都存在于提示词中。重复是有意为之。我们在两个地方说，因为模型们在一个地方错过了
- “未经询问不得新增依赖”为第 8 模块的人机协作工作奠定了基础

具体发生了什么变化

Agent 已经有工具了。它已经有描述了。新的是操作策略现在可携带。你可以把这个提示词复制到不同的 Agent，用不同的工具，策略依然适用。你可以做 A/B 测试。你可以把一部分抬起来做子 Agent。

一句话的话提示词做不到。一个被分区的提示词。

**** 注意：剖面解剖 ****

部分	作用
角色线	Agent 是谁，在哪里有效
Agent 机构	允许并指示执行任务
护栏	如何行动：极简、重用、保守地使用 deps。
工具使用（后期）	在混合工具的使用中，使用哪种工具
通信（后期）	如何汇报结果

从自主权和护栏开始。另外两位在提示词超过 20 行时才会获得属于自己的位置。

动手试试

在更换前后重复同一提示词，观察 Agent 的姿势变化：

```
bun run index.ts . "Find all TODO comments and tell me where they are"

1
```

对于这句单句话，Agent 可能会先列出自己的计划（“我用 grep 搜索。……”）。有了结构化提示词，它直接进入工具调用并报告发现。

```
npx tsc --noEmit

1
```

**** 注意：不要指望每次都会有戏剧性的转变 ****

如果你的 Agent 已经用一行提示词来操作（模块 2 的描述足够强，通常都能做到），那么明显的变化可能很细微。持久的好处是策略：你明确了操作风格，这意味着你可以更改它而不用重写整个 Agent。

提交

```
git add index.ts
git commit -m "feat(prompt): add Agency and Guardrails sections"
```

1
2

完成标准

- ☐ `instructions` 包括 `# Agency` 和 `# Guardrails` 部分
- ☐ 机构告诉 Agent 使用工具，而不是解释
- ☐ 护栏限制了范围（最小改动，重复使用，不新增 DEPS）
- ☐ 工作目录仍然会插入提示词
- ☐ `npx tsc --noEmit`

** 注意：测试提示词的边缘 **

试着完全删除 Agent 机构部分。重复同样的任务。Agent 会不会又回到解释模式？试着拆掉护栏。它会开始添加随机的 NPM 包吗？每个部门都在做功课。弄清楚哪个部位捕捉到哪种行为，就是整个游戏的全部。

参考实现

```
const agent = new ToolLoopAgent({
  model: "anthropic/claude-haiku-4-5",
  instructions: `You are a coding agent working in: ${cwd}`

# Agency
- USE your tools. Read files, search code, run commands, then answer.
- Do NOT explain what you WOULD do. Actually do it.
- Prefer grep for searching, read for viewing files.
- Use bash only for commands that aren't covered by other tools.

# Guardrails
- Prefer simple, minimal changes
- Search before creating, and reuse existing patterns
- No new dependencies without asking`,
  tools: { read, grep, bash },
  stopWhen: stepCountIs(10),
});
```

1
2
3
4
5
6
7
8

9
10
11
12
13
14
15
16
17

动态构建系统提示词

模块 3 · 系统提示词 · 第 9/39 页 · 原文: <https://agentharnesszh.vercel.app/dynamic-prompt-construction>

动态构建系统提示词

上一课的分段提示词是硬编码的。当只有一个项目、一个沙箱和一个固定工具集时，这没问题。只要这些人动了，提示词就得跟着动。

不同的工作目录。后端沙箱不同。一个只有 `read` 和 `grep` 的子 Agent。硬编码字符串无法承载这些内容。函数可以。

学习成果

`src/system.ts` 中的 `buildSystemPrompt(context)` 函数返回了类型 `PromptContext` 的系统提示词字符串。Agent 的 `instructions` 现在是从运行时状态派生的，而不是粘贴进去的。

快速路径

1. 创建 `src/system.ts`，配备 `PromptContext` 接口和 `buildSystemPrompt(ctx)` 函数
2. 从多个章节组成提示词，包括选修部分如 `gitBranch` 和 `projectContext`
3. 调用 `buildSystemPrompt(...)` 在 `index.ts` 中，并将结果传 `instructions`

动手练习 3.2

把提示词提取到一个类型化的构建器中。

要求：

1. 定义 `PromptContext`，包含 `workingDirectory`、`sandboxType`、`toolNames`、可选 `gitBranch`、可选 `projectContext`
2. 写入返回分段提示词的 `buildSystemPrompt(ctx: PromptContext): string`
3. 将 `gitBranch` 和 `projectContext` 设为有条件，只有在设置时才包含它们的部分
4. 在 `index.ts` 中，构建上下文对象，并为 `instructions` 字段调用 `buildSystemPrompt(ctx)`

实现提示：

- 把部分推入数组，最后 `join("\n")`。纯字符串 `concat`，无模板引擎
- 条件段使用简单的 `if (ctx.foo) sections.push(...)`。不要追求更花哨的模式
- 保持 `buildSystemPrompt` 纯净。同样的情境，同样的环境，提示词出，没有副作用。这使得它可以进行单元测试

语境的形态

提示词取决于运行时状态。把这些状态装瓶成一个整体：

```
export interface PromptContext {
  workingDirectory: string;
  sandboxType: string;
  toolNames: string[];
  gitBranch?: string;
  projectContext?: string;
}
```

1
2
3
4
5
6
7

`workingDirectory` 和 `sandboxType` 总是适用。 `toolNames` 让提示词列出实际接线的工具（这在给予 Agent 子集时很重要）。 `gitBranch` 和 `projectContext` 是可选的，因为它们并不总是可知的。

构建器

```
export function buildSystemPrompt(ctx: PromptContext): string {
  const sections: string[] = [];

  sections.push(`You are a coding agent working in: ${ctx.workingDirectory}`);
  sections.push(`Sandbox: ${ctx.sandboxType}`);

  sections.push(`
# Agency
- USE your tools. Read files, search code, run commands, then answer.
- Do NOT explain what you WOULD do. Actually do it.
- Available tools: ${ctx.toolNames.join(", ")}`);

  if (ctx.gitBranch) {
    sections.push(`- Current branch: ${ctx.gitBranch}`);
  }

  sections.push(`
# Guardrails
- Prefer simple, minimal changes
- Search before creating, and reuse existing patterns
- No new dependencies without asking`);

  if (ctx.projectContext) {
    sections.push(`
```

```
# Project Instructions (from AGENTS.md)
${ctx.projectContext}`);
}

return sections.join("\n");
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30

这里没有模板引擎。没有什么 DSL。有一个阵列，几个 `push` 调用，还有一个 `join`。这是故意的。提示词是一根绳子。构建它应该看起来像搭一根绳子。

接入主流程

`index.ts`，将内联 `instructions` 的字面值替换为建调用：

```
import { buildSystemPrompt } from "./src/system";

const instructions = buildSystemPrompt({
  workingDirectory: cwd,
  sandboxType: "local",
  toolNames: Object.keys({ read, grep, bash }),
});

const agent = new ToolLoopAgent({
  model: "anthropic/claude-haiku-4-5",
  instructions,
  tools: { read, grep, bash },
  stopWhen: stepCountIs(10),
});
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Agent 在单一任务上的行为可能看起来和以前一样。这种胜利是结构性的。添加 git 上下文行、交换沙箱类型或剥离子 Agent 的部分，现在都需要编辑一个专注于函数的函数，而不是在多行字符串中查找并替换。

**** 注意：为什么是函数，而不是字符串 ****

提示词是 Harness 最重要的配置。把它设为函数意味着它可测试（断言输出在特定上下文下的样子）、可组合（添加部分而不影响其他部分）、可替换（用户可以自己构建构建器）、确定性（每次都相同的上下文、相同的提示词）。费用是一份文件。好处是在你第三次添加某个部分时才显现出来的。

动手试试

运行你以前用过的任何一个提示词：

```
bun run index.ts . "Find all TODO comments in this project"
```

1

输出应该和上一节课一样，因为提示词内容是一样的。这种变化是内心的。通过记录一次该提示词来确认 Agent 是否还具备所需工具：

```
console.log(instructions);
```

1

你应该能看到完整的 Agent 和护栏部分，并插值了工作目录和工具名称。

```
npx tsc --noEmit
```

1

提交

```
git add src/system.ts index.ts
git commit -m "refactor(prompt): extract buildSystemPrompt with runtime context"
```

1

2

完成标准

- ☐ `src/system.ts` 出口 `PromptContext` 和 `buildSystemPrompt`
- ☐ `buildSystemPrompt` 在相同的上下文下返回与上一课相同的提示词内容
- ☐ `gitBranch` 和 `projectContext` 为可选，仅在提供时才包含
- ☐ `index.ts` 调用 `buildSystemPrompt(...)` 代替使用内联字符串
- ☐ `npx tsc --noEmit`

**** 注意：写个测试给提示词 ****

添加一个快速断言：用 `gitBranch: "main"` 构建提示词，并确认输出中包含“当前分支：主”。不 `gitBranch` 地构建，确认这条线路不存在。这是提示词中最小的单元测试，它捕捉到了那种几乎无法通过

读取模型输出发现的漏洞。

参考实现

```
export interface PromptContext {
  workingDirectory: string;
  sandboxType: string;
  toolNames: string[];
  gitBranch?: string;
  projectContext?: string;
}

export function buildSystemPrompt(ctx: PromptContext): string {
  const sections: string[] = [];

  sections.push(`You are a coding agent working in: ${ctx.workingDirectory}`);
  sections.push(`Sandbox: ${ctx.sandboxType}`);

  sections.push(`
# Agency
- USE your tools. Read files, search code, run commands, then answer.
- Do NOT explain what you WOULD do. Actually do it.
- Available tools: ${ctx.toolNames.join(", ")}`);

  if (ctx.gitBranch) {
    sections.push(`- Current branch: ${ctx.gitBranch}`);
  }

  sections.push(`
# Guardrails
- Prefer simple, minimal changes
- Search before creating, and reuse existing patterns
- No new dependencies without asking`);

  if (ctx.projectContext) {
    sections.push(`
# Project Instructions (from AGENTS.md)
${ctx.projectContext}`);
  }

  return sections.join("\n");
}
```

1

2

3

4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38

```
import { buildSystemPrompt } from "./src/system";  
  
const tools = { read, grep, bash };  
  
const agent = new ToolLoopAgent({
```

```
model: "anthropic/claude-haiku-4-5",
instructions: buildSystemPrompt({
  workingDirectory: cwd,
  sandboxType: "local",
  toolNames: Object.keys(tools),
}),
tools,
stopWhen: stepCountIs(10),
});
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14

验证门

模块 3 · 系统提示词 · 第 10/39 页 · 原文: <https://agentharnesszh.vercel.app/verification-gates>

验证门

“我修好了 bug”是 Agent 无论是否修好都能自信地说的一句话。它还会告诉你“所有测试都通过”，但实际上从未运行过测试。这不是恶意。模型是模式匹配者，他们看到的模式常以“所有测试都通过”结尾。

解决办法是让验证变成契约，而不是氛围。我们将在系统提示词中添加一个章节，告诉 Agent 该运行什么、无法执行时该怎么办，以及之后如何范围理赔。

学习成果

`buildSystemPrompt` 包含一个验证部分，告诉 Agent 在存在时进行真实检查（`tsc`、`lint`、测试、构建），并在没有时诚实地进行报告范围。

快速路径

1. 给 `buildSystemPrompt` 添加一个 `# Verification` 部分
2. 告诉 Agent 运行本项目中存在的检查，而不是假设它们
3. 告诉 Agent 报告运行了什么、被阻挡了什么、哪些无法使用

动手练习 3.3

在你的提示词构建器中添加验证部分，并清楚地写出契约。

要求：

1. 在 `buildSystemPrompt` 中将 `# Verification` 段推入 `sections` 数组
2. 该章节指示 Agent 运行 `typecheck`（`TypeScript` 存在时）、`lint`、测试、构建，但仅限于现有的
3. 明确指示不要在未完成检查前宣称成功
4. 要求有范围的报告：运行的、被阻挡的、不可用的

实现提示：

- “不跑就宣称 NOT”是这句话的重担。没有它，模型们会填补空白
- 契约讲的是诚实，而非覆盖范围。一个说“我运行了 TSC，测试被审批阻挡”的 Agent 比声称“所有测试都通过”但不检查的“更有用”
- 不要把项目特定的命令内置到提示词里。第 3.4 课讲述了让项目告诉 Agent 该运行什么

契约

把这个区块加到 `buildSystemPrompt`：

```
sections.push(`
# Verification
After making changes, verify your work:
1. Run \`npx tsc --noEmit\` when TypeScript is present
2. Run lint, test, or build commands only if they exist in this project and are
  ↳ allowed by the current approval mode
3. Report exactly what you ran, what was blocked, and what was unavailable
4. Do NOT inflate partial verification into a blanket success claim

Do NOT claim "tests pass" without running them.
Scope your claims honestly. "Verification was limited because writes were blocked"
  ↳ is honest.
"All tests pass" when you didn't run them is not.`);
```

1
2
3
4
5
6
7
8
9
10
11

契约明确指出了不良行为。模型擅长避开你说的那些事。他们不擅长回避你只是暗示的事情。

范围化的索赔，并排呈现

以下是契约试图呈现的报道内容：

Agent 会说什么	你想让它说什么
“所有测试都通过”	“ <code>npm test</code> 过： 47 个通过， 3 个没通过（已有问题，与我的变更无关） ”
“我修好了那个 bug”	“ <code>auth.ts:42</code> 修正了空检。 <code>npx tsc --noEmit</code> 过去了。测试被审批模式阻挡了。”
“构建成功了”	“跑 <code>npm run build</code> ： 4.2 秒成功，未收到警告。”

右侧列是具体的。上面写了检查过什么、发现了什么，以及界限在哪里。左侧列是模型在提示词不反击时的默认声音。

这有什么作用，什么不做

验证部分并不保证检查通过。这让 Agent 对支票的报告变得真实。如果 `tsc` 失败，Agent 报告失败。如果测试从未进行是因为项目中不存在，Agent 也这么说。如果审批模式阻挡了 `lint` 命令，Agent 也会显示这个。

听起来很谦虚。实际上，这就是信任 Agent 输出和自己重新执行每次检查之间的区别。

**** 注意：核查关注的是范围，而非覆盖范围 ****

你不是让 Agent 检查所有东西。你让它准确地告诉你它检查了什么。一个小而诚实的瞄准镜比一个听来自信的全扫更有用。这与模块 2 中契约的描述是同一学科：大声说出极限，Agent 留在其中。

动手试试

做个小编辑（删除评论、重命名变量，或者任何可以恢复的），并请 Agent 确认：

```
bun run index.ts . "Rename the cwd variable to workingDir, then verify your work"
```

1

Agent 应当：

1. 选择 `edit` 或 `bash`
2. 快跑 `npx tsc --noEmit`
3. 具体报告 `tsc` 的结果，而不是毯子“看起来不错”。

如果你的审批模式阻碍了测试命令，Agent 应该大声说出来，而不是假装测试通过了。

```
npx tsc --noEmit
```

1

****Warning：虚构存在于 language****

注意那些柔和的短语：“应该没问题”、“看起来不错”、“我希望这能奏效。”那些是迹象。运行检查的模型使用过去时和特定结果。一个不使用对冲未来时态的模型。阅读这种措辞是评分 Agent 成果的一半工作。

提交

```
git add src/system.ts
git commit -m "feat(prompt): add Verification section to system prompt"
```

1

2

完成标准

- ☐ `buildSystemPrompt` 包含 `# Verification` 部分
- ☐ 这部分明确告诉 Agent 不能在不跑步的情况下宣称成功
- ☐ 该章节要求有范围的报告（已运行/阻塞/不可用）
- ☐ 在测试编辑中，Agent 会特别报告 `tsc` 的结果
- ☐ `npx tsc --noEmit`

**** 注意：了解 package.json 验证步骤 ****

目前验证部分是硬编码的。试着阅读 `package.json` 的 `scripts` 块，只包含实际存在的步骤。如果 `scripts.typecheck` 有明确定义，就列出来。如果 `scripts.lint` 丢失，不要让 Agent 跑 `lint`。这和下一节课（`AGENTS.md`）的原理相同，只是更窄、更早。

参考实现

```
export function buildSystemPrompt(ctx: PromptContext): string {
  const sections: string[] = [];

  // ... role, sandbox, Agency, Guardrails sections from previous lesson

  sections.push(`
# Verification
After making changes, verify your work:
1. Run \"npx tsc --noEmit\" when TypeScript is present
2. Run lint, test, or build commands only if they exist in this project and are
↳ allowed by the current approval mode
3. Report exactly what you ran, what was blocked, and what was unavailable
4. Do NOT inflate partial verification into a blanket success claim

Do NOT claim "tests pass" without running them.
Scope your claims honestly. "Verification was limited because writes were blocked"
↳ is honest.
"All tests pass" when you didn't run them is not.`);

  // ... projectContext section

  return sections.join("\n");
}
```

1
2
3
4
5

6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

项目上下文

模块 3 · 系统提示词 · 第 11/39 页 · 原文: <https://agentharnesszh.vercel.app/project-context>

项目上下文

你的 Harness 是通用的。它涉及的每个项目都不是。

有一个项目使用 `bun test`。下一个用的是 `vitest`。第三种用 `npm run check` 来做 `typechecklint` 组合，完全不符合你在提示词中设定的任何惯例。Agent 不应该需要猜测，你也不应该需要一个项目一个项目地教它。

大多数生产 Harness 使用的技巧和团队对人类贡献者用的一样：在仓库里放一个 `markdown` 文件，解释项目的工作原理。Agent 会读取并调整。

学习成果

如果工作目录中存在 `AGENTS.md` 文件，Harness 会读取该文件并将其内容注入系统提示词中，作为“项目指令”部分。没有文件时，提示词只回退到基础指令。

快速路径

1. 检查 `AGENTS.md` 里有没有工作目录
2. 如果有，读成字符串
3. 把它当作 `buildSystemPrompt` 传给 `projectContext`

动手练习 3.4

发现并注入工作目录 `AGENTS.md`。

要求：

1. `index.ts` 时，检查 `cwd/AGENTS.md` 是否存在
2. 如果有，就读作 UTF-8
3. 将内容（或 `undefined`）作为 `buildSystemPrompt` 传递给 `projectContext`
4. 确认提示词构建器现有的 `projectContext` 处理在有该部分时添加，缺席时省略

实现提示：

- `node:fs` 的 `existsSync` 在这里没问题。检查只在启动时进行一次，而不是在热循环中
- 使用 `path.join(cwd, "AGENTS.md")`，而非模板文字，以保持路径处理的一致性
- 暂时不要添加单仓库目录巡回。下面的挑战涵盖了这一点，模块 4 的沙箱抽象无论如何都会改变文件发现的工作方式

发现经过

整个故事只有几句话：

```
import { existsSync, readFileSync } from "node:fs";
import { join } from "node:path";

const agentsPath = join(cwd, "AGENTS.md");
const projectContext = existsSync(agentsPath)
  ? readFileSync(agentsPath, "utf-8")
  : undefined;
```

1
2
3
4
5
6
7

把它传给构建器：

```
const instructions = buildSystemPrompt({
  workingDirectory: cwd,
  sandboxType: "local",
  toolNames: Object.keys(tools),
  projectContext,
});
```

1
2
3
4
5
6

就这些。约定胜于配置。没有插件系统，没有注册，没有事件总线。一个已知位置的文件。

AGENTS.md 里会有什么

这个文件是用于 harness 无法从代码中推断出的项目特定事实：

```
# Project Instructions

## Commands
- `bun test` runs the test suite
- `bun run build` builds for production
- `bun run lint` checks code style
```

Architecture

- Monorepo, packages live in `packages/`
- Each package has its own `tsconfig.json`
- Shared types in `packages/shared/`

Style

- Functional components, no classes
- Named exports, not default
- Error messages must be user-facing

Lessons learned

- Auth middleware must run before rate limiting
- Don't modify migration files directly, generate new ones

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20

Agent 读到这些后做出调整。它知道指令。它熟悉架构。它了解项目中反复出现的错误。现在，这种 Harness 在 React 项目中表现得像 React 项目 Agent，在 CLI 项目中像 CLI 项目 Agent，因为项目本身告诉 Agent 自己是哪种项目。

**** 注意：一个熟悉的模式，文件名不同 ****

这和 Cursor 用在 [.cursorrules](#)、Codex 用 [AGENTS.md](#)、Claude 用代码 [CLAUDE.md](#)、用 π 用自己约定的技巧一样。文件名会有所不同。模式没有。发现一个 markdown 文件，注入为指令。

为什么现在一个文件就足够了

真正的 Harness 会在目录中父 Agent 单存储库中寻找 `AGENTS.md` 文件。它会把它们合并。它也可能支持 `.cursorrules`、`.github/copilot-instructions.md` 或 `~/.agents/default.md`。

我们还没有构建这些。工作目录里有一个文件涵盖了教学点：项目上下文来自文件，Harness 在启动时发现，提示词吸收。行走和合体叠加在上面，形状不变。

动手试试

在工作目录中输入一个 `AGENTS.md`，Agent 无法从代码中推断出具体指令：

```
# Project Instructions
- All commits must use the format `feat(scope): message`
- The verification step is `bun test`, not `npm test`
```

1
2
3

运行 Agent：

```
bun run index.ts . "What command do I use to run the tests in this project?"
```

1

Agent 应该回答，因为档案里已经说明了 `bun test`。移除 `AGENTS.md`，继续运行同样的提示词。Agent 会猜（大概率 `npm test`）。

```
npx tsc --noEmit
```

1

提交

```
git add index.ts
git commit -m "feat(prompt): inject AGENTS.md as project context"
```

1
2

完成标准

- ☐ `index.ts` 检查 `AGENTS.md` 的工作目录
- ☐ 如果存在，其内容会作为 `buildSystemPrompt` 传递给 `projectContext`
- ☐ 在有 `AGENTS.md` 的情况下，Agent 会从文件中回答项目相关的具体问题
- ☐ 没有 `AGENTS.md`，Harness 仅使用基础指令运行，没有错误
- ☐ `npx tsc --noEmit`

**** 注意：单存储库的目录行进 ****

真正的单仓库在根节点有一个 `AGENTS.md`，每个包内部也有另一个。从 `cwd` 开始，走到仓库根点（找 `.git`），沿途收集所有 `AGENTS.md`。合并它们，最深的文件覆盖或扩展根节点。现在想想冲突：当根节点显示 `use npm` 而包显示 `use pnpm` 时会发生什么？不同的 Harness 解决这个问题方式不同。Pi 会合并所有找到的东西。Cursor 只用最深的。Codex 将根和 `cwd` 连接起来。试试一种策略，注意它哪里出了问题。

参考实现

```
import { existsSync, readFileSync } from "node:fs";
import { join } from "node:path";
import { buildSystemPrompt } from "../src/system";

const cwd = resolve(process.argv[2] || process.cwd());

const agentsPath = join(cwd, "AGENTS.md");
const projectContext = existsSync(agentsPath)
  ? readFileSync(agentsPath, "utf-8")
  : undefined;

const tools = { read, grep, bash };

const agent = new ToolLoopAgent({
  model: "anthropic/claude-haiku-4-5",
  instructions: buildSystemPrompt({
    workingDirectory: cwd,
    sandboxType: "local",
    toolNames: Object.keys(tools),
    projectContext,
  }),
  tools,
  stopWhen: stepCountIs(10),
});
```

1
2
3

4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24

设计接口

模块 4 · 沙箱抽象 · 第 12/39 页 · 原文: <https://agentharnesszh.vercel.app/designing-the-interface>

设计沙箱接口

你的工具效果很好。他们知道的也太多了。

`read` 知道 `readFileSync`。`bash` 知道 `execSync`。两人都知道自己运行在 `Node` 上。一旦你想让它们在别的地方运行，比如沙箱、远程虚拟机、内存文件系统，所有工具都得重写。

在构建任何后端之前，我们先写下接口。一个沙箱工具要调用，抽象上需要做什么？一旦契约存在，工具就会对其进行重构，后端则会补上。

学习成果

一个定义为 `readFile`、`exec`、`stop` 加单位字段的 `Sandbox` 接口。这三种工具（`read`、`grep`、`bash`）都调用接口，而不是直接使用 `Node API`。

快速路径

1. 定义 `Sandbox` 接口，包含 `type`、`workingDirectory`、`readFile`、`exec`、`stop`，以及可选的 `expiresAt` 和 `snapshot`
2. 重构 `read` 为调用 `sandbox.readFile(path)` 而非 `readFileSync`
3. 重构 `grep` 和 `bash`（或你的 `localOps.exec`）通过 `sandbox.exec(command)`

动手练习 4.1

编写接口并重构三个工具来使用它。

要求：

1. 在 `src/sandbox.ts` 中定义 `Sandbox`，包含 `type`、`workingDirectory`、`readFile`、`exec`、`stop` 以及可选的 `expiresAt` 和 `snapshot`
2. 每种方法都是 `async` 的，即使实现在底层是同步的
3. 把 `Sandbox` 传到工具工厂。请更新 `read`、`grep` 和 `bash` 以调用 `sandbox.readFile` 和 `sandbox.exec`
4. 构建还无法运行（你还没写实现）。没关系。我们下节课会讲这个

实现提示：

- 所有方法都 `async`，因为云后端需要，而不同实现间的签名不一致会带来混乱
- 使用可选方法（`expiresAt?`、`snapshot?(): Promise<...>`）来处理那些不适用于所有后端的功能
- `type: string` 用于日志和调试。别太快把它变成工会。如果你愿意，以后可以 `"local" | "just-`

bash" | "cloud"

接口

```
export interface Sandbox {
  type: string;
  workingDirectory: string;
  readFile(path: string): Promise<string>;
  exec(command: string): Promise<{ stdout: string; exitCode: number }>;
  stop(): Promise<void>;
  expiresAt?: number;
  snapshot?(): Promise<{ snapshotId: string }>;
}
```

1

2

3

4

5

6

7

8

9

值得一提的几个选择：

- 每个方法返回一个 **Promise** 。本地后端会将同步调用包裹。云后端确实是异步的。两者签名相同，保持工具简单
- **type** 和 **workingDirectory** 是恒等域。工具有时需要知道它们的位置和对应的是什么
- **expiresAt** 和 **snapshot** 是可选的。本地沙箱不会过期。 **just-bash** 沙箱不会快照。界面兼容两者，无需强制存根

每种方法的收益

方法	作用	必须吗?
readFile	通过路径读取文件	是
exec	执行命令	是
stop	优雅地关闭	是的（无操作也可以）
type	在日志中识别后端	是
workingDirectory	工具的基础路径	是
expiresAt	超时时间戳	没有（仅限云端）
snapshot	救救状态	没有（仅限云端）

把界面尽可能小。你现在添加的任何东西，都会是每个实现永远支持的。

重构工具

`read` 的重构只有一行：

```
// Before
execute: async ({ path: filePath }) => {
  const content = readFileSync(resolve(cwd, filePath), "utf-8");
  // ...
}

// After
execute: async ({ path: filePath }) => {
  const content = await sandbox.readFile(filePath);
  // ...
}
```

1
2
3
4
5
6
7
8
9
10
11

`grep` 和 `bash` 都采用了相同的处理方式，通过 `sandbox.exec(command)` 路由，而不是通过 `execSync` 或我们在模块 2 中构建的 `localOps` 对象。工厂函数现在接受 `sandbox` 参数并对其闭包。

工具的输入 `schema`、描述、行上限和匹配上限都保持不变。模型仍然看到相同的契约。发动机盖下的管道才是移动的。

**** 注意：胜利在于便携性，而非行为问题 ****

经过这个重构后，`Agent` 在同一个提示词上表现相同。同样的工具，同样的效果。这就是判断重构是结构性而非行为性的测试。胜利体现在你在第 4.3 课中加入第二个后端，无需动工具就能实现。

动手试试

你还没写过实现，所以代码不会端到端运行。你可以检查这些类型是否对应：

```
npx tsc --noEmit
```

1

如果你持续重构，这种情况会过去。所有关于工具内 `readFileSync` 和 `execSync` 的提及都应该被删除。现在这些工具期望一个 `Sandbox` 参数。

提交

```
git add src/sandbox.ts src/tools.ts
git commit -m "refactor(tools): route through Sandbox interface"
```

1

2

完成标准

- ☐ `src/sandbox.ts` 导出了 `Sandbox` 界面
- ☐ `read`、`grep` 和 `bash` 接受 `Sandbox`、调用 `sandbox.readFile` 和 `sandbox.exec`
- ☐ 现在没有工具可以直接导入 `readFileSync` 或 `execSync` 了
- ☐ `expiresAt` 和 `snapshot` 被指定为可选类型
- ☐ `npx tsc --noEmit`

**** 注意：再加一个方法而不破坏世界 ****

假设你也想要写文件的工具。在界面上添加 `writeFile(path: string, content: string): Promise<void>`。现在每个实现都必须支持它，包括那些写不合理（比如只读审查的沙箱）。正确的做法是什么？新的可选方法？有没有一个专门用于写入的接口沙箱？是实现中投出的错误，无法实现？每个项目的费用都不同。选一个，注意它在其他地方都被强迫了什么。

参考实现

```
export interface Sandbox {
  type: string;
  workingDirectory: string;
  readFile(path: string): Promise<string>;
  exec(command: string): Promise<{ stdout: string; exitCode: number }>;
  stop(): Promise<void>;
  expiresAt?: number;
  snapshot?(): Promise<{ snapshotId: string }>;
}
```

1
2
3
4
5
6
7
8
9

```
import type { Sandbox } from "./sandbox";

export function createReadTool(sandbox: Sandbox) {
  return tool({
    description: `Read a file from the project. Returns numbered lines.
    WHEN TO USE: viewing file contents, checking configs, reading source code.
    WHEN NOT TO USE: searching across files (use grep instead).`,
    inputSchema: z.object({
      path: z.string(),
      offset: z.number().optional(),
      limit: z.number().optional(),
    }),
    execute: async ({ path: filePath, offset, limit }) => {
      const content = await sandbox.readFile(filePath);
      // ... same line numbering and truncation logic
    },
  });
}

export function createBashTool(
  sandbox: Sandbox,
  needsApproval: (input: { command: string }) => boolean,
) {
  return tool({
    // ... same description and schema
    execute: async ({ command }) => {
      if (needsApproval({ command })) {
        return `Blocked: "${command}" requires approval.`;
      }
      const { stdout } = await sandbox.exec(command);
      return stdout || "(no output)";
    },
  });
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34

本地实现

模块 4 · 沙箱抽象 · 第 13/39 页 · 原文: <https://agentharnesszh.vercel.app/local-implementation>

本地实现

上一节课的界面本身没有任何反应。我们需要一个后端。

本地沙箱才是无聊的。它包裹的是你一直使用的 `readFileSync` 和 `execSync` 调用。不同的是，现在这些设备都隐藏在界面后面，所有工具都以相同的方式调用它们。

无聊才是重点。本地沙箱证明了该接口在不引入新复杂性的情况下可行。它是所有其他后端都会被比较的基线。

学习成果

`src/sandbox-local.ts` 导出 `createLocalSandbox(dir)`，一个工厂，返回一个 `Sandbox`，其方法包裹了 Node 的 `readFileSync` 和 `execSync`。Agent 运行方式和之前一样，但通过界面实现。

快速路径

1. 创建 `src/sandbox-local.ts` 导出 `createLocalSandbox(dir): Sandbox`
2. 用 `async readFile` 包裹 `readFileSync`
3. 用 `async exec` 包裹 `execSync`，尝试/接球回 `{ stdout, exitCode }`
4. 让 `stop()` 变成异步无操作者

动手练习 4.2

实施本地沙箱。

要求：

1. `createLocalSandbox(dir: string): Sandbox` 返回一个满足接口的对象
2. `readFile` 解析路径 `dir` 并读 UTF-8
3. `exec` 在有 `cwd: dir` 和 30 秒超时的情况下执行该命令
4. `exec` 错误时，`{ stdout: <whatever output there was>, exitCode: <non-zero> }` 回击而非投掷
5. `stop` 是 `async () => {}`

实现提示：

- 整个文件大约有 15 行。如果你的后台更长，可能你处理的是云后端会关注而本地后端不关心的案件
- `exec` 绝不应该扔球，即使是在非零的出场口。工具期望结果对象。发现错误并返回是正确的形状
- `type: "local"` 是 `sandboxType` 从模块 3 插入到系统提示词的部分

实现

```
import { readFileSync } from "node:fs";
import { resolve } from "node:path";
import { execSync } from "node:child_process";
import type { Sandbox } from "../sandbox";

export function createLocalSandbox(dir: string): Sandbox {
  return {
    type: "local",
    workingDirectory: dir,
    readFile: async (p) => readFileSync(resolve(dir, p), "utf-8"),
    exec: async (command) => {
      try {
        const stdout = execSync(command, {
          cwd: dir,
          encoding: "utf-8",
          timeout: 30_000,
        });
        return { stdout, exitCode: 0 };
      } catch (e: any) {
        return {
          stdout: e.stdout || e.stderr || e.message || "",
          exitCode: e.status ?? 1,
        };
      }
    },
    stop: async () => {},
  };
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

16
17
18
19
20
21
22
23
24
25
26
27
28

这就是整个后端。`stop` 是无营运的，因为没有什么需要清理的。本地文件系统和 `child_process` 会比 Agent 更持久。

完成接线

```
import { createLocalSandbox } from "./src/sandbox-local";
import { createReadTool, createGrepTool, createBashTool } from "./src/tools";

const sandbox = createLocalSandbox(cwd);
console.error(`Sandbox: ${sandbox.type}`);

const tools = {
  read: createReadTool(sandbox),
  grep: createGrepTool(sandbox),
  bash: createBashTool(sandbox, createApproval({ mode: "interactive" })),
};
```

1
2
3
4
5
6
7
8
9
10
11

现在工厂接管了沙箱。他们封闭它，从内部调用 `execute`。同样的工具，同样的 Agent，同样的提示词。

动手试试

用你一直在用的提示词。输出应保持不变：

```
bun run index.ts . "Read the tsconfig.json"
bun run index.ts . "Find all TODO comments"
bun run index.ts . "List all files in this directory"
```

1
2
3

Agent 应该完全一样。引擎盖下的管道不同。确认沙箱身份一次：

```
console.error(`Sandbox: ${sandbox.type}`);
```

1

你应该看看 `Sandbox: local`。

```
npx tsc --noEmit
```

1

**** 注意：如果行为发生变化，重构就会泄露 ****

经过这节课，Agent 在同一提示词上的行为应与模块 3 完全一致。如果有变化（路由不同、输出不同、新的错误），找一个工具仍然直接访问节点 API 而不是通过 `sandbox` 的地方。

提交

```
git add src/sandbox-local.ts index.ts
git commit -m "feat(sandbox): add local backend wrapping Node APIs"
```

1
2

完成标准

- ☐ `src/sandbox-local.ts` 出口 `createLocalSandbox(dir)`
- ☐ 返回的对象满足 `Sandbox` 接口
- ☐ `readFile` 和 `exec` 通过 Node API 路由，就像之前一样
- ☐ `stop` 是个不会崩溃的无操作者
- ☐ 这三种工具都还能用，和模块 3 一样
- ☐ `npx tsc --noEmit`

**** 注意：把执行流变成 `buffer` 而不是 `buffer**`**

`execSync` 等命令结束后，一次性把所有 `stdout` 都丢掉。对于长构建来说，这很痛苦。试着切换到 `spawn`，流式输出回每个区块。挑战在于：`Sandbox.exec` 签名最后一次返回 `{ stdout, exitCode }`。要流式传输，你需要不同的形状，比如异步迭代器。注意这种情绪如何反复影响到每一个调用 `exec` 工具。界面决策很棘手。

参考实现

```
import { readFileSync } from "node:fs";
import { resolve } from "node:path";
import { execSync } from "node:child_process";
import type { Sandbox } from "../sandbox";

export function createLocalSandbox(dir: string): Sandbox {
  return {
    type: "local",
    workingDirectory: dir,
    readFile: async (p) => readFileSync(resolve(dir, p), "utf-8"),
    exec: async (command) => {
      try {
        const stdout = execSync(command, {
          cwd: dir,
          encoding: "utf-8",
          timeout: 30_000,
        });
        return { stdout, exitCode: 0 };
      } catch (e: any) {
        return {
          stdout: e.stdout || e.stderr || e.message || "",
          exitCode: e.status ?? 1,
        };
      }
    },
    stop: async () => {},
  };
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28

内存实现

模块 4 · 沙箱抽象 · 第 14/39 页 · 原文: <https://agentharnesszh.vercel.app/in-memory-implementation>

内存实现

本地沙箱在真实文件上运行真实命令。这很好，直到你想让 Agent 探索代码而不信任它不会破坏任何东西。

`just-bash` 就是答案。这是一个语义写时复制虚拟文件系统：Agent 从真实磁盘读取，但写入的内容存于内存中，沙箱停止时会消失。快速、便宜、安全。非常适合探索、测试，或者任何你不想让任何 Agent 对你实际文件泄露时使用。

学习成果

`createJustBashSandbox(dir)` 返回了一份由 `just-bash` 支持的 `Sandbox`。Harness 可以通过环境变量在本地和内存后端之间切换，Agent 对任一运行相同的提示词。

快速路径

1. 用 `bun add just-bash` 安装 `just-bash`
2. 在 `src/sandbox-just-bash.ts` 中实现 `createJustBashSandbox(dir)`
3. `readFile` 并 `exec` 到 `just-bash` API，注意虚拟骑乘点
4. 在创业时根据 `process.env.SANDBOX` 选择后端

动手练习 4.3

加 `just-bash` 后端，接好环境变量开关。

要求：

1. `createJustBashSandbox(dir: string): Promise<Sandbox>`（注意 `Promise`，因为创建是异步的）
2. 用 `JustBashSandbox.create({ overlayRoot: dir })` 启动虚拟 FS
3. 在 `readFile` 和 `exec` 中，通过虚拟挂载点进行路径转换，`/home/user/project`
4. 在 `index.ts` 中，根据 `process.env.SANDBOX` 选择 `local` 或 `just-bash`

实现提示：

- `JustBashSandbox.create` 回应了一个承诺。你的工厂也必须是异步的
- 挂钩点就是陷阱。`overlayRoot: "/Users/you/project"` 不是安装在 `/`，而是安装在 `/home/user/project`。沙箱内的每条路径都必须加上该常数
- `runCommand` 返回的是命令句柄，而不是结果。退出码调用 `wait()`，`output()` 综合 `stdout/stderr`

just-bash API

简单介绍一下你要包的部分：

```
import { Sandbox as JustBashSandbox } from "just-bash";

const jb = await JustBashSandbox.create({ overlayRoot: "/path/to/project" });

const content = await jb.readFile("/home/user/project/package.json");

const cmd = await jb.runCommand("ls", { cwd: "/home/user/project" });
const finished = await cmd.wait();
console.log(await cmd.output());
console.log(finished.exitCode);
```

1
2
3
4
5
6
7
8
9
10

****Warning：安装点 trap****

当你通过 `overlayRoot: "/path/to/project"` 时，`just-bash` 会将该目录挂载在虚拟文件系统内的 `/home/user/project`。/ 没有。不是在原来的道路上。每个 `readFile` 和 `runCommand` 调用都必须使用虚拟挂载点。这会让你陷入困境。这会让大家困惑。

实现

```
import { Sandbox as JustBashSandbox } from "just-bash";
import type { Sandbox } from "./sandbox";

const MOUNT = "/home/user/project";

export async function createJustBashSandbox(dir: string): Promise<Sandbox> {
  const jb = await JustBashSandbox.create({ overlayRoot: dir });

  return {
    type: "just-bash",
    workingDirectory: dir,
    readFile: async (p) => {
```

```
    const virtualPath = `${MOUNT}/${p}`;  
    return jb.readFile(virtualPath);  
  },  
  exec: async (command) => {  
    const cmd = await jb.runCommand(command, { cwd: MOUNT });  
    const finished = await cmd.wait();  
    return {  
      stdout: await cmd.output(),  
      exitCode: finished.exitCode,  
    };  
  },  
  stop: async () => {},  
};  
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26

MOUNT 常数是 just-bash 后端唯一关心、本地后端不关心的。每一条进出的路都通过它被翻译。

接线环境变换开关

```
import { createLocalSandbox } from "./src/sandbox-local";
import { createJustBashSandbox } from "./src/sandbox-just-bash";

const sandboxType = process.env.SANDBOX || "local";
const sandbox =
  sandboxType === "just-bash"
    ? await createJustBashSandbox(cwd)
    : createLocalSandbox(cwd);

console.error(`Sandbox: ${sandbox.type}`);
```

1
2
3
4
5
6
7
8
9
10

工厂的 `local` 是同步的，`just-bash` 的工厂是异步的。条件句柄了这一点。所有下游（工具、Agent、提示词构建器）都是一样的。

写时复制，一句话

读取来自真实磁盘。写入会进入内存。真实的文件系统从未被修改。当沙箱停止时，虚拟文件系统会被回收。Agent 可以读取你的 `package.json`，然后创建和删除 `test.txt` 一百次，而你的磁盘项目依然不受影响。

动手试试

同样提示词，两个后端：

```
bun run index.ts . "Read the package.json"
```

1

```
SANDBOX=just-bash bun run index.ts . "Read the package.json"
```

1

你应该会得到相同的答案，分别在各自的运行中印有 `Sandbox: local` 和 `Sandbox: just-bash`。这就是界面在工作。

试试在内存后端做写形任务：

```
SANDBOX=just-bash bun run index.ts . "Create a file called scratch.txt with the  
↪ text 'hello'"
```

1

Agent 会写入文件。现在检查真正的磁盘： `scratch.txt` 不存在。写入发生在覆盖层，内存中。

```
npx tsc --noEmit
```

1

**** 注意：不是所有工具第一次尝试就能携带 ****

有些工具在两个后端之间工作方式完全相同。有些人会默默失败，因为他们对宿主有某种假设。 `grep` 是常见的“问题”，因为 `just-bash` 下的壳体行为是模拟的，且不总是与系统 `grep` 字节完全相同。可移植性测试是真实存在的，不是理论上的。计划换完后修理一两件工具。

提交

```
git add src/sandbox-just-bash.ts index.ts package.json  
git commit -m "feat(sandbox): add just-bash backend with in-memory FS"
```

1

2

完成标准

- ☐ `just-bash` 已安装
- ☐ `src/ sandbox-just-bash.ts` 导出 的 `createJustBashSandbox(dir)` 会 返回 `Promise<Sandbox>`
- ☐ 路径通过 `MOUNT` 常数
- ☐ `SANDBOX=just-bash bun run ...` 在内存后端运行 Agent
- ☐ `just-bash` 上的写入任务不涉及真实文件系统
- ☐ `npx tsc --noEmit`

**** 注意：找到泄漏的工具 ****

选择一个对 `local` 后端有效但在 `just-bash` 下失败或表现不同的提示词。追踪哪个工具在做特定主机的假设。然

后决定：你是修工具，还是让界面吸收差异（比如让 `just-bash` 为该命令提供垫片）？这两种方式都是设计上的选择。注意哪种方法让工具更简单。

参考实现

```
import { Sandbox as JustBashSandbox } from "just-bash";
import type { Sandbox } from "./sandbox";

const MOUNT = "/home/user/project";

export async function createJustBashSandbox(dir: string): Promise<Sandbox> {
  const jb = await JustBashSandbox.create({ overlayRoot: dir });

  return {
    type: "just-bash",
    workingDirectory: dir,
    readFile: async (p) => {
      const virtualPath = `${MOUNT}/${p}`;
      return jb.readFile(virtualPath);
    },
    exec: async (command) => {
      const cmd = await jb.runCommand(command, { cwd: MOUNT });
      const finished = await cmd.wait();
      return {
        stdout: await cmd.output(),
        exitCode: finished.exitCode,
      };
    },
    stop: async () => {},
  };
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13

14
15
16
17
18
19
20
21
22
23
24
25
26

```
const sandboxType = process.env.SANDBOX || "local";  
const sandbox =  
  sandboxType === "just-bash"  
    ? await createJustBashSandbox(cwd)  
    : createLocalSandbox(cwd);
```

1
2
3
4
5

云端实现

模块 4 · 沙箱抽象 · 第 15/39 页 · 原文: <https://agentharnesszh.vercel.app/cloud-implementation>

云端实现

本地后端免费且快速。just-bash 后台是免费且安全的。云端后端既不是。

云端沙箱是运行在别处的真实虚拟机。它拥有真实的文件系统、真实的 git、真实的 npm 和真实的网络。它还按分钟收费，每调用开一会儿增加延迟，无论你是否完成，三十分钟后都会失效。

这节课是概念和分析，不是逐步构建。云端沙箱工具（配置、快照、计费）是针对供应商的，变化速度快于课程的跟上。保持稳定的是形状：界面相同，权衡不同，工具也不在意。

学习成果

你可以阅读并推理云沙箱实现，并描述为什么 expiresAt 和 snapshot 作为可选字段存在于界面上。

云层沙箱长什么样

```
import type { Sandbox } from "./sandbox";

export async function createCloudSandbox(config: {
  template?: string;
  snapshotId?: string;
}): Promise<Sandbox> {
  const vm = await VercelSandbox.create(config);

  return {
    type: "cloud",
    workingDirectory: "/workspace",
    expiresAt: Date.now() + 30 * 60 * 1000,

    readFile: async (p) => {
      return vm.files.read(resolve("/workspace", p));
    },

    exec: async (command) => {
      const result = await vm.commands.run(command, { cwd: "/workspace" });
      return {
        stdout: result.stdout + result.stderr,
        exitCode: result.exitCode,
      };
    },
  },
}
```

```
    stop: async () => {  
      await vm.close();  
    },  
  
    snapshot: async () => {  
      const snap = await vm.snapshot();  
      return { snapshotId: snap.id };  
    },  
  };  
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32

33

34

35

形状和 `local`、`just-bash` 一样。这些方法恰好让网络调用。从工具的角度来看，这几乎是看不见的。从挂钟的角度看，那是唯一重要的事。

权衡取舍			
	地方	<code>just-bash</code>	克劳德
成本	免费	免费	每分钟
延迟	微秒级	微秒级	每调用有几十到几百毫秒
隔离	没有	部分（读取实数，写虚拟）	完整且独立的虚拟机
持久性	永久性	垃圾收集	快照或恢复
<code>git</code> ， <code>npm</code>	你的本地安装	模拟	真实的，单独安装的
暂停	没有	没有	硬性限制（通常为 30 到 60 分钟）

界面能兼容这三种，因为形状保持不变。可选字段（`expiresAt`、`snapshot`）之所以有其 `?`，是因为它们不适用于更简单的后端。

何时选择每种	
后端	最佳
<code>local</code>	本地开发、调试、可信环境
<code>just-bash</code>	探索、测试、不可信代码审查
<code>cloud</code>	生产、配置、多用户、完全沙盒执行

真正的 `Harness` 让用户在启动时可以选择。`Agent` 不知道也不在乎。

** 注意：为什么云后端在这里只是概念 **

我们不会针对实时供应商构建 `createCloudSandbox`，因为配置端的配置、网络细节、快照语义等各 `API` 因供应商和频繁变动而异。保留下来的教导点是界面是相同的。如果你想把它连接到 `Vercel Sandbox`，方法可以直接转换。

expiresAt 和 snapshot 给你买什么

`expiresAt` 让 Harness 知道计时在进行。长期运行的任务可以检查 `expiresAt`，决定是开始新操作还是结束。没有它，Agent 会运行直到出现网络错误，然后才得弄清楚发生了什么。

`snapshot` 让 Harness 在跑动中救状态。云沙箱三十分钟后就会消散。如果你在第 28 分钟截图，可以从快照中重新开始，重新开始沙箱，从中断处继续。第 7 模块对此有深入介绍。

两者都是可选的。本地沙箱不该假装有这种病。`just-bash` 沙箱没有有意义的方式来揭露这些问题。界面允许每个后端选择自己实际能提供的功能。

动手试试

本课没有代码可运行，但有一个概念检查：

1. 不要回头看表格，写下云后端 `readFile` 比本地后端 `readFile` 慢的两个原因
2. 请用一句话解释为什么 `expiresAt` 在界面上是可选的
3. 画出 `createCloudSandbox` 和你之前用过的供应商 API 对比的样子。`exec` 需要做什么而 `local.exec` 不需要？

答案的形状告诉你抽象概念是否已经理解。

提交

本课没有代码更改，所以没有提交。如果你画了一个云实现作为思考练习，就把它放在一个临时文件里。

完成标准

- ☐ 你可以解释为什么云端 `readFile` 沙箱会增加延迟
- ☐ 你可以解释为什么 `expiresAt` 存在于界面上
- ☐ 你可以描述一下你会选择 `cloud` 而不是 `local` 或 `just-bash`
- ☐ 如果有人问，你可以 API 对真正的医生实施 `createCloudSandbox`

**** 注意：绘制成本意识型 Harness ****

云端沙箱每分钟收费。在 Harness 上设计一个护栏，在运行成本超过阈值时提醒你。Agent 本身看不到成本，但 Harness 包裹能看到。支票在循环的哪个位置？当任务中途超过阈值时会发生什么？你是停下来、快照，还是直接问用户？每个答案都指向不同的操作模型。

生命周期钩子

模块 4 · 沙箱抽象 · 第 16/39 页 · 原文: <https://agentharnesszh.vercel.app/lifecycle-hooks>

生命周期钩子

制作沙箱是一半的工作。另一半则是围绕它发生的一切。

一个全新的云虚拟机没有你的 `git` 配置。它没有节点 `_modules`。它没有你的 `.env`。在 Agent 做任何有用的事情之前，必须先配置 `git`、安装依赖、复制环境文件。在沙箱关闭之前，必须先检查是否有未承诺的工作，并决定如何处理。

这些东西是生命周期的挂钩。当地沙箱几乎不需要它们。没有它们，云端的沙箱就无法使用。

学习成果

沙箱以 `index.ts` 调用 `afterStart` 和 `beforeStop` 钩子进行设置，类型定义则用 `src/sandbox.ts`。本地沙箱用空钩。模块 7 的云和生命周期工作已具备完善。

快速路径

1. 为 `src/sandbox.ts` 添加一个 `SandboxLifecycle` 界面，并可选 `afterStart`、`beforeStop`、`onTimeout`
2. `index.ts` 年创建沙箱后，调用 `await lifecycle.afterStart?.(sandbox)`
3. 在我之前，`sandbox.stop()`，调用 `await lifecycle.beforeStop?.(sandbox)`
4. 保持生命周期为空，留给本地。钩点存在，身体不必有

动手练习 4.5

在沙箱周围装可选的生命周期钩线。

要求：

1. 定义 `SandboxLifecycle`，包含三种可选方法，每个方法取一个 `Sandbox` 并返回 `Promise<void>`
2. 在 `index.ts` 中，将一个 `lifecycle` 对象传递到沙箱创建旁边
3. 调用 `await lifecycle.afterStart?.(sandbox)` 刚创建沙箱
4. 调用 `await lifecycle.beforeStop?.(sandbox)` 之前的 `sandbox.stop()`
5. 默认为空 `lifecycle = {}`，这样本地沙箱就不变了

实现提示：

- 可选链 (`?()`) 会帮你完成条件性调用。不需要 `if (lifecycle.afterStart)` 块
- 即使是空无一物的生命周期，依然是生命循环。不要让它在外层变得可选
- `onTimeout` 是吊带触发的钩子，不是你。云后端在达到 `expiresAt` 时触发。现在就删掉它，在模块 7 里用

接口

```
export interface SandboxLifecycle {  
  afterStart?(sandbox: Sandbox): Promise<void>;  
  beforeStop?(sandbox: Sandbox): Promise<void>;  
  onTimeout?(sandbox: Sandbox): Promise<void>;  
}
```

1
2
3
4
5

这三者都是可选的。本地沙箱可能根本不需要这些。生产 Harness 中的云端沙箱很可能同时使用这三种设备。

每个钩子的作用

afterStart 在沙箱创建并准备好接收指令后运行。这里是设置的地方：

```
const cloudLifecycle: SandboxLifecycle = {  
  afterStart: async (sandbox) => {  
    await sandbox.exec('git config user.name "Agent"');  
    await sandbox.exec('git config user.email "agent@example.com"');  
    await sandbox.exec("npm install");  
    await sandbox.exec("cp .env.example .env");  
  },  
};
```

1
2
3
4
5
6
7
8

beforeStop 在沙箱关闭前逃跑，这样重要的东西就有机会逃脱：

```
beforeStop: async (sandbox) => {  
  const { stdout } = await sandbox.exec("git status --porcelain");  
  if (stdout.trim()) {  
    await sandbox.exec('git add -A && git commit -m "WIP: auto-save"');  
  }  
}
```

```
    if (sandbox.snapshot) {  
      await sandbox.snapshot();  
    }  
  },  
}
```

1
2
3
4
5
6
7
8
9

onTimeout 在沙箱达到时间限制时运行。云端调用的是这个，不是你。尸体通常会重复利用一些 **beforeStop** 并进行一些日志记录：

```
onTimeout: async (sandbox) => {  
  console.error("Sandbox timed out, saving state");  
  await cloudLifecycle.beforeStop?.(sandbox);  
},
```

1
2
3
4

接到 **Agent** 循环

```
const sandbox = await createSandboxByEnv(cwd);  
const lifecycle: SandboxLifecycle = {};  
  
await lifecycle.afterStart?.(sandbox);  
  
try {  
  const { text, steps } = await agent.generate({ prompt });  
  console.log(text);  
  console.log(`\n(${steps.length} steps)`);  
} finally {  
  await lifecycle.beforeStop?.(sandbox);  
  await sandbox.stop();  
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13

`try/finally` 很重要。即使 `Agent` 在跑动中投掷，`beforeStop` 也应该开火。这才是未完成工作的支票该去的地方。

对于本地空 `lifecycle = {}` 沙箱，钩子都不会运行。`Agent` 的行为和之前完全一样。这个结构是为我们在模块 7 中添加真实钩子时准备的。

**** 注意：钩子是在云端，而不是本地的，而非本地 ****

对于本地后端，生命周期钩子大多是仪式性的。对于云后端来说，跳过 `beforeStop` 意味着虚拟机死机时会失去未提交的工作。界面让你同时思考这两者正是重点。局部情况是云壳的更简单形状，而不是不同的形状。

动手试试

`Agent` 应与上一课完全相同，因为本地生命周期为空。

```
bun run index.ts . "Read the package.json"
```

1

通过添加临时木桩确认管道类型是否有效：

```
const lifecycle: SandboxLifecycle = {  
  afterStart: async (sb) => console.error(`[lifecycle] after start: ${sb.type}`),  
  beforeStop: async (sb) => console.error(`[lifecycle] before stop: ${sb.type}`),  
};
```

1
2
3
4

随便跑任何提示词。你应该能看到两条原木线，把 Agent 的 Opus 包围起来。

```
npx tsc --noEmit
```

1

提交

```
git add src/sandbox.ts index.ts
git commit -m "feat(sandbox): add lifecycle hook points"
```

1

2

完成标准

- ☐ `SandboxLifecycle` 接口定义了三种可选方法
- ☐ `afterStart` 在沙箱创建后被召唤一次
- ☐ `beforeStop` 在 `sandbox.stop()` 前被叫一次，在 `finally` 内
- ☐ 空 `lifecycle` 时，Agent 保持不变
- ☐ 使用日志记录钩时，火灾生命周期调用有序
- ☐ `npx tsc --noEmit`

** 注意：快照和恢复作为生命周期对 **

生命周期钩子不仅仅是用来设置和拆除的。试试这个配对：`afterStart` 检查已知位置的快照，找到后从中恢复。`beforeStop` 自动快照后关机。现在你的 Harness 在调用点没有额外代码的情况下出现了崩溃-恢复行为。快照在哪里？你怎么分辨真正的新跑和恢复的跑？当快照来自不同版本的代码时会发生什么？模块 7 对此有深入介绍，但形状来自你刚才定义的生命周期界面。

参考实现

```
export interface SandboxLifecycle {
  afterStart?(sandbox: Sandbox): Promise<void>;
  beforeStop?(sandbox: Sandbox): Promise<void>;
  onTimeout?(sandbox: Sandbox): Promise<void>;
}
```

1

2

3
4
5

```
import type { SandboxLifecycle } from "../src/sandbox";

const sandbox = await createSandboxByEnv(cwd);
const lifecycle: SandboxLifecycle = {};

await lifecycle.afterStart?.(sandbox);

try {
  const { text, steps } = await agent.generate({ prompt });
  console.log(text);
  console.log(`\n(${steps.length} steps)`);
} finally {
  await lifecycle.beforeStop?.(sandbox);
  await sandbox.stop();
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

问题所在

模块 5 · 上下文管理 · 第 17/39 页 · 原文: <https://agentharnesszh.vercel.app/the-problem>

问题所在

你的 Agent 一直在逍遥法外。

在第 1 到第 4 模块期间，你们一直在执行短任务。读一个文件，找到 TODO，列出目录。五步，最多十步。上下文窗口保持舒适，Agent 保持敏锐，一切都很顺利。

这部分我们要转身看看真实任务中会发生什么。二十步。三十。那种 Agent 读取三个文件，搜索代码库两次，运行测试，修复一个问题，再运行测试，然后写摘要的任务。

在解决问题之前，我们需要先看到它。修正不大。看见才是难的部分。

学习成果

`onStepFinish` 在每一步记录输入和输出 token。运行多步任务使上下文增长可见：输入 token 线性上升，而输出大致保持平稳。

快速路径

1. 给 `ToolLoopAgent` 加 `onStepFinish` 回调
2. 每步记录 `usage.inputTokens` 和 `usage.outputTokens`
3. 运行一个多步任务，看着输入数字不断上升

动手练习 5.1

接线 token 日志，运行一个足够显示曲线的任务。

要求：

1. 把 `onStepFinish: ({ usage, stepNumber }) => { ... }` 传给 `ToolLoopAgent`
2. 使用 `console.error`（这样它会显示在 Agent 正常输出旁边，但不会出现在 stdout）
3. 运行一个强制 4+ 工具调用的提示词，这样曲线就能看到
4. 读数字。别急着修复任何东西

实现提示：

- `onStepFinish` 每一步都跑，不仅仅是成功的。`usage` 字段是相关的
- 用 `console.error` 来遥测。`console.log` 和 Agent 的反应混在一起，变得很糟糕
- 提示词需要做真正的工作。“读 package.json”只是一步。“读 package.json，然后 tsconfig，再读入口点，最后总结”是四。

添加日志记录

```
const agent = new ToolLoopAgent({
  // ... existing config
  onStepFinish: ({ usage, stepNumber }) => {
    console.error(
      `Step ${stepNumber}: ${usage.inputTokens} input, ${usage.outputTokens}
↪   output`,
    );
  },
});
```

1
2
3
4
5
6
7
8

这就是全部插桩代码。每一步后都会 SDK 会替你调用。

执行一项痛苦的任务

```
bun run index.ts . "Read package.json, then tsconfig.json, then index.ts, then
↪   summarize everything"
```

1

你应该看到类似这样的内容：

```
Step 0: 1,200 input, 450 output
Step 1: 2,800 input, 200 output
Step 2: 4,100 input, 180 output
Step 3: 8,900 input, 350 output
Step 4: 9,200 input, 600 output
```

1
2
3
4
5

具体数字会在你的项目中有所不同。形状不会变。输入 token 每一步都要攀爬。输出 token 大致保持平稳。

为什么输入 token 增长

每一步都会将整个消息历史发送给模型。用户提示词。系统提示词。Agent 的每一次工具调用。Agent 收到的每一个工具结果。

第一步的 package.json 在第四步仍然处于上下文中，尽管 Agent 已经结束了。第二步的 tsconfig 还在。没有什么东西会自己离开。

组成部分	token	行为
系统提示词	~500	修复了，每隔一调用
每一个工具结果	200 到 2000	始终保留在历史记录中
20 工具后调用	4000 到 40000	线性累积

上下文窗口是 20 万 token 。一个忙碌的 Agent 读取大文件，通常只需 30 到 50 步就能完成。当它出现时：

- 顶部的指示被忽视了
- 模型开始忽视自己的系统提示词
- 工具选择会退化
- Agent 会循环或产生幻觉

什么不行

三个诱人的非修复方案：

- **Hoping** 这并不 happen. 在实际任务中总是这样。
- **Reducing** 步 count. 十步对真正的工作来说太少了。五十是正常的
- **Using** 更大的 model. 更大的上下文窗口会拖延问题，并不能解决问题。而且每只 token 的费用也更高

解决办法是把旧工具结果从消息历史中移除，避免它们溢出注意力。第 5.2 课正是做到这一点的。

注意： 遥测先修，修第二

这课并不能解决问题。这是故意的。除非先测量问题，否则无法判断修复是否有效。token 日志会保留到模块的其余部分，方便你在每一步前后进行比较。

动手试试

执行多步骤任务。看看这些数字。确认：

1. 输入 token 随着每一步增长
2. 产量 token 保持相对较小
3. 到最后一步，你发送的 token 是第 0 步的三倍以上

```
bun run index.ts . "Read package.json, then tsconfig.json, then index.ts, then  
  summarize everything"
```

1

试试更长的任务，看看曲线变陡：

```
bun run index.ts . "Read every .ts file in src/, then tell me what each one does"
```

1

```
npx tsc --noEmit
```

1

提交

```
git add index.ts
git commit -m "feat(telemetry): log token usage per step"
```

1

2

完成标准

- ☐ `onStepFinish` 接线用于日志和 `usage.inputTokens` 和 `usage.outputTokens`
- ☐ 一个 4+ 步骤的任务显示输入 token 攀爬
- ☐ 输出 token 在各步间保持相对平稳
- ☐ 日志记录是 `console.error` 的，不是 `console.log`
- ☐ `npx tsc --noEmit`

** 注意：绘制曲线 **

记录数字给 `stderr` 没问题。把它们记录在 CSV 上更好。在 `onStepFinish` 中将 `step, inputTokens, outputTokens` 附加到文件中，然后加载到你选择的电子表格中。曲线的形状告诉你任务是读取量大（陡峭爬升）还是计算量大（较缓和爬升）。Agent 一样，曲线不同，取决于它在做什么。

参考实现

```
const agent = new ToolLoopAgent({
  model: "anthropic/claude-haiku-4-5",
  instructions: buildSystemPrompt({
    workingDirectory: cwd,
```

```
    sandboxType: sandbox.type,  
    toolNames: Object.keys(tools),  
    projectContext,  
  }),  
  tools,  
  stopWhen: stepCountIs(15),  
  onStepFinish: ({ usage, stepNumber }) => {  
    console.error(  
      `Step ${stepNumber}: ${usage.inputTokens} input, ${usage.outputTokens}  
↪   output`,  
    );  
  },  
});
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

修剪旧结果

模块 5 · 上下文管理 · 第 18/39 页 · 原文: <https://agentharnesszh.vercel.app/pruning-old-results>

修剪旧结果

解决办法是四行。

这部分会让人觉得有些反高潮。你在上一节课中测量了题目，观察输入 token 攀升，在第三十步勾勒出灾难场景。现在我们加上四条线，曲线变平。

台词本身很简单。他们去哪里，为什么去，这就是教训。

学习成果

`prepareCall` 在每个模型调用前运行 `pruneMessages`，移除比前三条消息更早的工具调用/结果对。上一节课的 token 增长曲线趋于平稳，而不是永远攀升。

快速路径

1. 从 `ai` 进口 `pruneMessages`
2. 给你的 `ToolLoopAgent` 配置添加一个 `prepareCall`
3. 里面，调用 `pruneMessages({ messages, toolCalls: "before-last-3-messages" })`
4. 先 `...options`，并防止 `messages` 在第一调用

动手练习 5.2

将线材修剪到 Agent 中，重复第 5.1 课的多步骤任务。

要求：

1. 把 `prepareCall: async (options) => ({...})` 添加到 Agent 配置中
2. 展开 `...options`，确保必填字段如 `model` 和 `tools` 得以延续
3. 定义明确后有条件修剪 `options.messages`
4. 目前先用 `toolCalls: "before-last-3-messages"`（最简单且合理的策略）
5. 确认输入 token 台阶上停滞，而不是攀爬

实现提示：

- `prepareCall` 在每个模型调用前运行，完整请求 `options`。你进来时还在修改消息
- 先分散 `...options`，否则会失去 `model`、`tools` 和 `system`。修剪过的消息覆盖了传播
- 在第一个调用还没有消息（`prompt` 已设置，`messages`undefined`）。那就省略修剪了

修复方案

```
import { ToolLoopAgent, stepCountIs, tool, pruneMessages } from "ai";

const agent = new ToolLoopAgent({
  // ... existing config
  prepareCall: async (options) => ({
    ...options,
    messages: options.messages
      ? pruneMessages({
          messages: options.messages,
          toolCalls: "before-last-3-messages",
        })
      : undefined,
  }),
});
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14

四条线，一条进口。大部分代码是第一代调用的守卫。

到底发生了什么

每模型一调用前，`prepareCall` 就运行。它接收了 SDK 即将发送的完整请求。我们用修剪过的版本替换 `messages`，删除所有比最近三条消息更旧的工具调用和结果。

```
Before pruning at step 15:
[user prompt]
[assistant + tool_call] -> [tool_result]    (old, will be pruned)
[assistant + tool_call] -> [tool_result]    (old, will be pruned)
... 12 more pairs ...
[assistant + tool_call] -> [tool_result]    (recent, kept)
```

```
[assistant + tool_call] -> [tool_result]    (recent, kept)
[assistant] -> [user]                      (recent, kept)
```

After pruning:

```
[user prompt]                            (kept, original prompt)
[assistant + tool_call] -> [tool_result]    (recent)
[assistant + tool_call] -> [tool_result]    (recent)
[assistant] -> [user]                      (recent)
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14

原始用户提示词总是存活。最近的工具互动得以保存。对话中段，工具结果堆积起来，每个调用都被抛到脑后。

****Warning：** 有两个值得说的陷阱 loud**

Spread ...options first. `prepareCall` 收到完整的请求选项，包括 `model`、`tools` 和 `system`。忘记了牌数会悄然掉落，Agent 以令人困惑的方式断裂。

Guard messages. 在第一个调用，SDK 给你一个 `prompt` 场，但没有 `messages` 阵列。调用 `pruneMessages({ messages: undefined })` 投掷。三元系统句柄。

为什么要三条消息

`toolCalls: "before-last-3-messages"` 设置保留了对话的最后三条消息，而不仅仅是最近三对工具。这对模型来说，足够让模型知道自己在多步骤任务中的位置，而无需保留全部历史。

你可以调音这个。`before-last-1` 更激进，节省更多 token。`before-last-5` 更温和，也保留了更多上下文。三是个合理的默认配置，适用于各种任务形状。从这里开始。如果你有具体任务需要，可以稍后调整。

动手试试

运行第 5.1 课中的同样多步骤任务，并比较 token 曲线：

```
bun run index.ts . "Read package.json, tsconfig, index.ts, then summarize"
```

1

你应该看到类似这样的内容：

```
Step 0: 1,200 input, 450 output
Step 1: 2,800 input, 200 output
Step 2: 3,100 input, 180 output    (old results pruned)
Step 3: 3,400 input, 350 output    (growth plateaus)
Step 4: 3,200 input, 600 output    (stays flat)
```

1

2

3

4

5

具体数字取决于你的项目。形状才是关键。输入 `token` 第二或第三步时会停滞，而不是一直爬。

```
npx tsc --noEmit
```

1

**** 注意：证明在于形状，而不是数字 ****

不要指望能得到和示例相同的数字。`token` 数量取决于文件大小、模型选择以及提示词的具体措辞。需要验证的是曲线形状：之前是线性，之后是平台。

提交

```
git add index.ts
git commit -m "feat(context): prune old tool results in prepareCall"
```

1

2

完成标准

- ☐ `pruneMessages` 是从 `ai` 进口的
- ☐ `prepareCall` 是有线连接到 `Agent` 配置里的

- ☐ `...options` 先被传播，然后消息被覆盖
- ☐ 处理未定义消息的情况
- ☐ 在 4+ 步任务中，输入 token 停滞而非线性增长
- ☐ `npx tsc --noEmit`

**** 注意：找出你任务的修剪阈值 ****

默认 `before-last-3-messages` 是猜测。选择一个需要 Agent 记住几步前读过的内容（配置值、函数名、找到的 TODO）的任务。用 `before-last-1`、`before-last-3` 和 `before-last-5` 运行，看看什么时候线会断线 Agent。你 Harness 的正确数量取决于它的工作类型。

参考实现

```
import { ToolLoopAgent, stepCountIs, tool, pruneMessages } from "ai";

const agent = new ToolLoopAgent({
  model: "anthropic/claude-haiku-4-5",
  instructions: buildSystemPrompt({ /* ... */ }),
  tools,
  stopWhen: stepCountIs(15),
  onStepFinish: ({ usage, stepNumber }) => {
    console.error(
      `Step ${stepNumber}: ${usage.inputTokens} input, ${usage.outputTokens}
↪ output`,
    );
  },
  prepareCall: async (options) => ({
    ...options,
    messages: options.messages
      ? pruneMessages({
          messages: options.messages,
          toolCalls: "before-last-3-messages",
        })
      : undefined,
  }),
});
```

1
2
3
4
5
6
7
8
9

10
11
12
13
14
15
16
17
18
19
20
21
22

工具输出设计

模块 5 · 上下文管理 · 第 19/39 页 · 原文: <https://agentharnesszh.vercel.app/tool-output-design>

工具输出设计

修剪会断章取义地使用旧结果。这是必要的。这还不够。

如果一根工具结果是 5000token，修剪下一根并不能救你。伤害已经造成了。该模型已经输出了 5000token 的 grep，现在还得维持至少三回合。

更好的解决办法是上游。工具应默认产生小而具有结构且有界限的输出。修剪是清理团队。工具设计就是预防。

学习成果

你机束中的每个工具都有明确的输出上限（行数、匹配数、字符数），截断行为会反馈给模型，以便它需要分页时进行分页。

快速路径

1. `read` 最大 500 行，并带有偏移/限页码
2. 上限 `grep` 50 场比赛，返回总比赛数
3. 输出上限 `bash` 5000 字符，保留尾部
4. 每个帽都会显示模型能看到并处理的截断消息

动手练习 5.3

对这三种工具应用有界输出契约。

要求：

1. `read` 保留了模块 1 的 500 行上限，并设有 `offset` 和 `limit` 参数用于分页
2. `grep` 保留了模块 1 的 50 场限制，截断时加上“（ N total and show first 50 ）”后缀
3. `bash` 增加了 stdout 的 5000 字符上限。保留尾部（最后 5000 个角色），不要保留首部，因为错误通常留在结尾
4. 每个截断都附加一个清晰的信息，比如 `"... (truncated, showing last 5000 chars)"`

实现提示：

- 截断信息是模型唯一表明数据更多的信息。它必须是可见的
- 对于 `bash` 来说，切尾通常是正确的。构建输出、测试失败和栈跟踪通常放在最后。如果你的工具能执行对头很重要的命令，那就换个方式
- “有界限”并不意味着“很小”。500 条线，50 根火柴，5000 个角色。足够回答问题，又足够小以保持语境

帽表

工具	帽	为什么要这么做
<code>read</code>	500 行	足够读懂大多数文件。要足够大以抓住结构，又要小到不会把模型埋没
<code>grep</code>	50 场比赛	搜索结果返回了 50 条，回答了这个问题。五百个数据就是数据倾倒
<code>bash</code>	5000 条	大多数指令输出都能合适。 <code>npm install</code> 和朋友们制造了模型不需要的噪音

这些数字并不神圣。他们通过执行真实任务并注意疼痛部位来调校。如果你的 Harness 持续执行输出较长的指令，那就提高上限。如果你主要做快速搜索，那就降低它。

带电容的 Bash 输出

bash 工具之前没有输出上限。添加一个：

```
const MAX_BASH_CHARS = 5000;

const stdout = result.stdout || "(no output)";
const cappedStdout =
  stdout.length > MAX_BASH_CHARS
    ? stdout.slice(-MAX_BASH_CHARS) +
      `\\n... (truncated, showing last ${MAX_BASH_CHARS} chars)`
    : stdout;

return cappedStdout;
```

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10

从末端切片是有意为之。Agent 执行的大多数命令最后都会大声失败。失败的测试会最后打印失败的部分。失败的构建会最后打印错误。保留尾巴可以保留 Agent 需要行动的部分。

结构化回报，而非原始数据倾销

grep 工具从模块 1 已经做到了，但值得重申这个模式：

```
const lines = stdout.trim().split("\n").filter(Boolean);
const MAX_MATCHES = 50;
const truncated = lines.length > MAX_MATCHES;
const result = truncated ? lines.slice(0, MAX_MATCHES) : lines;

return truncated
  ? result.join("\n") + `\n... (${lines.length} total, showing first $
↳ {MAX_MATCHES})`
  : result.join("\n") || "No matches found.";
```

1
2
3
4
5
6
7
8

截断消息给模型提供了两个可作用的信息：结果比显示的多，以及具体数量。这样，Agent 可以决定缩小搜索范围或分页。

截断契约

所有能够产生无界输出的工具都应遵循相同的形状：

1. 将输出限制在一个合理的上限
2. 告诉模型，输出被截断了多少，删减了多少
3. 在工具支持的分页参数上提供（[read](#) 偏移/限位，[grep`glob](#) 模式更窄）。

契约让 Agent 能够做出反应。一个无声截断的工具比完全没有截断更糟糕，因为模型认为自己掌握了全貌，却基于不完整的数据采取行动。

**** 注意：瓶盖是 Agent 在分页中缴纳的税 ****

有界输出会让某些任务稍微变慢一些。要读取一个 2000 行的文件，Agent 现在需要四个 [read](#) 调用而不是一个。这是正确的权衡。四次有界读取在成本和 token 上都比一次大量读取更便宜，后者会在整个会话中污染上下文。

动手试试

进行一个你知道会返回大量匹配的搜索：

```
bun run index.ts . "Find all import statements in this project"
```

1

你应该会看到 `grep` 回了 50 个匹配，尾部会统计总匹配数。如果你让 `Agent` 继续，它应该缩小搜索范围或使用更具体的球状，而不是要求无界转储。

试试一个能输出大量输出的命令：

```
bun run index.ts . "Run: ls -laR"
```

1

如果递归列表超过 5000 字符，你应该会看到截断信息。`Agent` 应通过缩小列表范围或要求特定子目录来应对。

```
npx tsc --noEmit
```

1

提交

```
git add src/tools.ts
git commit -m "feat(tools): cap bash output at 5000 chars with tail-keep"
```

1

2

完成标准

- ☐ `read` 最多 500 行，带有偏移/限位分码
- ☐ `grep` 最多匹配 50 个，截断时加上 `(N total)` 后缀
- ☐ `bash` stdout 最多 5000 字符，保留尾部
- ☐ 每个帽都会显示模型能识别的清晰截断信息
- ☐ 没有工具能将无界数据直接导入上下文
- ☐ `npx tsc --noEmit`

**** 注意：让电容可配置 ****

硬编码的上限是一个起点。一个快速检查的子 `Agent` 可能需要 100 行，而不是 500 行。深入分析可能需要 2000 美元。重构你的工具工厂，让它接受 `caps` 配置对象。现在来电者可以按每个 `Agent` 调音。注意权衡：可配置电容意味着用户有更多旋钮可以设置错。正确的默认在哪里？

参考实现

```
export function createBashTool(  
  sandbox: Sandbox,  
  needsApproval: (input: { command: string }) => boolean,  
) {  
  const MAX_BASH_CHARS = 5000;  
  
  return tool({  
    description: `Execute a shell command in the working directory.  
WHEN TO USE: build commands, package install, tests, git, directory listings.  
WHEN NOT TO USE: reading file contents (use read).  
DO NOT USE FOR: reading files (use read), searching code (use grep).`,  
    inputSchema: z.object({  
      command: z.string().describe("Shell command to execute"),  
    }),  
    execute: async ({ command }) => {  
      if (needsApproval({ command })) {  
        return `Blocked: "${command}" requires approval.`;  
      }  
      const result = await sandbox.exec(command);  
      const stdout = result.stdout || "(no output)";  
      return stdout.length > MAX_BASH_CHARS  
        ? stdout.slice(-MAX_BASH_CHARS) +  
          `\n... (truncated, showing last ${MAX_BASH_CHARS} chars)`  
        : stdout;  
    },  
  });  
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14

15
16
17
18
19
20
21
22
23
24
25
26
27

缓存控制

模块 5 · 上下文管理 · 第 20/39 页 · 原文: <https://agentharnesszh.vercel.app/cache-control>

缓存控制

修剪解决了问题的一半。另一半原因是每个调用仍然会传递那些未变的上下文部分。

你在十调用系统提示词和一时调用是一样的。早期用户提示词也是如此。工具定义是一样的。供应商以前见过这些 token，你得付钱再寄一次。

缓存控制是医生表达“我记得这件事，不要再处理”的方式。在支持它的地方，成本下降是真实的。即使没有，这种模式仍然教会了一个有用的习惯：将稳定的情境与新的情境分开。

学习成果

一个 `prepareCall` 流水线修剪旧工具结果，然后将剩余上下文中稳定部分标记为可缓存。支持缓存控制的供应商，重复投入成本大幅下降。

快速路径

1. 添加一个 `addCacheControl(messages)` 助手，标记稳定消息可缓存
2. 在 `prepareCall` 后再写 `pruneMessages`
3. 将系统消息和早期对话标记为可缓存；保持最新消息为新鲜

动手练习 5.4

在修剪台阶后面缓存控制铁丝网。

要求：

1. 写入 `addCacheControl(messages)` 返回一个新的消息数组，并在适当情况下返回 `providerOptions.cacheControl` 设置
2. 第一个消息（系统或初始用户提示词）应始终可缓存
3. 比前两条更早的消息应可缓存
4. 最近的一两条消息应该保持不缓存
5. 更新 `prepareCall` 给调用 `addCacheControl(pruneMessages(...))`

实现提示：

- `cacheControl: { type: "ephemeral" }` 是带有拟人味的形状。其他供应商使用不同的密钥。如果你遇到不支持缓存控制的，调用依然能用，只是首部被忽略了
- 缓存断点按前缀作用。将消息 5 标记为可缓存，缓存了包括消息 5 在内的所有内容。提供者在下一次调用检查前缀
- 不要把最近消息标记为可缓存。它们快要被替换了

辅助函数

```
import type { ModelMessage } from "ai";

export function addCacheControl(messages: ModelMessage[]): ModelMessage[] {
  return messages.map((msg, i) => {
    if (i === 0) {
      return {
        ...msg,
        providerOptions: { cacheControl: { type: "ephemeral" } },
      };
    }
    if (i < messages.length - 2) {
      return {
        ...msg,
        providerOptions: { cacheControl: { type: "ephemeral" } },
      };
    }
    return msg;
  });
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19

第一条消息和除了最后两条外的其他都显示缓存标记。最近的更新会保持不在，这样就不会在下一个调用替换它们之前被缓存。

用修剪来作曲

在 `index.ts` 中, `prepareCall` 变成了一条管道:

```
import { addCacheControl } from "./src/cache";

prepareCall: async (options) => {
  const pruned = options.messages
    ? pruneMessages({
        messages: options.messages,
        toolCalls: "before-last-3-messages",
      })
    : undefined;

  return {
    ...options,
    messages: pruned ? addCacheControl(pruned) : undefined,
  };
},
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

修剪是因为它改变了消息数量。缓存会在存活的消息上进行。

节省的具体情况

这些数字取决于提供者、缓存命中率以及上下文的稳定性程度。对于典型的人类支持 Agent 进行长时间会话:

组成部分	无缓存的成本	含缓存的成本
50 调用, 每人 20 万输入	每一个调用全额输入	缓存的稳定前缀

组成部分	无缓存的成本	含缓存的成本
1000 万 token 全价	~ 每次 30 美元	~ 每次 6 美元

这对长时间训练来说是一个数量级的波动。短会话时，差额较小，因为缓存没有时间摊销。自律依然适用。

**** 注意：缓存控制是针对医疗服务提供者的 ****

Anthropic 在消息部分使用 `cacheControl`。OpenAI 使用不同的头部和不同的模型。有些服务商根本不在提示词层面暴露缓存。这种模式（稳定型和新鲜型分开）能在这些差异下存活下来。但 `providerOptions` 形状的确切形状则不然。

为什么即使在不支持缓存的情况下，模式也很重要

你强迫自己思考提示词哪些部分是稳定的。这是个有用的习惯。它告诉你你的系统提示词是每调用做的工作太多（每次重建）还是刚好（只做一次，重复使用）。

即使缓存明天消失，识别稳定上下文的纪律依然值得保留。稳定上下文也更容易测试、更容易版本化，也比每调用变化的上下文更容易推理。

动手试试

运行一个多步骤任务，观察 token 计数：

```
bun run index.ts . "Read package.json, tsconfig, index.ts, then summarize"
```

1

如果你的服务提供者在使用对象中返回缓存命中信息，请记录：

```
onStepFinish: ({ usage, stepNumber }) => {
  console.error(
    `Step ${stepNumber}: ${usage.inputTokens} input, ${usage.outputTokens} output,
↪  ${usage.cachedInputTokens ?? 0} cached`,
  );
},
```

- 1
- 2
- 3
- 4
- 5

你应该能看到 `cachedInputTokens` 从第一步开始增长，`inputTokens`（你付全价买的部分）保持小规模。

```
npx tsc --noEmit
```

1

提交

```
git add src/cache.ts index.ts
git commit -m "feat(context): add cache control to stable messages"
```

1

2

完成标准

- ☐ `addCacheControl(messages)` 表示稳定消息可缓存
- ☐ `prepareCall` 先修剪再缓存，顺序就是这样
- ☐ 最新的消息未缓存
- ☐ 在支持缓存的提供商中，使用 `cachedInputTokens` 显示
- ☐ `npx tsc --noEmit`

**** 注意：构建一个 token 预算的仪表盘 ****

你有 5.1 课的遥测，5.2 的剪枝，5.3 的 caps，还有本课的缓存。把它们接在一起。每一步后，记录输入 token（未缓存）、缓存 token、输出 token 和运行成本。会话结束时，打印总成本以及如果不剪枝和缓存会话的成本。数字决定了这门学科的价值。

参考实现

```
import type { ModelMessage } from "ai";

export function addCacheControl(messages: ModelMessage[]): ModelMessage[] {
  return messages.map((msg, i) => {
    if (i === 0) {
      return {
        ...msg,
        providerOptions: { cacheControl: { type: "ephemeral" } },
      };
    }
    if (i < messages.length - 2) {
      return {
        ...msg,
```

```
        providerOptions: { cacheControl: { type: "ephemeral" } },
      };
    }
    return msg;
  });
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19

```
prepareCall: async (options) => {
  const pruned = options.messages
    ? pruneMessages({
        messages: options.messages,
        toolCalls: "before-last-3-messages",
      })
    : undefined;

  return {
    ...options,
    messages: pruned ? addCacheControl(pruned) : undefined,
  };
},
```

1
2

3
4
5
6
7
8
9
10
11
12
13

为什么要委派

模块 6 · 子 Agent 委派 · 第 21/39 页 · 原文: <https://agentharnesszh.vercel.app/why-delegate>

为什么要委派

你刚刚花了一整模块讲情境管理。修剪、封盖、缓存。Agent 比以前更乖了。问题还没有完全解决。

在五十步任务中，Agent 失败的部分是修剪无法捕捉的。不是因为背景太长，而是因为 Opus 本身对 Agent 来说形状不对。探索、规划、执行和验证彼此交织。探索的 Agent 变成了正在做出改变的 Agent，探索时的所有背景依然保留着。

委托则把这些拆解开来。父 Agent 决定该怎么做。子 Agent 去做。每一个都是孤立运行的。父 Agent 得到答案，而不是旅程。

学习成果

你可以描述委派修复的三种单 Agent 失败模式，也能判断什么任务适合委派，什么时候更适合父 Agent。

失效模式

这些失败是修剪无法发现的。

环境污染

Agent 读取二十个文件以理解代码库。现在有二十个文件内容在上下文中。当 Agent 开始做更改时，与变更相关的文件会被埋在十五个五步前有用、现在毫无用处的文件里。

修剪最终会把它们去除，但前提是它们会把实际的任务从最上面的注意力中移开。

失去了专注

到了第三十步，Agent 已经漂移了。系统提示词说“重构认证模块”。在某个阶段，Agent 发现了一个 CSS 错别字，修正了它，然后发现了一个次优导入，重构了它，最后写了一条评论解释一个它感兴趣的函数。最初的任务被遗忘了。

这就是当一个人有太多顾虑和绳索过多时 Agent 会发生的事。

过宽能力

Agent 有 `write` 和 `bash`。在探索过程中，它“贴心”地修正了它在阅读文件中发现的一个拼写错误。修复会破坏别的东西。探索不应该被允许修改任何东西，但一个拥有完整工具的单 Agent 不会为自己划清界限。

模式

委派将 Agent 划分为不同工具和模型的角色。

Parent

- Plans the work
- Delegates research to an Explorer
- Delegates implementation to an Executor
- Synthesizes results
- Makes architectural decisions

Explorer subagent

- Read and grep only (cannot modify anything)
- Cheap, fast model (Haiku)
- Reports findings, does not act

Executor subagent

- Full tools, including write and bash
- Stronger model (Sonnet or Opus)
- Follows precise instructions from the parent
- Cannot ask user questions (no askUser)

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

父 Agent 是唯一提出问题、做决策或制定长期计划的 Agent 。其他所有事情都被委托和规划。

何时委托

代表	保持在父 Agent
跨多个档案进行研究	单列变更
并行独立任务	顺序依赖变更
机械散装作业	建筑决策
行动前的探索	模糊需求（使用 <code>askUser</code> ）

分歧在于 Opus 是否干净利落地交接。阅读三十个文件并返回一段摘要，是干净利落的交接。决定三种架构方法中的哪一种则不重要，因为决策本身就是工作本身。

**** 注意：委派并非免费 ****

子 Agent 调用是一种全新的模型，拥有自己的启动时间 token、自己的系统提示词和延迟。不要把所有事情都委派去做。把父 Agent 从看不到全部痕迹中受益的工作委派给他们。如果父 Agent 能三步完成任务，委托就没有自偿。

动手试试

这是一个概念课。没有代码可运行。自我检视：

1. 不回头，说出三种单 Agent 故障模式
2. 从上周选一个你自己的编码任务。哪些部分会是探索器好的？哪一个执行器效果更好？哪些必须留在计划本那里？
3. 如果你把整个任务交给一个拥有所有工具、五十个步骤、没有委派的 Agent，会发生什么？

提交

这节课没有代码。下一课构建了探索器。

完成标准

- ☐ 你可以说出三种单 Agent 故障模式
- ☐ 你可以描述什么时候委派有效，什么时候无效
- ☐ 你可以勾勒出实际任务的哪些部分会分为父 Agent、探索器和执行器

探索型子 Agent

模块 6 · 子 Agent 委派 · 第 22/39 页 · 原文: <https://agentharnesszh.vercel.app/explorer-subagent>

探索型子 Agent

探索器是最简单构建的子 Agent，也是最实用的起步工具。

它能读取文件。它可以搜索。它别无选择。没有 `write`，没有 `bash`，也没有向用户询问。它调查一个问题，总结发现，然后消失。

这听起来像是个限制。是功能。探索器不能漂移，不能无意中做出改变，也不能用创意 `find -exec` 毁掉你的项目。它做了一件事，完成后，父 Agent 会得到一个干净的答案，而不是中间文件读取的四十步。

学习成果

面向父 Agent 的 `task` 工具会生成一个全新的 `ToolLoopAgent`，只有 `read` 和 `grep`，是廉价的模型，预算是五步。父 Agent 可以委派研究工作，并获得文本摘要。

快速路径

1. 定义一个 `task` 工具，其 `schema` 接受 `description` 子 Agent
2. 在 `execute` 内，用 `read` 和 `grep` 新实例化一个新的 `ToolLoopAgent`
3. 使用 `claude-haiku-4-5` 和 `stopWhen: stepCountIs(5)`
4. 将子 Agent 的文本回复返回父 Agent，并以尝试/接住的形式包裹

动手练习 6.2

从父 Agent 加上一个分组接缝，变成探索器子 Agent。

要求：

1. 在你的工具注册表中添加一个 `task` 工具
2. `schema` 会用 `description: string`，父 Agent 用来告诉子 Agent 调查什么
3. 在 `execute` 里，创建一个新 `ToolLoopAgent`，用 `read` 和 `grep`（不 `bash`，不 `askUser`）
4. 选一个快的模型（`claude-haiku-4-5`），把步数限制在 5
5. 返回探索器的文本回复，错误被捕捉并以字符串形式返回

实现提示：

- 探索器是按每个调用实例化的。不要重复使用。每个代表团都会获得新的一上下文窗口
- 重复使用 `read` 和 `grep` 已有的工具。它们封闭在父 Agent 使用的沙箱上，这正是你想要的
- 用尝试/捕捉和返回 `"Subagent error: ${e.message}"` 包裹 `explorer.generate(...)`，而不是让异常传播。父 Agent 期望任何工具都能回线

task 工具

```

import { ToolLoopAgent, stepCountIs, tool } from "ai";
import { z } from "zod";
import type { Sandbox } from "../sandbox";

export function createTaskTool(sandbox: Sandbox, parentTools: {
  read: ReturnType<typeof createReadTool>;
  grep: ReturnType<typeof createGrepTool>;
}) {
  return tool({
    description: `Delegate research to a read-only subagent.
    WHEN TO USE: investigating a codebase, finding patterns, gathering context
    across many files.
    WHEN NOT TO USE: making changes (the subagent cannot write or run commands).
    DO NOT USE FOR: tasks that need decisions or askUser interactions.` ,
    inputSchema: z.object({
      description: z.string().describe("What the subagent should investigate"),
    }),
    execute: async ({ description }) => {
      const explorer = new ToolLoopAgent({
        model: "anthropic/claude-haiku-4-5",
        instructions: `You are an explorer agent. Investigate and report back
        ↪ concisely.
        Working directory: ${sandbox.workingDirectory}`,
        tools: { read: parentTools.read, grep: parentTools.grep },
        stopWhen: stepCountIs(5),
      });

      try {
        const { text, steps } = await explorer.generate({ prompt: description });
        return text
          ? `[Explorer: ${steps.length} steps]\n${text}`
          : "(no response from subagent)";
      } catch (e: any) {
        return `Subagent error: ${e.message}`;
      }
    },
  });
}

```

1
2
3
4
5
6

7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37

以下是一些值得指出的设计选择：

- **Fresh Agent 根据调用。** 探索器无法跨越调用存活。每个任务都有自己的上下文窗口，这正是委派工作的全部意义所在
- ****No `bash`，没有，`askUser`.** 探索器可以读取和搜索。它无法修改项目，也无法暂停用户输入。父 Agent 继续掌控决策权
- ****Haiku，探索 Sonnet.** 不是阅读和总结，而不是深度推理。更快、更便宜的模型才是合适的选择
- ****Five steps.** 足够看几份文件并反馈。如果探索器需要更多，父 Agent 应该把任务拆成更小的部分
- ****Errors 返回为 `strings`.** 工具返回字符串到模型。未接住的例外打破了工具循环。返回错误文本让父 Agent 决定下一步

接入父 Agent

```
const tools = {
  read: createReadTool(sandbox),
  grep: createGrepTool(sandbox),
  bash: createBashTool(sandbox, createApproval({ mode: "interactive" })),
};

const tools_with_task = {
  ...tools,
  task: createTaskTool(sandbox, { read: tools.read, grep: tools.grep }),
};

const agent = new ToolLoopAgent({
  // ...
  tools: tools_with_task,
});
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

父 Agent 现在有四个工具：[read](#)、[grep](#)、[bash](#)和[task](#)。前三个是直接的。第四位代表。

****Warning：** 边做边添加日志 debug**

当子 Agent 什么都不返回或送错东西时，你根本不知道它运行过程中发生了什么。开发过程中，在任务工具中记录子 Agent 的步数和文本长度。没有这些，你会看到混乱的父 Agent 输出，不知道子 Agent 是跑了一级还是五级，发现了什么，还是悄悄失败了。

动手试试

请父 Agent 探索器适合的相关内容：

```
bun run index.ts . "Delegate to a subagent: find every place this project uses zod  
↳ and tell me which files import from it."
```

1

父 Agent 应该调用 `task` 这个描述。探索器应该运行，找到导入数据并返回摘要。父 Agent 应该把这个摘要传回给你。

作为对比，运行同一个任务，但没有显式委派指令：

```
bun run index.ts . "Find every place this project uses zod and tell me which files  
↳ import from it."
```

1

父 Agent 可能会授权，也可能不会。如果有强有力的工具描述，可能会直接影响调用 `grep`。没关系。当搜索需要浏览大量文件，而父 Agent 不希望所有文本都包含在上下文中时，委托工具才真正发挥作用。

```
npx tsc --noEmit
```

1

提交

```
git add src/tools.ts index.ts  
git commit -m "feat(subagents): add explorer via task tool"
```

1

2

完成标准

- ☐ `createTaskTool` 存在并返回一个 `task` 工具
- ☐ 任务工具每调用生成一个新的 `ToolLoopAgent`
- ☐ 探索器只有 `read` 和 `grep`
- ☐ 探索器使用 `claude-haiku-4-5`，停在 5 步处
- ☐ 错误以字符串形式返回，而非例外
- ☐ 父 Agent 可以委派研究，并获得一个干净的总结
- ☐ `npx tsc --noEmit`

** 注意：平行探险者 **

单一的探索器是协程。从父 Agent 的工具循环并行生成两个机器人才是真正的并行。试着更改任务工具的 schema，让它接受一系列描述并用 `Promise.all` 运行。现在父 Agent 可以同时研究代码库的三个不同部分并综合结果。为了利用这一点，父 Agent 提示词发生了哪些变化？

参考实现

```
import { ToolLoopAgent, stepCountIs, tool } from "ai";
import { z } from "zod";
import type { Sandbox } from "../sandbox";

export function createTaskTool(
  sandbox: Sandbox,
  parentTools: { read: any; grep: any },
) {
  return tool({
    description: `Delegate research to a read-only subagent.
    WHEN TO USE: investigating a codebase, finding patterns, gathering context.
    WHEN NOT TO USE: making changes (the subagent cannot write or run commands).
    DO NOT USE FOR: tasks that need decisions or askUser interactions.` ,
    inputSchema: z.object({
      description: z.string().describe("What the subagent should investigate"),
    }),
    execute: async ({ description }) => {
      const explorer = new ToolLoopAgent({
        model: "anthropic/claude-haiku-4-5",
        instructions: `You are an explorer agent. Investigate and report back
        ↪ concisely.
        Working directory: ${sandbox.workingDirectory}`,
        tools: { read: parentTools.read, grep: parentTools.grep },
        stopWhen: stepCountIs(5),
      });

      try {
        const { text, steps } = await explorer.generate({ prompt: description });
        return text
          ? `[Explorer: ${steps.length} steps]\n${text}`
          : "(no response from subagent)";
      } catch (e: any) {
        return `Subagent error: ${e.message}`;
      }
    },
  });
}
```

1

2

3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36

执行型子 Agent

模块 6 · 子 Agent 委派 · 第 23/39 页 · 原文: <https://agentharnesszh.vercel.app/executor-subagent>

执行型子 Agent

探索器收集信息。执行器会根据它采取行动。

这两个角色的分工沿着委托分开的线条划分，就像委托分开了 Agent 本身一样。探索成本低廉，只读且流量大。执行成本更高，可能修改文件，且需要更强的模型，因为出错的成本更高。

执行器从父 Agent 那里继承信任。父 Agent 决定执行器可以做什么，执行器严格按照指令操作，两个角色都不会向用户提出任何要求。这就留在父 Agent 里。

学习成果

任务工具中的第二个分支会生成一个执行器子 Agent，包含 `read`、`grep` 和委托模式 `bash`，使用 `claude-sonnet-4-6` 和 15 步预算。父 Agent 现在可以在委派时选择探索器或执行器。

快速路径

1. 用 `subagentType: "explorer" | "executor"` 扩展 `task` 工具 schema
2. 添加一个执行器分支，使用更强的模型、更大的步长预算和委托模式的 `bash`
3. 更新描述，使路由对父 Agent 清晰可见

动手练习 6.3

添加执行器角色并从 `task` 工具中路由到它。

要求：

1. 将 `subagentType` 添加到任务工具的输入 schema 中，作为 `"explorer" | "executor"` 枚举，默认为 `"explorer"`
2. `subagentType === "executor"` 后，实例化一个包含 `read`、`grep` 和委托模式的 `ToolLoopAgent`bash`
3. 执行器使用 `claude-sonnet-4-6` 和 `stopWhen: stepCountIs(15)`
4. 用 `createApproval({ mode: "delegated", trust: [...] })` 建立执行器的 `bash`，通过一个小型信托名单 (`"npm test"`、`"npm run build"`、`"npx tsc"`)
5. 更新任务工具描述，向父 Agent 解释这两个角色

实现提示：

- 执行器需要自己的 `bash` 工具，并支持委托模式的审批。不要重复使用父 Agent 的互动式攻击，因为互动模式会暂停给提示词执行器无法回答的用户
- Sonnet 是执行器工作的正确默认选择。Opus 对大多数实现任务来说是大材小用，速度慢到让人感觉不到

- 信任名单故意较小。执行器应只运行父 Agent 认为安全的命令。测试运行器和构建命令通常是安全的。软件包安装和迁移则不是

执行器分支

```
export function createTaskTool(
  sandbox: Sandbox,
  parentTools: { read: any; grep: any },
) {
  return tool({
    description: `Delegate work to a subagent.
Explorer (default): read-only research with a fast model.
Executor: implementation with a stronger model and delegated trust on bash.

WHEN TO USE: research across many files (explorer), bulk implementation (executor).
WHEN NOT TO USE: ambiguous requirements (use askUser),
architectural decisions (the parent decides).`,
    inputSchema: z.object({
      description: z.string().describe("Task instructions for the subagent"),
      subagentType: z
        .enum(["explorer", "executor"])
        .default("explorer")
        .describe("Subagent role"),
    }),
    execute: async ({ description, subagentType }) => {
      if (subagentType === "executor") {
        const executorBash = createBashTool(
          sandbox,
          createApproval({
            mode: "delegated",
            trust: ["npm test", "npm run build", "npx tsc"],
          }),
        );

        const executor = new ToolLoopAgent({
          model: "anthropic/claude-sonnet-4-6",
          instructions: `You are an executor agent. Follow instructions precisely.
Working directory: ${sandbox.workingDirectory}
Do NOT ask questions. Do NOT explore beyond what's needed. Execute the task.`,
          tools: {
            read: parentTools.read,
            grep: parentTools.grep,
            bash: executorBash,
          },
          stopWhen: stepCountIs(15),
        });

        try {
```

```
    const { text, steps } = await executor.generate({ prompt: description });
    return text
      ? `[Executor: ${steps.length} steps]\n${text}`
      : "(no response from executor)";
  } catch (e: any) {
    return `Executor error: ${e.message}`;
  }
}

const explorer = new ToolLoopAgent({
  model: "anthropic/claude-haiku-4-5",
  instructions: `You are an explorer agent. Investigate and report back
↪ concisely.
Working directory: ${sandbox.workingDirectory}`,
  tools: { read: parentTools.read, grep: parentTools.grep },
  stopWhen: stepCountIs(5),
});

try {
  const { text, steps } = await explorer.generate({ prompt: description });
  return text
    ? `[Explorer: ${steps.length} steps]\n${text}`
    : "(no response from explorer)";
} catch (e: any) {
  return `Explorer error: ${e.message}`;
}
},
});
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57

58
59
60
61
62
63
64
65
66
67
68
69
70
71

探索器与执行器一览

	探索器	执行器
工具	<code>read</code> , <code>grep</code>	<code>read</code> 、 <code>grep</code> 、 <code>bash</code> (委派)
模型	<code>claude-haiku-4-5</code>	<code>claude-sonnet-4-6</code>
阶梯预算	5	15
可以修改	否	是的 (在信托名单内)
可以问问用户	否	否

这两个角色在工具能力、模型强度和预算上存在差异。他们一致同意一点：两人都不能向用户提问。这种责任属于父 Agent ，也就是让人类参与其中的角色。

教学质量对执行器更重要

探索器主要是四处看看。模糊的描述仍然能带来有用的信息。执行器字面上完全按照指示操作。模糊的描述会得到模糊（甚至有害）的结果。

缺点：

Fix the auth bug.

1

好：

```
In src/auth.ts, the login function at line 42 doesn't check for null email.  
Add a null check before the database query. Run `npx tsc --noEmit` after the change.
```

1
2

父 Agent 的工作是提供目标、程序、约束和验证步骤。执行器的工作就是跟随他们。系统提示词的“请问问题吗”这句话在这里 NOT 真正发挥了作用。它迫使执行器要么基于现有条件采取行动，要么失败，而不是拖延澄清。

**** 注意：委托信任，而非全面信任 ****

执行器的 `bash` 使用模块 2 审批配置中的 `mode: "delegated"`。父 Agent 决定哪些命令值得信任。执行器可以 `npm test`。它不能运行 `npm install`、无 `rm -rf` 或列表中其他任何项目。这正是最初可辨识联合类型的合理性。

动手试试

请父 Agent 委派需要行动的事务，而不仅仅是研究：

```
bun run index.ts . "Delegate to an executor: rename the 'cwd' variable in src/  
↳ sandbox-local.ts to 'workingDir'. Then run npx tsc --noEmit and report the  
↳ result."
```

1

父 Agent 应该调用 `task` 和 `subagentType: "executor"`。执行器应该进行更改，运行 `typecheck`，并返回摘要。把你用探索器模式做同一任务的效果做比较（它不会改变任何东西）。

```
npx tsc --noEmit
```

1

提交

```
git add src/tools.ts  
git commit -m "feat(subagents): add executor role with delegated bash"
```

1
2

完成标准

- ☐ `task` 工具 schema 包括 `subagentType: "explorer" | "executor"`
- ☐ 执行器使用 `claude-sonnet-4-6` 和 15 步预算
- ☐ 执行器在 `mode: "delegated"` 有自己的 `bash`，信任名单较小
- ☐ 执行器严格执行指令，不问问题
- ☐ 探索器行为与上一节课无异
- ☐ `npx tsc --noEmit`

**** 注意：从父 Agent 处继承信任 ****

执行器的信任名单现在是硬编码的。试着把父 Agent 的信任列表串联进去：当父 Agent 生成执行器时，执行器会接收父 Agent 的安全命令。现在想想界限。执行器是否应该被允许以同样的信任孕育出另一个执行器？还是说每升级一级就应该缩小信任组？生产 Harness 的做法不同，答案取决于你对 Agent 规划的信任程度。

参考实现

完整实现请参见上文 `createTaskTool` 代码块。练习的解法是同样的代码，应用到你的 `src/tools.ts` 上。

任务工具

模块 6 · 子 Agent 委派 · 第 24/39 页 · 原文: <https://agentharnesszh.vercel.app/task-tool>

任务工具

你已经拥有大部分任务工具了。第 6.2 和 6.3 课构建了探索器，执行器分支纳入其 `execute` 功能。

本课讲的是把工具当作它实际的布线层来对待。父 Agent 调用 `task`。工具会选择正确的子 Agent 类型，验证父 Agent 生成该类型，并返回结果。以后增加更多角色（审查员、架构师、验证员）应该只是添加一个分支，而不是重新设计工具。

学习成果

`task` 工具结构为一个明确的路由器，包含一个整合的描述、角色特定模型，以及需要时的派生权限检查清晰位置。

快速路径

1. 收紧 `task` 工具描述，让父 Agent 知道什么时候该选择哪个角色
2. 把子 Agent 结构提取成一个小助手，这样以后添加角色就只剩一个区块
3. 勾勒出生成权限检查的形状，即使你还没强制执行

动手练习 6.4

重构 `createTaskTool` 让路由成为你首先看到的内容，下面是针对特定角色的构造。

要求：

1. 任务工具的描述会列出这两个角色，说明各自的用途，并指向非委派场景的父 Agent `askUser` 和直接工作
2. `execute` 体是一台精简路由器。每个角色都由同一文件内的独立辅助函数构建
3. 每个角色助手会拿沙箱和父 Agent 工具，返回 `ToolLoopAgent`，并在定义顶部展示模型和步骤预算
4. 保持错误处理作为字符串返回，而不是抛出的异常

实现提示：

- 这两个助手可以共享生成和格式化功能，使 `[Role: N steps]` 格式化集中在一个地方
- 不要过于抽象。两个助手和一个路由器就够了。当你有五个角色，而不是两个角色时，采用注册表与工厂系统才是正确的选择
- 描述就是父 Agent 里读的。WHEN TO USE 和 WHEN NOT TO USE 也适用于路由层，而不仅仅是单个工具

路由器结构

```
function buildExplorer(sandbox: Sandbox, parentTools: { read: any; grep: any }) {  
  return new ToolLoopAgent({
```

```

    model: "anthropic/claude-haiku-4-5",
    instructions: `You are an explorer agent. Investigate and report back concisely.
Working directory: ${sandbox.workingDirectory}`,
    tools: { read: parentTools.read, grep: parentTools.grep },
    stopWhen: stepCountIs(5),
  });
}

function buildExecutor(sandbox: Sandbox, parentTools: { read: any; grep: any }) {
  const executorBash = createBashTool(
    sandbox,
    createApproval({
      mode: "delegated",
      trust: ["npm test", "npm run build", "npx tsc"],
    }),
  );
  return new ToolLoopAgent({
    model: "anthropic/claude-sonnet-4-6",
    instructions: `You are an executor agent. Follow instructions precisely.
Working directory: ${sandbox.workingDirectory}
Do NOT ask questions. Do NOT explore beyond what's needed. Execute the task.`,
    tools: { read: parentTools.read, grep: parentTools.grep, bash: executorBash },
    stopWhen: stepCountIs(15),
  });
}

async function runSubagent(role: string, agent: ToolLoopAgent, description: string)
↪ {
  try {
    const { text, steps } = await agent.generate({ prompt: description });
    return text ? `[${role}: ${steps.length} steps]\n${text}` : `(no response from
↪ ${role})`;
  } catch (e: any) {
    return `${role} error: ${e.message}`;
  }
}

export function createTaskTool(
  sandbox: Sandbox,
  parentTools: { read: any; grep: any },
) {
  return tool({
    description: `Delegate work to a subagent.
Explorer (default): read-only research with Haiku. Use for searching across files,
understanding patterns, and gathering context.
Executor: implementation with Sonnet and delegated bash. Use for focused
changes with explicit instructions and a known verification step.

```

```
WHEN TO USE: research across many files (explorer), bulk implementation (executor).
WHEN NOT TO USE: ambiguous requirements (use askUser), architectural decisions
(the parent decides).
DO NOT USE FOR: single-step tasks the parent can do directly.`
  inputSchema: z.object({
    description: z.string().describe("Task instructions for the subagent"),
    subagentType: z
      .enum(["explorer", "executor"])
      .default("explorer")
      .describe("Subagent role"),
  }),
  execute: async ({ description, subagentType }) => {
    const agent =
      subagentType === "executor"
        ? buildExecutor(sandbox, parentTools)
        : buildExplorer(sandbox, parentTools);
    return runSubagent(subagentType, agent, description);
  },
});
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22

23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64

65
66
67
68

路由器现在有五条线路。其他都是按角色构建的。

生成权限的去向

现在任何 Agent 都可以调用 `task` 任何 `subagentType`。这对入门 Harness 来说没问题。在更分层的设置中，你需要一个按角色设置权限映射：

```
const SPAWN_PERMISSIONS: Record<string, string[]> = {
  orchestrator: ["explorer", "executor", "reviewer"],
  executor: ["explorer"],
  explorer: [],
};

function canSpawn(parentRole: string, subagentType: string): boolean {
  return SPAWN_PERMISSIONS[parentRole]?.includes(subagentType) ?? false;
}
```

1
2
3
4
5
6
7
8
9

支票放在 `execute` 顶。如果不允许生成，返回错误字符串并不要构建子 Agent 。

我们还没把它加到工作 Harness 里，因为父 Agent 目前还没有角色。当你开始使用子 Agent 本身调用 `task` 时，权限表就是你接下来需要的。在此之前，缺席没问题，形状也有记录。

按角色模型，而非按会话模型

模型是角色定义的一部分，而非全局环境：

职责	模型	为什么
探索器	Haiku	快速、便宜、只读
执行器	Sonnet	实现可靠

职责	模型	为什么
评论员（后期）	Opus	代码审查的深层推理
编曲者（后期）	Sonnet	多工具布线

不同的角色，不同的模式。不要只选一个模型到处用。成本差异在长期任务中会叠加，失败模式也不同。

**** 注意：两个角色是正确的起点 ****

你可以建立更复杂的层级结构：架构师、规划师、审查员、整合者。我们做这件事并不是因为两个岗位涵盖了大多数人在意的工作。当有真正需要的任务时，再加多一些。不要随意添加。每个角色都是指令漂移的新场所，也是新的模型账单需要跟踪。

动手试试

请让父 Agent 按顺序分配两项工作：先做调研，然后是实施：

```
bun run index.ts . "First, delegate to an explorer: find every file that uses the
↳ zod schema for tools. Then delegate to an executor: in those files, add a
↳ comment above each tool() call saying which lesson introduced it."
```

1

你应该会看到父 Agent 有两个任务调用。第一个返回文件列表。第二位负责编辑并反馈。

```
npx tsc --noEmit
```

1

提交

```
git add src/tools.ts
git commit -m "refactor(subagents): split task tool into router and role helpers"
```

1

2

完成标准

- ☐ `createTaskTool` 是一个按 `subagentType` 调度的精简路由器
- ☐ 每个角色都存在于一个独立的助手中（`buildExplorer`，`buildExecutor`）

- ☐ 任务工具描述中列出了两个角色以及使用它们的合适时间
- ☐ 错误以字符串形式返回，而非例外
- ☐ 增加第三个角色只是一个新的助手和一个新分支，仅此而已
- ☐ `npx tsc --noEmit`

**** 注意：添加审核员角色 ****

试着添加一个 `reviewer` 子 Agent：只读工具、Opus 级模型，以及一个 `verdict` 工具，能返回带有反馈的 `pass` 或 `fail`。执行器结束后，会自动生成一个审核员，显示原始任务和执行器的差异。如果评审失败，请重新运行执行器并附上反馈。重试次数上限为两次。哪种模型组合能产生最佳的评测质量？评审什么时候会草率放行，而不是抓住真正的问题？

参考实现

请看上面的路由器形状。练习解法是同样的代码，应用到你的 `src/tools.ts` 上。

状态机

模块 7 · 沙箱生命周期 · 第 25/39 页 · 原文: <https://agentharnesszh.vercel.app/state-machine>

状态机

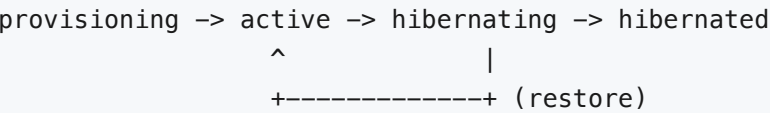
云沙箱并不只有“运行”和“停止”两种状态。它会依次经历四个状态，而状态之间的差别会直接反映在费用上。

应当把沙箱生命周期看作一台状态机。听起来很复杂，其实只需要四个状态、两个计时器和一个活动跟踪器。任何一块缺失或实现错误，往往都会在月底的账单上暴露出来。

学习成果

你将能够说出四个状态，解释两类超时如何推动沙箱在状态之间切换，并判断活动跟踪器应该计入或忽略哪些事件。

四种状态



- 1
- 2
- 3

状态	发生的事情	成本
配置中	VM 正在启动并安装依赖	已开始计费
活动	Agent 正在工作：运行命令、修改文件	全额按分钟计费
正在休眠	正在创建快照，沙箱即将结束运行	全额按分钟计费
已休眠	VM 已停止，快照已保存	仅产生存储费用

“活动”是成本最高的状态，“已休眠”则最便宜。中间的两个过渡状态（配置中和正在休眠）虽然持续时间不长，却仍按全额计费，因此应尽量缩短它们。

两类超时

每个云沙箱都有两个时钟。它们因不同原因到期，而你只能控制其中一个。

硬到期

服务商会设定最长生命周期，具体可能是一到四小时。时钟归零后，无论 VM 正在做什么都会被终止，而且你无法延长这一期限。你只能在到期前完成任务，或提前创建快照。

无活动窗口

无活动窗口由你设置。连续 N 分钟没有活动后，沙箱会进入休眠。对大多数 Agent 工作负载来说，五分钟是合理默认值；两分钟比较激进，沙箱可能在两个回合之间休眠；二十分钟则较宽松，但你需要为空闲时间付费。

```
const INACTIVITY_WINDOW = 5 * 60 * 1000;
```

1

硬到期决定最坏情况，无活动窗口决定日常成本，两者都必须正确处理。

哪些事件算作活动

活动跟踪器是无活动窗口正常工作的关键。实现方法很简单：每个“真实”活动都更新时间戳。难点在于界定哪些事件才算真实活动：

事件	算作活动?
用户发送聊天消息	是
执行工具调用	是
沙箱事件（文件写入、进程启动）	是
状态轮询	否
重连探测	否
健康检查	否

如果把状态轮询算作活动，沙箱将永远无法休眠，你会一直为空闲时间付费。如果工具调用反而不算活动，沙箱可能在任务执行过程中休眠，导致进行中的工作丢失。这两种失效模式都很常见。

沿时间线分析

```
0:00 User sends message, sandbox is provisioned
0:02 Sandbox active, agent starts working
0:05 Agent runs npm install (3 minutes)
0:08 Agent finishes, responds to user
0:13 Inactivity window expires (5 min since last activity)
0:13 Sandbox starts hibernating, snapshot in progress
0:14 Snapshot complete, sandbox hibernated
0:20 User sends another message, sandbox restores from snapshot
```

```
0:21 Sandbox active again, agent continues where it left off
1:30 Hard expiry hits, VM is killed
```

1
2
3
4
5
6
7
8
9
10

针对这条时间线，有两个问题值得回答：

1. Harness 应该在什么时候提醒用户硬到期即将到来？
2. 自动快照应在何时触发，以使用户在过期后继续使用？

答案取决于平台的存储费用，以及强制到期会在多大程度上打断 Agent 任务。这里没有唯一正确答案，但有明显错误的选择（什么都不做），也有较好的选择（在硬到期进度达到 80% 时自动创建快照）。

动手试试

这是一节概念课。请自我检查：

1. 凭记忆画出状态机，并标记每个过渡点
2. 用一句话解释为什么硬到期不能延长
3. 从活动表中选一个事件，解释为什么它算或不算

如果你已经在模块 4 接入生命周期钩子，那么构建真正的云后端时，就有位置挂接状态、`lastActivityAt` 和 `hardExpiryAt`。本地后端与 `just-bash` 后端不会真的触发这些超时，但整体结构已经确定。

提交

本课没有代码提交。本地沙箱暂时没有值得跟踪的真实超时。

完成标准

- ☐ 你可以按顺序说出这四个状态
- ☐ 你可以描述硬到期和无活动窗口之间的区别
- ☐ 你可以区分哪些算作活动，哪些算作噪音
- ☐ 你可以沿着上面的时间线追溯，并解释自动快照何时触发

快照与恢复

模块 7 · 沙箱生命周期 · 第 26/39 页 · 原文: <https://agentharnesszh.vercel.app/snapshot-and-restore>

快照与恢复

快照会冻结沙箱文件系统并返回一个 ID。恢复会从该 ID 创建一个新沙箱。文件相同，状态相同，虚拟机不同。

机制很简单。出错的地方却没有。这三者都接近幂等性，即调用某事两次的结果与调用一次相同的性质。沙箱充满了你真的不想在不考虑的情况下重复调用的操作。

学习成果

你可以描述 **snapshot** 保护什么、**restore** 创建了什么，并识别生产生命周期代码中出现的三种幂等性危害。

API

```
const { snapshotId } = await sandbox.snapshot!();  
  
const restored = await createCloudSandbox({ snapshotId });
```

1
2
3

那只是表面。所有有趣的事情都围绕着它发生。

快照保存了什么

快照捕捉文件系统状态。 **/workspace** 的内容（或者你沙箱根的地方）、任何已安装包的状态、 **Agent** 创建的任何文件。它们不会捕捉正在运行的进程、在运行中的网络连接或内存状态。一个运行已经过编译步骤一半的构建，在还原后不会从中断处继续。文件系统快照；正在进行的工作则没有。

这对 **Agent** 的心理模型很重要。恢复后， **Agent** 会保留之前所有的文件，但如果快照触发时正在运行测试，测试必须重新运行。

三大幂等性险

1. 快照正在进行中

用户（或生命周期 workflow）在快照正在运行时触发快照。没有守卫，你会看到两个快照争夺同一个虚拟机，或者出现提供者错误，或者一个看起来有效但实际上不存在的部分快照。

```
let inFlight: Promise<{ snapshotId: string }> | null = null;

snapshot: async () => {
  if (inFlight) return inFlight;
  inFlight = vm.snapshot();
  try {
    return await inFlight;
  } finally {
    inFlight = null;
  }
},
```

1
2
3
4
5
6
7
8
9
10
11

缓存承诺，第二调用天还回，工作完成后清除。

2. 沙箱已经在还原中运行

用户重新连接到已经活跃的会话，Harness 会在会话上触发恢复。现在你有两个虚拟机。新车是在浪费钱。旧的那台还在为交通服务。

```
async function attachOrRestore(sessionId: string, snapshotId: string) {
  const existing = await findActiveSandbox(sessionId);
  if (existing) return existing;
  return createCloudSandbox({ snapshotId });
}
```

1
2
3
4
5

创作前先看清楚。恢复路径应先检查是否有活跃沙箱。

3. 双音

`stop` 由非活动计时器调用一次，用户调用一次。或者一次在休眠，一次在硬到期。第二个调用遇到的沙箱已经没了，根据供应商不同，要么大声故障，要么悄无声息地损坏状态。

```
let stopped = false;

stop: async () => {
  if (stopped) return;
  stopped = true;
  await vm.close();
},
```

1
2
3
4
5
6
7

一个布尔值就足够了。关键是第二个 `stop` 没有做任何事，而不是直接攻击死机。

恢复解决不了的问题

快照是时间中的一个瞬间。从昨天的快照恢复，仍然会得到昨天的代码、昨天的依赖和昨天的环境。如果项目已经转移（新提交、新包、新环境变量），恢复后的沙箱就是化石。

生产生命周期通过在项目变更时失效快照，或在恢复后重运行安装钩子（模块 4 `afterStart`）来句柄这一点。这两者都不是自动的。两者都必须刻意接线。

**** 注意：当地沙箱可以伪造这个 ****

本地后端没有真正的快照机制，但你可以用 `git stash` 或 `tarball` 来绘制一个快照工作目录。它不等同于云快照（没有虚拟机状态，没有安装缓存），但它教会了接缝。如果你想玩玩形状，那是个不错的起点。

动手试试

这是一个概念课。自我检视：

1. 不要回头，说出三个幂等性危险
2. 用你自己的话写下 `snapshot` 保存和不保存的内容
3. 用序列图勾勒 `attachOrRestore` 流程。活跃沙箱检查相对于快照查找在哪里进行？

如果你想本地玩玩形状，可以在本地沙箱添加一个 `snapshot`，运行 `git stash push` 并返回仓库引用作为快照 ID。语义上有问题（真实快照会冻结所有内容，不仅仅是追踪文件），但接缝是真实存在的。

提交

这节课里没有代码，除非你是在绘制本地快照。如果需要，建议把它提交到另一个分支，这样就不会和工作 Harness 纠缠。

完成标准

- ☐ 你可以描述快照保留哪些内容，哪些不保留
- ☐ 你可以说出三个幂等性危险和每个的防守模式
- ☐ 你可以解释为什么恢复后的快照仍可能需要 `afterStart` 重新运行

持久化 workflow

模块 7 · 沙箱生命周期 · 第 27/39 页 · 原文: <https://agentharnesszh.vercel.app/durable-workflows>

持久化 workflow

沙箱生命周期想做点简单的事情。每三十秒检查沙箱是否已闲置足够长以进入休眠。如果是，就截图并停止。如果没有，再等三十秒。

在一个长期运营的服务器里，那是个 `setInterval`，然后你就回家了。在无服务器中，函数在一两分钟后会失效，并带走计时器。沙箱继续运行，你继续付费，而你的生命周期代码却被放在某个已经不存在的进程里。

你需要持久的基础设施。一个能跨越功能边界沉睡，并在另一端继续存在的东西。Vercel Workflow 就是这样做的。即使使用不同的运行时，模式也很重要。

学习成果

你可以解释为什么 `setTimeout` 在无服务器环境中沙箱生命周期内失败，也可以勾勒出一个轮询沙箱、空闲时快照、部署后恢复的持久化 workflow 循环。

`setTimeout` 的问题

```
setTimeout(() => checkAndSnapshot(), 30_000);
```

1

这在本地有效。它在无服务器中不工作。

调用 `setTimeout` 的函数返回。运行时会清理函数的进程。超时时间是垃圾回收。支票从未运行。沙箱还在继续运转。你的月账单会显示无限沙箱运行时间的费用。

即使函数在第一次检定前存活足够久，也不能保证下一次部署没有替换它。每次重新部署时，计时器状态都会丢失。

工作流程模式

Vercel Workflow 揭示了一个不依赖于宿主进程存活的 `sleep()`：

```
"use workflow";
import { sleep } from "workflow/sleep";

const POLL_INTERVAL = 30;
const INACTIVITY_WINDOW = 5 * 60;

export async function sandboxLifecycle(sandboxId: string) {
```

```
while (true) {  
  await sleep(POLL_INTERVAL);  
  
  const status = await checkSandboxStatus(sandboxId);  
  
  if (status === "expired") {  
    break;  
  }  
  
  if (status.lastActivity + INACTIVITY_WINDOW < Date.now() / 1000) {  
    await snapshotAndStop(sandboxId);  
    break;  
  }  
}  
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22

`sleep(30)` 不会暂停功能。它会检查工作流程到耐用存储和退货。三十秒后，工作流程会从上次中断的地方恢复，无论在哪个函数实例中。跨部署。跨主机重启。无论平台在底层做什么。

这就是诀窍。循环体是普通代码。`sleep` 就是魔力。

阶梯边界

在工作流程中，调用外部系统（提供者 API、数据库、任何副作用）都存在于 **"use step"** 功能中：

```
"use step";

export async function checkAndSnapshotStep(sandboxId: string) {
  const sandbox = await getSandbox(sandboxId);
  if (!sandbox.isActive) return { action: "stop" };

  const idle = Date.now() - sandbox.lastActivityAt;
  if (idle > INACTIVITY_WINDOW) {
    await sandbox.snapshot();
    await sandbox.stop();
    return { action: "hibernated" };
  }

  return { action: "continue" };
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

step 函数在瞬态失败时重试，成功时缓存。工作流程循环调用它们像正常功能一样。运行时间让耐用性得以实现。

成本数学

节省多少取决于你平时的闲置时间。一个合理的折衷情况：

设置	行为	每次会谈费用
没有生命周期	沙箱运行至硬到期（4 小时）	4 小时 x 0.02 美元/分钟 = 4.80 美元
基于不活动的休眠	沙箱闲置 5 分钟后休眠	25 分钟 x \$0.02/分钟 = \$0.50

长时间的训练大约是一个数量级。节省的费用会随着用户和时间的积累而复利。

**** 注意：该模式能持续运行 ****

Vercel Workflow 是一个实现方式。时间是另一种。AWS Step Functions 也是一个例子。原理是一样的：能够通过函数边界存活的睡眠，即使你没有，也能像写一个长时间运行的进程一样编写生命周期代码。如果你用的是不同的运行时，找 `sleep()` 和步进函数的等价物。只有当你真的想做时，才自己卷。

演示版戛然而止

本课程的本地和 `just-bash` 后端不运行持久化工作流。他们不需要。生命周期就是过程生命周期。没有什么不活跃的存在可以冬眠。

模块 4 的生命周期钩子就是云后端的定位。`afterStart` 会启动持久化工作流。`beforeStop` 会让工作流程结束。工作流程本身会通过同一个 `Sandbox` 界面调用回到沙箱，调用 `snapshot()` 和 `stop()`，就像其他消费者一样。

这就是界面设计的原因。同步的进程中世界（本地）和异步多重部署的世界（云）都放在同一个表面之下，因为工作流运行句柄最难的部分。

动手试试

这是一个概念课。自我检视：

1. 请用一句话解释为什么 `setTimeout` 在无服务器的生命周期沙箱失败
2. 追踪工作流程循环，列出部署中所有可能检查 `sleep` 的地方
3. 计算一下你实际运行过的工作负载节省了多少。你每次游戏的平均闲置时间是多少？

提交

这节课没有代码。下一课列出了即使持久化工作流接线正确，仍会被咬的生产陷阱。

完成标准

- ☐ 你可以解释为什么 `setTimeout` 在无服务器中不起作用
- ☐ 你可以画出工作流程流程，识别耐用性接缝
- ☐ 你可以根据自己选择的工作量做成本计算

实战经验

模块 7 · 沙箱生命周期 · 第 28/39 页 · 原文: <https://agentharnesszh.vercel.app/hard-won-lessons>

实战经验

以下五种情况都导致了真正的停机、成本飙升或真正的工作损失。一旦你看过它们，它们就很明显了。在那之前这些问题并不明显，所以才会一直发生。

这些数据来自运行生产 Agent 对抗云端沙箱的团队。这些都不是理论上的。这种模式在供应商、平台和实施中反复出现。

学习成果

你可以列出五个生产品生命周期的陷阱，描述每个的失败模式，并应用修复模式。

重新连接后句柄陈旧

你重新连接到已有的沙箱。句柄是你之前用的那个，或者是从录音室录音重建的 Harness。无论哪种情况，命令流都坏了。命令输入，垃圾输出，或者调用永远挂着。

句柄在断线后幸存下来。里面的会话不会。

Fix： 使用前先探测重新连接的句柄。

```
const sandbox = await reconnect(sandboxId);
const probe = await sandbox.exec("echo probe");
if (probe.exitCode !== 0 || probe.stdout.trim() !== "probe") {
  sandbox = await createFromSnapshot(lastSnapshotId);
}
```

1
2
3
4
5

探针是只读且快速的。每次重新连接前运行一个 Agent 的成本远低于与死句柄通信的成本。

过期到期数据

沙箱报告 `expiresAt` 创建时间。如果你缓存了这个值，之后再对照，你是在对比存储时已经很旧的数据。更糟的是，如果你在缓存失效后 API 把衍生值（`remainingTimeout = expiresAt - now()`）传递给提供者，可能会意外创建一个已过期的沙箱。

Fix： 在生命周期决策前，务必向提供者索取新的有效期。

```
const { expiresAt } = await sandbox.getStatus();
if (expiresAt < Date.now()) {
  await beforeStop?.(sandbox);
}
```

1
2
3
4

缓存的过期信息用于显示，而非控制流。

民调重置不活跃

你的生命周期工作流程每三十秒轮询一次沙箱状态。如果状态检查被计为活动，无活动窗口永远不会关闭。沙箱持续到硬到期。账单到了。

这是一个伪装成集成问题的纯函数漏洞。修复方法同时存在两个方面：活动跟踪器必须忽略状态调用，状态调用必须小心避免触发活动编码事件。

Fix： 活动跟踪器只统计用户主动发起的工作。

```
function recordActivity(event: SandboxEvent) {
  if (event.kind === "user_message" || event.kind === "tool_call" || event.kind ===
  ↩ "fs_change") {
    sandbox.lastActivityAt = Date.now();
  }
}
```

1
2
3
4
5

状态 ping、健康检查、重新连接探针、计费显示：这些都不会重置定时器。

自动恢复循环

用户重新连接。沙箱会自动从上一次快照恢复。自动恢复触发生命周期检查，尚未发现任何活动，决定进行快照。快照触发休眠。休眠触发下一次自动恢复。

你把两段单独看起来正确的代码组成了一个无限循环。

Fix： 仅在初次进入时自动恢复。后续的重连会加入主动沙箱。

```
if (isInitialEntry && sandbox.state === "hibernated") {  
  await restore(sandbox.snapshotId);  
}
```

1

2

3

状态机是你的好朋友。如果沙箱已经激活，挂上去是正确的选择。如果是休眠状态，就恢复。如果是其他状态，就等一等，否则失败。不要自动连锁过场。

状态分歧

沙箱状态存在三个地方：提供商的 API、你的数据库、客户端的本地缓存。它们会分道扬镳。你向用户展示的内容有时会出错，而你信任的平台决定了它是以让你花钱的方式错了，还是以让你失去信任的方式错了。

Fix： 提供者 API 是真相的来源。从那里导出所有展示的内容。

```
const { state } = await provider.getSandboxStatus(sandboxId);  
ui.showState(state);
```

1

2

你的数据库就像一个缓存。客户端缓存就是缓存。事实也不是这样。有疑问时，去取。

****Warning**： 组合比 individuals** 还糟

每个陷阱本身就很糟糕。昂贵的 bug 来自于将它们组合起来。陈旧的句柄加上民调计入活动跟踪器意味着你一直在为一个无法沟通的沙箱买单。发散缓存加上自动恢复循环意味着你为一个用户创建了三个重复的沙箱。纵深防守是正确的立场。修复全部五个，即使其中一两个在你的环境中看起来不太可能。

动手试试

这是一个概念课。自我检视：

1. 不回头，说出五个陷阱
2. 先选最有可能咬你环境的那种。为什么？
3. 每遇到一个陷阱，画出失败时间线。哪里有固定门能抓住它？

如果你是朝真正的云后端构建，先把修复门写进生命周期钩子里，再写云端沙箱。门对本地后端没有影响，但现在加装比以后改装便宜。

提交

这节课没有代码。

完成标准

- ☐ 你可以说出全部五个陷阱
- ☐ 你可以描述每种的固定模式
- ☐ 你可以识别模块 4 生命周期钩子中每个修复会插入哪些门

**** 注意：构建混沌模式 ****

生产沙箱故障和本地开发失败看起来是不同的。为测试运行构建一个 `--chaos` 标志，每次会话随机注入一次失败：命令中途终止沙箱进程、重新连接时返回过时的句柄、强制缓存和提供者之间状态分歧，或者跳过一次状态更新。把你的全部 **Agent** 循环都用混沌模式运行。首先出问题的是你忘了防御的陷阱。

结构化提问

模块 8 · 人机协作 · 第 29/39 页 · 原文: <https://agentharnesszh.vercel.app/structured-questions>

结构化提问

Agent 不会问你问题。单靠这个就不行了。

你可以构建一个 `askUser` 工具。Agent 会看到它，阅读描述，然后继续不使用。在开发者聊天中训练的模型吸收了大量“让我帮你弄明白”的能量。他们觉得问很软弱。他们宁愿猜测。

修复分为两部分。工具很小，系统提示词则负责告诉 Agent 询问是正确的选择。

学习成果

这是一个 `askUser` 工具，可以回答一个问题 and 2 到 4 个选项，还有一个系统提示词里的 `# Handling Ambiguity` 部分，可以写出什么时候使用。模糊的话提示词触发工具。具体的提示词不支持。

快速路径

1. 加一个 `askUser` 工具，带 `question` 和 `options`（2 到 4 根弦）
2. 在 `buildSystemPrompt` 中添加一个 `# Handling Ambiguity` 部分，告诉 Agent 先搜索，然后询问，再行动
3. 用两个提示词验证：一个模糊，一个具体

动手练习 8.1

制作工具，添加提示词部分，确认两半都正常工作。

要求：

1. 用上面的 schema 加 `askUser` 到 `src/tools.ts`
2. 在 `execute` 中，将选项格式化为编号列表，并返回模型将传递给用户的字符串
3. 再加上 `# Handling Ambiguity` `buildSystemPrompt`。告诉 Agent：先搜寻，第二问，第三行动
4. 运行两个提示词：一个是歧义的（“添加认证”）和一个特定的（“在 `auth.ts` 的第 42 行添加一个空检查”）。确认只有第一个触发 `askUser`

实现提示：

- `askUser` 工具的 `execute` 其实并不等用户。它返回一个描述问题和选项的字符串。围绕其的 Harness（或读取输出的用户）在下一回合提供答案
- 系统提示词脚本的工作比这里的工具描述还多。如果没有提示词部分，模型也会把 `askUser` 当作可选，即使提示词很模糊
- 提示词中通常有两个例子就足以固定模式。别让它过载

工具

```
import { tool } from "ai";
import { z } from "zod";

export function createAskUserTool() {
  return tool({
    description: `Ask the user a multiple-choice question.
    WHEN TO USE: scoping ambiguous tasks, choosing between approaches,
    resolving a missing detail before acting.
    WHEN NOT TO USE: you already have enough context to proceed.
    DO NOT USE FOR: rhetorical questions or progress updates.`,
    inputSchema: z.object({
      question: z.string().describe("The question to ask the user"),
      options: z
        .array(z.string())
        .min(2)
        .max(4)
        .describe("Two to four options for the user to pick from"),
    }),
    execute: async ({ question, options }) => {
      const formatted = options.map((o, i) => `${i + 1}. ${o}`).join("\n");
      console.log(`\nQuestion: ${question}\n${formatted}\n`);
      return `Asked: "${question}"\nOptions:\n${formatted}\n\n(Awaiting user
↪ response.)`;
    },
  });
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

17
18
19
20
21
22
23
24
25

工具会打印问题和选项 `stdout`（让用户看到），并返回相同的字符串内容（模型在消息历史中看到）。模型知道问题正在进行中，不会假设问题已经被回答。

系统提示词的增建

```
sections.push(`
# Handling Ambiguity
When the task is ambiguous or has multiple valid approaches:
1. Search the code or docs to gather context first
2. Use askUser to let the user choose. Do NOT guess.
3. Examples: "add auth" -> ask OAuth or JWT; "set up a db" -> ask Postgres or SQLite

Specific tasks (with file paths, line numbers, or precise instructions) do not
need askUser. Act directly.`);
```

1
2
3
4
5
6
7
8
9

编号协议很重要。“搜索、询问、行动”为模型提供了可遵循的顺序。没有它，**Agent** 要么提得太早（在没有足够上下文让问题有用之前），要么提问太晚（已经开始构建错误的答案）。

****Warning：** 模型更愿意探索而非 `ask`

即使有协议，**Agent** 还是会读取三四个文件后才抽出 `askUser`。这是正确的行为，因为第一步是“先搜索”。这点仍然值得知道，因为如果你看着它运行并感到不耐烦，模型并没有忽视你。这需要收集背景，以便提出一个有用的问题。

如果 `bash` 被审批阻挡，**Agent** 就无法执行获取上下文所需的命令。可能永远都达不到第二步。审批系统

和 `askUser` 处于紧张状态，这种紧张是真正的架构摩擦，而非需要修复的漏洞。

接入工具

```
const tools = {
  read: createReadTool(sandbox),
  grep: createGrepTool(sandbox),
  bash: createBashTool(sandbox, createApproval({ mode: "interactive" })),
  task: createTaskTool(sandbox, { read, grep }),
  askUser: createAskUserTool(),
};
```

1
2
3
4
5
6
7

Agent 的工具清单现在包括 `askUser`。系统提示词告诉它什么时候使用。用户终端显示问题。模型的消息历史显示该问题正在等待中。

动手试试

运行一个模糊的任务，观察 Agent 提出的问题：

```
bun run index.ts . "Add authentication to this project"
```

1

你应该看到 Agent 读取几个文件，然后调用 `askUser` “我应该用哪种认证策略？”以及“OAuth”、“JWT”、“会话 cookies”等选项。终端打印出问题。模型坐着等待。

现在运行一个具体任务，确认 Agent 没有要求：

```
bun run index.ts . "Add a null check at line 42 of src/auth.ts before the database  
↪ query"
```

1

Agent 应该直接做出改变。没有 `askUser` 调用。

```
npx tsc --noEmit
```

1

提交

```
git add src/tools.ts src/system.ts index.ts
git commit -m "feat(askUser): add structured question tool with ambiguity protocol"
```

1

2

完成标准

- ☐ `askUser` 工具是有线的，支持 2 到 4 种选项
- ☐ `# Handling Ambiguity` 部分在系统提示词
- ☐ 模糊提示词触发 `askUser`
- ☐ 具体的提示词不行
- ☐ `npx tsc --noEmit`

**** 注意：让问题记住 ****

现在返回字符串 `askUser` 模型继续。在真实的 Harness 中，Harness 会暂停，收集用户的选择，并作为下一条用户消息传回。画出那个停顿是什么样子。Harness 在哪里拦截了工具调用的结果？用户的回答会入对话的哪里？模块 8.2 的事件方式自然是将这一点连接起来的自然地点。

参考实现

```
export function createAskUserTool() {
  return tool({
    description: `Ask the user a multiple-choice question.
    WHEN TO USE: scoping ambiguous tasks, choosing between approaches,
    resolving a missing detail before acting.
    WHEN NOT TO USE: you already have enough context to proceed.
    DO NOT USE FOR: rhetorical questions or progress updates.`,
    inputSchema: z.object({
      question: z.string().describe("The question to ask the user"),
      options: z
        .array(z.string())
        .min(2)
        .max(4)
    })
  })
}
```

```
        .describe("Two to four options for the user to pick from"),
    },
    execute: async ({ question, options }) => {
        const formatted = options.map((o, i) => `${i + 1}. ${o}`).join("\n");
        console.log(`\nQuestion: ${question}\n${formatted}\n`);
        return `Asked: "${question}"\nOptions:\n${formatted}\n\n(Awaiting user
↪ response.)`;
    },
});
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22

审批配置

模块 8 · 人机协作 · 第 30/39 页 · 原文: <https://agentharnesszh.vercel.app/approval-config>

审批配置

你已经有审批系统了。模块 2 构建了可辨识联合类型，包含 `interactive`、`background` 和 `delegated` 模式。这回答了一个问题: *who* 决定

它没有回答另一个问题: 具体适用 *what* 策略

CI 运行时使用 `mode: "background"` 自动审批命令。好吧。但你也要屏蔽任何写入 `.env`，无论用什么模式。你希望 `bash` 命令被更严格的操作系统级沙箱包裹，无论使用什么模式。这些规则不符合可辨识联合类型。他们住在一层之下。

本课奠定了第二层（基于事件的拦截），并展示了两种模型的交汇点。事件图层在构建过程中是概念性的，但现在就值得看看形状，这样你需要时可以接线。

学习成果

你可以描述两种审批模式，识别哪一种适合哪种使用场景，并解释它们如何在生产 Harness 中组合使用。

两个模型

配置模型就是你现在的配置：

```
type ApprovalConfig =  
  | { mode: "interactive" }  
  | { mode: "background" }  
  | { mode: "delegated"; trust: string[] };
```

1
2
3
4

设定在启动时。在一次治疗过程中不会改变。答案 *who decides* 。

事件模型是下面的层：

```
harness.on("tool_call", async (event) => {  
  const { toolName, input } = event;  
  
  if (toolName === "write" && input.path.endsWith(".env")) {  
    return { block: true, reason: "Cannot modify .env files" };  
  }  
});
```

```
    }

    if (toolName === "bash") {
      event.input.command = `sandbox-exec -p '(deny default)' ${input.command}`;
    }

    return { block: false };
  });
```

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11
- 12
- 13

每工具调用都有火。扩展可以阻挡、修改或穿透。答案 *what* 策略 *apply* 。

何时使用哪种

使用场景	配置	活动
CI 运行，自动审批所有内容	<code>mode: "background"</code>	过度杀戮
子 Agent 从父 Agent 那里继承信任	<code>mode: "delegated"</code>	关卡错了
块写入特定文件	太粗糙了	文件级策略
操作系统级沙箱中的封装命令	无法修改输入	输入修改
项目专用安全规则	仅限全球	每个项目扩展

配置层是整个会话的操作模式。事件层用于细粒度、通常针对项目特定且常可插拔的策略。它们有些重叠。它们不会互相替代。

它们如何结合

真正的 Harness 同时使用：

```
const approval = createApproval({ mode: "interactive" });

harness.on("tool_call", async (event) => {
  if (event.toolName === "write" && event.input.path.endsWith(".env")) {
    return { block: true, reason: "Protected file" };
  }
});
```

1
2
3
4
5
6
7

配置里写着“交互模式，人工审批”。事件处理者说：“无论人类如何审批，都不要碰 `.env`。”该事件在配置后但工具运行前触发。纵深防御。

这很重要，因为操作模式和策略往往来自不同的地方。模式由负责控制 Harness 的人决定（CI、开发人员、委派子 Agent）。策略由项目设定（`.env` 敏感，构建目录为唯读，任何涉及生产凭证的内容都需要系统层面的沙箱）。一个配置旋钮无法同时处理两种决策而不容易打结。

**** 注意：事件层插入模块 11 ****

我们将在第 11 模块的扩展性工作中构建实际事件总线，生命周期事件是主要扩展点。审批事件是生命周期事件的一种特定类型。一旦总线存在，审批拦截器就是几行代码，订阅 `tool_call`。

随行教学缺少什么

你目前构建的 Harness 包含配置层。它还没有事件图层。没关系。配置层涵盖了课程需要教授的大部分情况。

添加事件图层的做法如下：

1. 在 Harness（模块 11）中构建一个小型类型事件发射器
2. 在每个工具运行前，从 Agent 循环发出 `tool_call` 事件
3. 让订阅者返回 `{ block, reason }` 或修改输入
4. 把一个阻止写入硬编码文件的用户接线，作为烟雾测试

工作量不大。它不在这个模块里，因为前提条件（事件、扩展）属于扩展性故事的其他部分。当你进入模块 11 时，基于事件的审批成为事件总线有用的唯一具体例子。

动手试试

这是一个概念课。自我检视：

1. 对于上表中的五个用例，请决定哪种模式最适合
2. 勾勒出一个两种模式同时适用的单一用例。每一层都做什么？
3. 找出你参与过的一个项目，事件审批本来会发现真正的错误。规则会是什么？

提交

这节课没有代码。事件层出现在模块 11 。

完成标准

- ☐ 你可以描述配置方法及其解答
- ☐ 你可以描述事件的方法及其解决方案
- ☐ 你可以根据具体用例选择哪一个
- ☐ 你可以勾勒出两者如何结合进行纵深防御

**** 注意：建立一个风险评分自动审批系统 ****

二元认可和拒绝的做法很粗糙。试试一个 `riskScore(command)` 函数，返回一个从 0 到 100 的数字。评分因子：写入磁盘加 30，网络访问加 20，文件删除加 50，修改配置加 40，只读 0。设定一个门槛，比如说 40。下面是自动审批的。用户提示词上方。把每次自动审批记录下来，方便之后审计。添加 `--risk-threshold` 标志，方便用户调整舒适度。现在想办法用不同于 `rm -rf /` 的方式给 `rm -rf /tmp/test` 评分，而不只靠关键词评分。

Todo 工具

模块 9 · 规划与验证 · 第 31/39 页 · 原文: <https://agentharnesszh.vercel.app/todo-tool>

Todo 工具

如果你给 Agent 一个复杂的任务并观察它运作，你会发现它做的事情和人类在压力下做的一样：它同时启动五件事，一个都没完成，然后解释它即将做的事情。

解决办法和人类同样有效。列个清单。选一个。把它做完。划掉它。选下一个。

待办工具就是那个清单，Agent 有一条规则无法反驳：一次只有一项正在进行中。

学习成果

一个 `todo` 工具，包含 `add`、`start`、`complete` 和 `list` 动作，并由内存列表中支持。多步骤任务被拆解并跟踪。单步任务则完全跳过了该工具。

快速路径

1. 添加一个 `todo` 工具，包含 `add`、`start`、`complete` 和 `list`
2. 用 `pending`、`in_progress` 和 `completed` 状态跟踪内存数组中的项目
3. 当另一个项目已经在进行中时拒绝 `start`

动手练习 9.1

构建工具并验证一次激活一个的约束。

要求：

1. `todo` 工具接受 `action` 枚举和可选的 `description` 和 `id`
2. `add` 创建一个带有简短生成 ID、`pending` 状态和描述的新项目
3. `start` 如果有其他物品被 `in_progress`，则会拒绝。否则它会将命名的项目设置为 `in_progress`
4. `complete` 标记了命名的项目 `completed`
5. `list` 返回多行字符串，带有状态标签

实现提示：

- 状态存在于模块范围内。同一个 Agent 系列共用一个列表。新一轮的开始
- `crypto.randomUUID().slice(0, 8)` ID 就足够用于存储列表中的列表。你不需要串口计数器
- 请明确拒绝信息：“正在处理：[id] 描述。先完成它。”

工具

```
interface TodoItem {
```

```
  id: string;
  description: string;
  state: "pending" | "in_progress" | "completed";
}

const todos: TodoItem[] = [];

export function createTodoTool() {
  return tool({
    description: `Manage a task list for multi-step work.
    WHEN TO USE: tasks with 3+ steps, multiple files, or dependencies between
    changes. Plan once, then track progress as you go.
    WHEN NOT TO USE: single-file fixes, simple questions, exploratory reads.
    DO NOT USE FOR: status updates to the user (just answer them directly).`,
    inputSchema: z.object({
      action: z.enum(["add", "start", "complete", "list"]),
      description: z.string().optional(),
      id: z.string().optional(),
    }),
    execute: async ({ action, description, id }) => {
      if (action === "add") {
        const item: TodoItem = {
          id: crypto.randomUUID().slice(0, 8),
          description: description ?? "(unnamed)",
          state: "pending",
        };
        todos.push(item);
        return `Added: [${item.id}] ${item.description}`;
      }

      if (action === "start") {
        const active = todos.find((t) => t.state === "in_progress");
        if (active) {
          return `Already working on: [${active.id}] ${active.description}.
↪ Complete it first.`;
        }
        const next = todos.find((t) => t.id === id);
        if (next) {
          next.state = "in_progress";
          return `Started: [${next.id}] ${next.description}`;
        }
        return `No todo with id ${id}.`;
      }

      if (action === "complete") {
        const item = todos.find((t) => t.id === id);
        if (item) {
```

```
        item.state = "completed";
        return `Completed: [${item.id}] ${item.description}`;
      }
      return `No todo with id ${id}.`;
    }

    return todos
      .map((t) => `[${t.state}] ${t.id}: ${t.description}`)
      .join("\n") || "No todos.";
  },
});
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30

31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59

单主动规则是承重部分。没有它，Agent 会从前方开始所有物品，然后并行快速处理，失去对每个物品的关注。

接入主流程

```
const tools = {  
  // ...everything else  
  todo: createTodoTool(),  
};
```

1
2
3

4

系统提示词的 Agent 和护栏部分已经引导 Agent 走向表演。工具的 WHENUSE 描述告诉它什么时候先计划，而不是直接行动。

什么时候该计划，什么时候不该

先做计划	跳过计划本
完成任务需 3 步或更多步骤	一次文件变更，位置已知
多个文件受影响	一个不需要文件的简单问题
变更之间的依赖关系	探索阶段，尚未有明确结果
用户要求多部件功能	带有精确错误信息的修复

如果 Agent 为一个一句拼写错误列出待办事项清单，那说明描述过于激进。把 WHEN NOT 拧紧到 USE 。

** 注意：这份名单被刻意留在记忆中 **

todos 数组不会在多次运行中持续存在。这是故意的。跨场次的长期列表往往会变成一个陈旧物品的垃圾抽屉。如果你以后想要持久化，可以在会话结束时将列表快照到文件。不要带着陈旧的 in_progress 物品进入新游戏，因为 Agent 不记得它们为何开始。

动手试试

执行一个多阶段任务，观察规划过程：

```
bun run index.ts . "Add a 'verify' npm script that runs typecheck, lint, and tests
↪ in sequence. Then run it and report the result."
```

1

你应该先看 Agent 调用 todo add 两三次来制定计划，然后在项目处理过程中 todo start 和 todo complete。清单上绝不应该有两个 in_progress 项。

运行一个简单任务来确认 Agent 跳过了该工具：

```
bun run index.ts . "What does the cwd variable in src/sandbox-local.ts do?"
```

1

Agent 应该在不调用 todo 的情况下回答。如果一题就成了待办事项，说明描述太急切了。

```
npx tsc --noEmit
```

1

提交

```
git add src/tools.ts index.ts
git commit -m "feat(planning): add todo tool with single-active constraint"
```

1

2

完成标准

- ☐ `todo` 工具有接线，可以接受 `add`、`start`、`complete`、`list`
- ☐ 一次只能 `in_progress` 一件物品
- ☐ 多步骤任务被分解
- ☐ 单步任务不会触发工具
- ☐ `npx tsc --noEmit`

**** 注意：添加依赖 ****

目前物品是独立的。试着给每个项目添加 `dependsOn: string[]`，列出必须先完成的项目 ID。如果任何依赖仍在处理或进行中，`start` 操作应当拒绝。现在多步任务可以表达真实的排序：“重命名函数”依赖于“找到每个调用者”。这从哪里开始让人觉得有些过头？

参考实现

见上 `createTodoTool`。锻炼方案是同样的代码，应用到你的 `src/tools.ts` 上。

快速理解上下文

模块 9 · 规划与验证 · 第 32/39 页 · 原文: <https://agentharnesszh.vercel.app/fast-context-understanding>

快速理解上下文

看一个新版 Agent 做真实任务，你会发现它每次都做同样的事情。请阅读 `package.json`。请阅读 `tsconfig.json`。阅读入门点。 `src/` 里每个文件都要仔细看。二十步之后，它才开始思考实际的工作。

这就是“先读完再行动”的策略。感觉很全面。它污染了背景，消耗了预算，还没让 Agent 做任何有用的事情就被忽视了。

解决办法很简单：先搜索，后阅读，第三行动。整个变化都存在于系统提示词中，Agent 从中养成了习惯。

学习成果

系统提示词的机构部分引导该 Agent 在未命名相关文件时 `read`grep`。多文件任务只需几步就能解决，而不是几十步。

快速路径

1. 在机构部分加两行：先搜索再阅读，“以防万一”不要先看文件
2. 用真实任务验证：Agent 应该 `grep` 模式，然后只 `read` 匹配的
3. 确认直接提交的问题仍然直接问 `read`

动手练习 9.2

引导 Agent 在提示词中以 `grep` 为先探索。

要求：

1. `buildSystemPrompt` 更新 `# Agency` 部分，新增两个项目符号：
 - 阅读前先搜索。先用 `grep`，然后只 `read` 你要改的部分
 - 不要“以防万一”去看文件。你需要什么就读什么
2. 运行一个任务，提示词中没有提及相关文件。先确认 Agent 使用 `grep`
3. 运行一个命名特定文件的任务。确认 Agent 直接通向 `read`

实现提示：

- 变化在于 `src/system.ts`，而非任何工具。工具没有改变
- Agent 区是行动策略的所在。这就是行动策略
- 不要把这个加到 `grep` 工具描述里。它应该属于 Agent 的策略层面，而不是工具本身

提示词的增建

```
sections.push(`
# Agency
- USE your tools. Read files, search code, run commands, then answer.
- Do NOT explain what you WOULD do. Actually do it.
- Available tools: ${ctx.toolNames.join(", ")}
- Search before reading. Use grep first, then read only what you'll change.
- Don't read files "just in case." Read what you need when you need it.`);
```

1
2
3
4
5
6
7

两颗子弹。这就是策略的全部变化。

实际模式

关于“在授权路由中添加速率限制”的简单（缓慢）流程：

```
read package.json
read tsconfig.json
read src/index.ts
read src/routes/index.ts
read src/routes/auth.ts
read src/routes/users.ts
read src/middleware/index.ts
... 20 more files
start implementing
```

1
2
3
4
5
6
7
8
9

同一提示词上的 **grep** 优先（快速）流：

```
grep pattern: "router\.post.*auth|router\.get.*auth"
  -> matches src/routes/auth.ts
read src/routes/auth.ts
grep pattern: "rateLimit|rate-limit|middleware"
  -> matches src/middleware/rate-limit.ts
read src/middleware/rate-limit.ts
start implementing
```

1
2
3
4
5
6
7

五步而不是三十步。Agent 现在拥有了行动所需的恰当背景，几乎没有其他东西。

并行阅读，适用时

一旦 `grep` 缩小了文件列表，读段就可以向外扩展。当工具调用不相互依赖时，AI SDK 并行运行：

```
Step 1: grep for the pattern (1 call)
Step 2: read auth.ts AND read rate-limit.ts (2 parallel calls)
Step 3: start implementing
```

1
2
3

别强求。如果 Agent 能单独并行读取文件，那就太好了。如果没有，那先搜索的习惯本身就是胜利。

**** 注意：快速模式需要 `grep` 才能锋利 ****

这个策略的效果取决于 `grep` 工具的效果。如果 `grep` 因为模式模糊而返回数百个匹配，Agent 最终还是会读取一半的代码库。第五模块的 50 场比赛上限在这里确实起到了作用。模型在看到上限生效时写出更窄的模式的习惯也同样重要。

动手试试

运行一个多文件任务，没有特定文件命名：

```
bun run index.ts . "Find every place this project parses JSON and tell me which
↳ ones might fail on malformed input"
```

1

你应该在任何 **read** 之前看到 **grep** 调用，**reads** 应该会出现在 **grep** 找到的文件上。

运行一个命名特定文件的任务：

```
bun run index.ts . "What's in src/sandbox-local.ts?"
```

1

Agent 应该直接送给 **read**。不需要多说 **grep**。该模式会根据文件是否已知而调整。

```
npx tsc --noEmit
```

1

提交

```
git add src/system.ts
git commit -m "feat(prompt): steer agent toward grep-first exploration"
```

1

2

完成标准

- ☐ Agent 区新增两颗子弹
- ☐ 多文件任务在 **read** 之前使用 **grep**
- ☐ 特定文件任务跳过 **grep** 直接进入 **read**
- ☐ 探索任务的总步数明显下降
- ☐ **npx tsc --noEmit**

**** 注意：当 **grep** 不够时 ****

有些模式很难被 **grep**。架构问题（“如何在应用中处理认证”）并不映射到单一的正则表达式。画一个 **survey** 工具，能接收一个高级问题并返回文件列表，并为每个问题设定一个一行角色。这应该是第 6 模块的那种探索器子 Agent，但提示词更紧凑吗？或者换个新工具？工具和子 Agent 之间的界限在哪里？

验证契约

模块 9 · 规划与验证 · 第 33/39 页 · 原文: <https://agentharnesszh.vercel.app/verification-contract>

验证契约

你在模块 3 的提示词中增加了验证部分。那是开始。这是完整版。

Agent 应合理地运行项目实际拥有的门，并报告结果，区分其导致的失败和已存在的失败。最后一点比人们想象的更重要。一个在三个人已经失败时说“考试通过”的 Agent，比一个在没有测试运行但略微失败时说“所有测试都通过”更有用。两人都在撒谎。真实的说法是“三个已有的失败，我的变更没有带来新的失败。”

学习成果

Agent 发现项目 `package.json`（以及 `AGENTS.md`（如果存在）的验证门，按已知顺序运行，并报告区分失败与既有失败的范围声明。

快速路径

1. 发现 `package.json` 脚本中的可用门
2. 按顺序运行：typecheck、lint、测试、构建
3. 报告精确命令和输出
4. 区分“我导致了这次失败”和“这本来就已经失败了”

动手练习 9.3

通过项目感知的门序列和范围化的权利要求契约扩展系统提示词。

要求：

1. 在 `index.ts`（或辅助工具）中，阅读 `package.json` 脚本并建立可用的验证命令列表
2. 将列表作为新的上下文字段传递给 `buildSystemPrompt`，然后 `verificationCommands: string[]`
3. 更新 `# Verification` 部分，列出项目的实际门，而不是通用列表
4. 添加明确的范围索赔规则：区分你的失败与已有的失败

实现提示：

- 检查 `scripts.typecheck`、`scripts["type-check"]`、`scripts.lint`、`scripts.test`、`scripts.b` 不同项目使用不同名称
- 当没有 `typecheck` 脚本且 TypeScript 处于依赖状态时，可以回退到 `npm tsc --noEmit`
- 顺序很重要。typecheck 第一个，因为它失败得最快。最后建，因为最慢
- 范围索赔规则对 Agent 诚信影响最大。状态明确地说

探索 `package.json` 的门

```
import type { Sandbox } from "./sandbox";

export async function discoverGates(sandbox: Sandbox): Promise<string[]> {
  try {
    const raw = await sandbox.readFile("package.json");
    const pkg = JSON.parse(raw);
    const scripts = pkg.scripts ?? {};
    const gates: string[] = [];

    if (scripts.typecheck || scripts["type-check"]) {
      gates.push("npm run typecheck");
    } else if (pkg.devDependencies?.typescript || pkg.dependencies?.typescript) {
      gates.push("npx tsc --noEmit");
    }

    if (scripts.lint) gates.push("npm run lint");
    if (scripts.test) gates.push("npm test");
    if (scripts.build) gates.push("npm run build");

    return gates;
  } catch {
    return [];
  }
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18

19
20
21
22
23
24

该函数返回的是 Agent 实际可以执行的命令数组。如果缺少 `package.json` 或无法读取，则数组为空，Agent 不运行门。这总比不存在的门要好。

传到提示词

```
export interface PromptContext {
  workingDirectory: string;
  sandboxType: string;
  toolNames: string[];
  gitBranch?: string;
  projectContext?: string;
  verificationCommands?: string[];
}

// In buildSystemPrompt, replace the existing Verification section:
const gates = ctx.verificationCommands?.length
  ? ctx.verificationCommands.map((c, i) => `${i + 1}. \`${c}\``).join("\n")
  : "(no verification commands discovered for this project)";

sections.push(`
# Verification
After making changes, verify your work by running these gates in order:
${gates}

Run each gate, capture the output, and report what passed and what didn't.

Distinguish failures you caused from failures that were already there:
- "Ran tsc: passed."
- "Ran npm test: 47 passed, 3 failed. The 3 failures are pre-existing in
  ↪ user.test.ts and unrelated to my changes."

Do NOT claim "tests pass" without running them. Do NOT inflate partial
verification into a blanket success claim.`);
```

1
2
3
4
5

6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27

Agent 现在看到的是项目的实际门，而不是通用的占位列表。

接入主流程

```
import { discoverGates } from "./src/verification";

const verificationCommands = await discoverGates(sandbox);

const agent = new ToolLoopAgent({
  // ...
  instructions: buildSystemPrompt({
    workingDirectory: cwd,
    sandboxType: sandbox.type,
    toolNames: Object.keys(tools),
    projectContext,
    verificationCommands,
  }),
});
```

1
2

3
4
5
6
7
8
9
10
11
12
13
14

对于一个有 `tsc` 和测试但没有构建脚本的项目，Agent 现在知道它有两个门。对于一个完全没有脚本的项目，Agent 确实知道范围需要验证。

范围化的索赔，并排呈现

Agent 会说什么	你想要什么
“所有测试都通过。”	“ <code>npm test</code> ：47 分通过，3 分不及格。这些失败其实 <code>user.test.ts</code> 就存在，和我的改动无关。”
“这套建筑有效。”	“跑 <code>npm run build</code> ：4.2 秒成功，未收到警告。”
“看起来不错。”	“跑了，TSC：通过了。lint 没有配置。测试套件通过（12 项测试）。”

左侧列是模型在提示词没有反击时的默认声音。右侧的柱子是契约的制造目标。

**** 注意：最难的门槛是 Agent 的诚实 ****

你可以布线完美门发现，Agent 仍然会显示“所有测试通过”，即使它没有运行测试。保护部队是系统提示词部分，不是发现代码。花时间在措辞上。“区分你造成的失败和已经存在的失败”是这句话的重担。

动手试试

做一个小改动，让 Agent 确认：

```
bun run index.ts . "Rename the cwd variable in src/sandbox-local.ts to workingDir,  
↩ then verify"
```

1

Agent 应当：

1. 重新命名
2. 按顺序运行发现的门
3. 报告每个门的结果，并附上具体命令和结果
4. 如果有任何门故障，请区分是重命名导致的还是本来就存在的故障

```
npx tsc --noEmit
```

1

提交

```
git add src/verification.ts src/system.ts index.ts
git commit -m "feat(verify): discover project gates and require scoped claims"
```

1

2

完成标准

- ☐ `discoverGates` 返回当前项目存在的门
- ☐ 系统提示词的验证部分列出了发现的门
- ☐ Agent 按顺序运行门，并报告准确结果
- ☐ Agent 区分了其失败与已有失败
- ☐ 在没有脚本的项目中，Agent 报告的验证是有限的
- ☐ `npx tsc --noEmit`

** 注意：快速失败，顺序正确 **

目前星门是固定顺序运行的。试着在项目中对每个项目做基准测试。`typecheck` 可能只有三秒钟。测试可能要三十个。构建可能要九十。按典型时长排序，先跑最快，这样故障能更早暴露。然后注意：有些门依赖于其他门。如果失败了，构建 `tsc` 就毫无意义。如何在不失去失效性质的情况下表达这种感觉？

参考实现

```
import type { Sandbox } from "./sandbox";

export async function discoverGates(sandbox: Sandbox): Promise<string[]> {
  try {
    const raw = await sandbox.readFile("package.json");
    const pkg = JSON.parse(raw);
    const scripts = pkg.scripts ?? {};
  }
}
```

```
const gates: string[] = [];  
  
if (scripts.typecheck || scripts["type-check"]) {  
  gates.push("npm run typecheck");  
} else if (pkg.devDependencies?.typescript || pkg.dependencies?.typescript) {  
  gates.push("npx tsc --noEmit");  
}  
  
if (scripts.lint) gates.push("npm run lint");  
if (scripts.test) gates.push("npm test");  
if (scripts.build) gates.push("npm run build");  
  
return gates;  
} catch {  
  return [];  
}  
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24

CLI 入口

模块 10 · 交互界面 · 第 34/39 页 · 原文: <https://agentharnesszh.vercel.app/cli-entry-point>

CLI 入口

你从第一模块开始就一直在 `bun run index.ts . "prompt"`。那是个 CLI。只是这不是一种礼貌的选择。

立场论证工作量太大了。沙箱后端没有标记，所以你一直在忙 `process.env.SANDBOX`。模型已硬编码在 Agent 中。如果你在跑动中按 `Ctrl-C`，沙箱也不会干净地关闭。本地的话没问题。对于云端沙箱来说，就是让虚拟机运行在别人的信用卡上。

本课正式化了切入点。通过 `parseArgs` 进行争论。沙箱通过工厂读取标志。通过带有默认值的标志来建模。通过始终运行的信号处理器关闭 `sandbox.stop()`。

学习成果

`index.ts` 解析 `--sandbox`、`--model`、位置工作目录和位置提示词。沙箱在正常退出和 `SIGINT` 时都能干净地关机。

快速路径

1. 用 `node:util` 的 `parseArgs` 的 `--sandbox` 和 `--model`
2. 从旗帜到一个小工厂构建沙箱
3. 布线一个 `SIGINT` 处理器，调用 `sandbox.stop()` 和退出
4. Agent 跑后一定要调用 `sandbox.stop()` 在 `finally`

动手练习 10.1

用 `parseArgs` 加上干净的关机替换临时 CLI。

要求：

1. 用 `node:util` 的 `parseArgs`，`--sandbox`（默认 `local`）和 `--model`（默认 `anthropic/claude-haiku-4-5`）
2. 允许位置战士：第一个是 `cwd`，其余的则合并成提示词
3. 通过 `sandboxFromFlag(name, cwd)` 助手从旗帜上构建沙箱
4. 把 Agent 跑包在 `try/finally` 里，这样 `sandbox.stop()` 总是运行
5. 注册一个 `SIGINT` 处理程序，停止沙箱并以代码 0 退出

实现提示：

- `parseArgs` 在 `node:util`。设置 `allowPositionals: true` 以混合旗帜和位置
- `sandboxFromFlag` 是一句话的切换，而不是 `"local"` 和 `"just-bash"`
- `finally` 对云端沙箱比本地更重要，但无论如何你都希望代码路径相同

CLI

```
import { parseArgs } from "node:util";
import { ToolLoopAgent, stepCountIs, pruneMessages } from "ai";
import { resolve } from "node:path";
import { existsSync, readFileSync } from "node:fs";
import { join } from "node:path";

import { createLocalSandbox } from "../src/sandbox-local";
import { createJustBashSandbox } from "../src/sandbox-just-bash";
import { buildSystemPrompt } from "../src/system";
import {
  createReadTool,
  createGrepTool,
  createBashTool,
  createTaskTool,
  createAskUserTool,
  createTodoTool,
} from "../src/tools";
import { createApproval } from "../src/approval";
import { addCacheControl } from "../src/cache";
import { discoverGates } from "../src/verification";
import type { Sandbox } from "../src/sandbox";

const { values, positionals } = parseArgs({
  args: process.argv.slice(2),
  options: {
    sandbox: { type: "string", default: "local" },
    model: { type: "string", default: "anthropic/claude-haiku-4-5" },
  },
  allowPositionals: true,
});

const cwd = resolve(positionals[0] || process.cwd());
const prompt = positionals.slice(1).join(" ") || "Hello!";

async function sandboxFromFlag(name: string, dir: string): Promise<Sandbox> {
  if (name === "just-bash") return createJustBashSandbox(dir);
  return createLocalSandbox(dir);
}

const sandbox = await sandboxFromFlag(values.sandbox!, cwd);
console.error(`Sandbox: ${sandbox.type}`);

const projectContext = existsSync(join(cwd, "AGENTS.md"))
  ? readFileSync(join(cwd, "AGENTS.md"), "utf-8")
  : undefined;
```

```
const verificationCommands = await discoverGates(sandbox);

const baseTools = {
  read: createReadTool(sandbox),
  grep: createGrepTool(sandbox),
  bash: createBashTool(sandbox, createApproval({ mode: "interactive" })),
};

const tools = {
  ...baseTools,
  task: createTaskTool(sandbox, { read: baseTools.read, grep: baseTools.grep }),
  askUser: createAskUserTool(),
  todo: createTodoTool(),
};

const agent = new ToolLoopAgent({
  model: values.model!,
  instructions: buildSystemPrompt({
    workingDirectory: cwd,
    sandboxType: sandbox.type,
    toolNames: Object.keys(tools),
    projectContext,
    verificationCommands,
  }),
  tools,
  stopWhen: stepCountIs(15),
  prepareCall: async (options) => {
    const pruned = options.messages
      ? pruneMessages({
          messages: options.messages,
          toolCalls: "before-last-3-messages",
        })
      : undefined;
    return {
      ...options,
      messages: pruned ? addCacheControl(pruned) : undefined,
    };
  },
  onStepFinish: ({ usage, stepNumber }) => {
    console.error(
      `Step ${stepNumber}: ${usage.inputTokens} input, ${usage.outputTokens}
↪ output`,
    );
  },
});

process.on("SIGINT", async () => {
  console.error("\nShutting down...");
});
```

```
    await sandbox.stop();
    process.exit(0);
  });

  try {
    const { text, steps } = await agent.generate({ prompt });
    console.log(text);
    console.log(`\n(${steps.length} steps)`);
  } finally {
    await sandbox.stop();
  }
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31

32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73

74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103

这就是完整档案。大部分内容是前九个模块已经编写好的汇编。CLI 变化包括：`parseArgs`、`sandboxFromFlag` 助手、`SIGINT` 处理员和 `try/finally`。

为什么这 `finally` 重要

如果 Agent 在中途投掷，`finally` 仍然会继续。作为本地沙箱，你没清理什么重要的东西。对于云端沙箱，你避免了让虚拟机持续运行。你释放了一些记忆，暂时有 `just-bash` 沙箱。代码相同，成本不同，但都处理得很干净。

`SIGINT` handler 是重复的。按下 Ctrl-C 的用户是一条路径。未被发现的例外是另一条路径。`finally` 覆盖正常退出，处理器覆盖显式中断。

** 注意：CLI 是薄包装 **

这份文件里几乎没有关于 CLI 担忧的内容。Agent、工具、提示词用具、沙箱：这些都是可重复使用的。CLI 部分包括五到六条 `parseArgs` 线和一个信号处理员。如果你构建了不同的表面（比如网页服务器、Slack 机器人、VS Code 扩展），唯一会变的代码就是那五六行。下面的所有东西都保持原地。

动手试试

用新旗帜跑：

```
bun run index.ts --sandbox=just-bash --model=anthropic/claude-haiku-4-5 . "Read the  
↩ package.json"
```

1

你应该在 `stderr` 看到 `Sandbox: just-bash`，然后是模型的响应 `stdout`，最后是步数。

通过在运行中途发送 `SIGINT` 来测试干净的关机：

```
bun run index.ts . "Run a long task"
```

1

按下 `Ctrl-C`。你应该看看《关闭中。.....》干净利落地退出，而不是绞刑。

```
npx tsc --noEmit
```

1

提交

```
git add index.ts  
git commit -m "feat(cli): parseArgs and SIGINT-aware shutdown"
```

1

2

完成标准

- ☐ `parseArgs` 显示 `--sandbox` 和 `--model` 旗帜
- ☐ 位置型提供 `cwd` 和提示词
- ☐ `sandbox.stop()` 正常出口（通过 `finally`）运行
- ☐ `sandbox.stop()` 在 `SIGINT` 上运行（通过 `Handler`）

□ `npx tsc --noEmit`

**** 注意：添加会话标志 ****

加上 `--session=<id>` 可以从磁盘加载之前运行的消息，并作为 `messages` 重放到 `agent.generate({ prompt, messages })`。退出时，把新消息保存回去。现在你明天可以继续聊天。文件存放在哪里？文件损坏时会发生什么？你应该给它做版本印章，这样旧会话在 Harness 更换时不会坏掉吗？

参考实现

详见上文完整 [index.ts](#)。

流式输出与工具渲染

模块 10 · 交互界面 · 第 35/39 页 · 原文: <https://agentharnesszh.vercel.app/streaming-and-tool-rendering>

流式输出与工具渲染

`agent.generate()` 封锁，直到整个回复完成。一旦 Agent 超过一两步，等待就变得不舒服。你盯着一个看起来死机的终端，完全不知道它到底是正常工作还是卡住了。

`agent.stream()` 是升级。他们到达时会收到短信 `delta`，开火时会调用工具，回来时工具结果。用户看到的是运动。你会看到 Agent 实际上在做什么。

本课将 `generate` 替换为 CLI 中的 `stream`，并决定每个区块的渲染方式。

学习成果

CLI 使用 `agent.stream()`。文本 `delta` 是实时写入 `stdout` 的。工具调用及其结果会渲染成 `stderr`，避免与 Agent 的实际响应混淆。

快速路径

1. 用 `agent.generate({ prompt })` 替换 `agent.stream({ prompt })`
2. 迭代 `result.fullStream` 并开启 `chunk.type`
3. 把 `text-delta` 发给 `stdout`，`tool-call`，`tool-result` `stderr`

动手练习 10.2

切换到流式输出，并适当渲染区块类型。

要求：

1. 用 `agent.generate(...)` 替换 `agent.stream(...)`
2. 用 `for await` 环绕 `result.fullStream`
3. 在 `text-delta` 上，将 `delta` 写入 `stdout`（无换行）
4. 在 `tool-call` 时，把工具名和 `args` 记录到 `stderr`
5. 在 `tool-result`，将截断预览记录为 `stderr`

实现提示：

- `result.fullStream` 是一个异步迭代。`for await (const chunk of result.fullStream)` 是自然环
- 工具结果可能会很长。切片到大约 100 字符以预览
- 不要把 `tool-result` 块渲染到 `stdout`。它们是元者。它们是与反应并行，而不是在其中

流式输出循环

```
const result = await agent.stream({ prompt });

for await (const chunk of result.fullStream) {
  switch (chunk.type) {
    case "text-delta":
      process.stdout.write(chunk.textDelta);
      break;
    case "tool-call":
      console.error(
        `\n[tool] ${chunk.toolName}(${JSON.stringify(chunk.args)})`,
      );
      break;
    case "tool-result": {
      const preview =
        typeof chunk.result === "string"
          ? chunk.result.slice(0, 100)
          : JSON.stringify(chunk.result).slice(0, 100);
      console.error(` -> ${preview}`);
      break;
    }
  }
}

console.log();
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18

19
20
21
22
23
24

这就是全部的改变。Agent、工具、沙箱和提示词保持不变。CLI 的工作从“等待答案”转变为“渲染直播”。

实际操作中的样子

和之前一样提示词，但多了动作：

```
[tool] grep({"pattern":"TODO","glob":"*.ts"})
-> src/auth.ts:42: // TODO: add rate limiting
-> src/routes.ts:15: // TODO: validate input

Based on my search, there are 2 TODO comments left in this project...
```

1
2
3
4
5

工具调用在模型决定制作的那一刻就会出现。结果在工具返回的那一刻就显现出来。文本答案在模型写入时会源源不断地输入。用户可以边读边看，不用等待。

各工具渲染选择

不同的工具需要不同的摘要。这里有一个起点：

工具	渲染为
read	文件路径与行数
grep	比赛数与前三场比赛
bash	命令与退出码
write	文件路径与字节数
edit	文件路径与“1 次替换”
task	子 Agent 类型和步数
askUser	完整问题和选项列表

把桌子当作提示，而不是契约。简单来说，CLI 上简单的 `tool-call` 和 `tool-result` 切换已经涵盖了大部

分需求。当某个工具的输出持续有噪声时，按工具格式化才会有其用。

**** 注意：当你流式输出时，在线审批会更难 ****

模块 1 的 `bash` 在命令未被审批时返回一个块串。这和 `generate` 一起用得很好。使用流式输出时，你可能需要暂停直播，询问用户，然后继续。那是一个交互循环，不是区块处理程序。完整的模式存在于下一个模块（可扩展性和事件）。目前，封锁举报的行为仍然是正确的做法。

动手试试

多步运行任何项目，看看 CLI 如何活起来：

```
bun run index.ts . "Find all TODO comments, then read the files that contain them"
```

1

你应该能看到工具调用在 `stderr` 中出现，模型的最终响应会流式输出 `stdout token`。

作为对比，重定向 `stderr`，只看到 Agent 的文本回复：

```
bun run index.ts . "Find all TODO comments, then read the files that contain them"
↪ 2>/dev/null
```

1

你会得到回应，没有工具噪音。这就是 `stdout/stderr` 分成给你的。

```
npx tsc --noEmit
```

1

提交

```
git add index.ts
git commit -m "feat(cli): stream agent output with chunk-level rendering"
```

1

2

完成标准

□ `agent.stream({ prompt })` 取代了 `agent.generate(...)`

- ☐ `for await` 反复强调 `result.fullStream`
- ☐ `text-delta` 给 `stdout` 写信
- ☐ `tool-call` 和 `tool-result` 写信给 `stderr`
- ☐ 重新定向 `stderr` 只剩下 Agent 的回应 `stdout`
- ☐ `npx tsc --noEmit`

**** 注意：每节课都算工具渲染 ****

用一个小型 `renderTool(chunk)` 函数替代通用 `tool-call` 和 `tool-result` 日志，该函数会启动 `chunk.toolName` 并为每个函数生成特定的摘要。`read` 显示文件路径和行数。`grep` 显示比赛数和首场比赛。`bash` 显示命令和退出码。观察新工具加入时会发生什么。每个工具的格式定义在哪里是合适的存放点？

Web 界面

模块 10 · 交互界面 · 第 36/39 页 · 原文: <https://agentharnesszh.vercel.app/web-surface>

Web 界面

Agent 是无界面。CLI 是单一表面。网络聊天界面也是一个。有趣的是，Agent 代码在它们之间不会改变。区别都出在包装上。

本课是一个概念攻略。CLI 是工作演示;Web 界面是建筑草图。关键是要让分隔足够明显，这样你就能在周五下午构建 Web 界面而不用动 Agent 。

学习成果

你可以描述 Web 界面如何重用无界面 Agent，表面在上面添加了什么（持久性、HTTP 流式输出、工具结果分量），以及 Agent 与表面边界的分布位置。

两个表面并排

	CLI	Web
输出	终端文本	聊天气泡
工具调用	stderr 文本行	工具结果组件
审批	stdin 提示	按钮组
生命周期	进程退出	会话持久化
流式输出	<code>process.stdout.write</code>	服务器发送事件
输入	通过 <code>argv</code> 一次性输入	通过文本框持续输入

Agent 代码是同一列。地面代码是其右侧的列。

Web 界面新增内容

持久性

终端关闭时，CLI 会话会中断。网页会话不应该有。表面上延续着对话：

```
await db.saveMessages(sessionId, messages);

const messages = await db.loadMessages(sessionId);
const result = await agent.stream({ prompt, messages });
```

1
2
3
4

Agent 不知道数据库的存在。它接受 `messages`（或没有）并继续前进。表面决定是否加载并保存。

流式输出 HTTP

CLI 流向 TTY。Web 界面会流向浏览器。服务器发送的事件是最简单的机制：

```
export async function POST(req: Request) {
  const { prompt, sessionId } = await req.json();
  const messages = await loadMessages(sessionId);

  const stream = new ReadableStream({
    async start(controller) {
      const result = await agent.stream({ prompt, messages });
      for await (const chunk of result.fullStream) {
        controller.enqueue(`data: ${JSON.stringify(chunk)}\n\n`);
      }
      controller.close();
    },
  });

  return new Response(stream, {
    headers: { "Content-Type": "text/event-stream" },
  });
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

16

17

18

CLI 消耗的块状和 HTTP 一样，只是超过而不是 `stdout` 。客户端会在每个块到达时渲染。

工具结果作为组成部分

CLI 将工具结果呈现为文本。Web 界面可以将它们渲染为 React 组件：

工具	CLI 渲染	Web 渲染
<code>read</code>	文件路径与行数	带语法高亮的代码块
<code>grep</code>	匹配数和前几项结果	带文件链接的搜索结果
<code>bash</code>	命令与退出码	带退出码标识的终端输出
<code>write</code>	路径与字节数	Diff 视图
<code>edit</code>	路径与“一处替换”	行内 diff
<code>askUser</code>	问题文本	选项按钮组

Agent 会发出相同的块状物质。表面决定如何展示它们。

可恢复流

用户在回复中途关闭了浏览器标签页。五分钟后他们重新打开了它。Web 界面可以从中断处恢复流（如果 Agent 仍在运行），或者继续使用持久状态（如果不运行）。CLI 做不到，因为没有地面可以回来。

这里是坚持与流式输出交汇的地方。会议是工作单元。流是该会话的一个渲染。

**** 注意：Agent 不知道有一张网 ****

Web 界面代码都不进入 Agent 。Agent 有可选项 `prompt` 且可选 `messages` 。它返回的是一连串区块。其他所有东西（认证、持久性、布局、组件）都存在于表面层。如果你发现自己在 Agent 上加上“这是网吗？”的分支，那就是缝线滑动。把特殊箱拉回表面。

随行教学缺少什么

这个模块的工作代码是 CLI 。课程仓库里没有任何 Web 界面。这个决定是有意为之。构建一个可运行的网页前端需要一个独立模块（或多个模块），教学重点是分离，而不是 React 。

如果你想尝试：先用一个包裹 `agent.stream()` 的 Next.js 路由，像 SSE 一样对区块进行管道处理，然后写一个小客户端来处理这些事件。如果你在模块 1 到模块 9 中写的 Agent 代码运行无变，你就知道分离干净了。

动手试试

这是一个概念课。自我检视：

1. 不要回头，列出 CLI 和网面之间的变化
2. 在网页案例中勾勒出单一用户的数据流提示词。持久性发生在哪里？该流在网络中穿过哪里？
3. 从你的 Harness 上选一个工具。写下 CLI 渲染，然后草图网页渲染的 React 组件。它们读到的是相同的块状吗？

提交

这节课没有代码。下一个模块（可扩展性）是基于事件的挂钩，为工具结果组件和在线审批打开了大门，以便能够干净利落地插入。

完成标准

- ☐ 你可以解释为什么 Agent 代码不会在不同表面之间变化
- ☐ 你可以草图持久化、流式输出和组件渲染，作为 Web 界面
- ☐ 你可以区分哪些留在 Agent 里，哪些会移动到表面

**** 注意：建立一个流式输出 Web 界面 ****

设置一条 Next.js 路线绕过 `agent.stream({ prompt, messages })`。把每个区块都作为服务器发送的事件进行管道处理。在客户端，使用带有 `EventSource` 或 `fetch` 加 `ReadableStream` 的事件。把 `text-delta` 块渲染到聊天气泡里。用工具名渲染 `tool-call` 块作为旋转器。`tool-result` 到时用一个小结果卡替换旋转器。添加一个取消按钮，可以终止流并存储部分结果。有趣的是文本和工具调用之间的视觉过渡，没有布局的卡顿。你怎么会句柄这样？

技能系统

模块 11 · 可扩展性 · 第 37/39 页 · 原文: <https://agentharnesszh.vercel.app/skills-system>

技能系统

给 Agent 专业知识的天真方式是把它粘贴到系统提示词里。“这是我们的授权约定。以下是我们的数据库模式。以下是测试策略。这是部署笔记。”

这对一两个包裹来说是有效的。到五岁时，系统提示词长达一万五千 token，你每调用一次都要为它们付费，Agent 则在翻找，寻找适用于今天任务的唯一一颗子弹。

技能在渐进披露中也能发挥同样的作用。名字和一句话描述永远存在系统提示词（廉价）。完整内容都存放在软盘的 markdown 文件里，只有 Agent 要求时才会加载（也很便宜，但只有在需要时才会加载）。

学习成果

`src/skills.ts` 从 `skills/<name>/SKILL.md` 文件中发现技能，在系统提示词中显示其名称和描述，并提供一个按需返回完整内容的 `loadSkill` 工具。

快速路径

1. 创建一个 `skills/` 目录，里面有一两个技能文件夹，每个文件夹包含 `SKILL.md`
2. 解析每个 `SKILL.md` 的前置内容以获取姓名和描述
3. 在系统提示词列表中添加 `skills` 部分，包含姓名和描述
4. 添加一个 `loadSkill(name)` 工具，返回全价

动手练习 11.1

实施技能发现和按需加载。

要求：

1. 定义一个 `Skill` 形状 (`name`、`description`、`path`)
2. 写一个扫描 `<dir>/<name>/SKILL.md` 并解析前置内容的 `discoverSkills(dirs: string[])`
3. 按名字去重复。第一个目录获胜，因此项目本地技能可以覆盖全局
4. 在 `buildSystemPrompt` 中添加一个 `# Skills` 部分，列出每个技能的名称和一行描述
5. 添加一个 `loadSkill` 工具，可以接收一个 `name` 并返回完整的降价内容

实现提示：

- Frontmatter 解析不需要库。--- 标记之间的两行切片对于 `name:` 和 `description:` 字段就足够了
- 限制加载技能内容，防止一个大技能上下文窗口爆（同样适用模块 5 的上限）
- 系统提示词部分应简短。每个技能一行是合适的密度。仅提供姓名和描述

技能形态

```

import { readdirSync, existsSync, readFileSync } from "node:fs";
import { join } from "node:path";

export interface Skill {
  name: string;
  description: string;
  path: string;
}

function parseFrontmatter(md: string): { description?: string } {
  if (!md.startsWith("---")) return {};
  const end = md.indexOf("\n---", 3);
  if (end < 0) return {};
  const block = md.slice(3, end);
  const descLine = block.split("\n").find((l) => l.startsWith("description:"));
  return {
    description: descLine?.replace("description:", "").trim().replace(/^[']|['"]$/
    ↵ g, ""),
  };
}

export function discoverSkills(dirs: string[]): Skill[] {
  const skills: Skill[] = [];
  const seen = new Set<string>();

  for (const dir of dirs) {
    if (!existsSync(dir)) continue;
    for (const entry of readdirSync(dir)) {
      const path = join(dir, entry, "SKILL.md");
      if (existsSync(path) && !seen.has(entry)) {
        seen.add(entry);
        const content = readFileSync(path, "utf-8");
        const { description } = parseFrontmatter(content);
        skills.push({
          name: entry,
          description: description ?? "(no description)",
          path,
        });
      }
    }
  }

  return skills;
}

```

2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43

`dirs` 是阵列，因为真正的 Harness 会在多个地方出现。项目本地技能（工作目录中）优先考虑。全局技能（在用户主目录中）填充剩余部分。

他们在提示词中浮出水面

```
export interface PromptContext {
  // ...existing fields
  skills?: { name: string; description: string }[];
}

// Inside buildSystemPrompt, after Guardrails:
if (ctx.skills?.length) {
  const lines = ctx.skills
    .map((s) => `- ${s.name}: ${s.description}`)
    .join("\n");
  sections.push(`
# Skills
The following skills are available. Call \`${s.name}\` with the name to get full
↪ content.
${lines}`);
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

Agent 每条调用都会看到技能名称和一句简短的描述。只有当 Agent 决定加载完整内容时，完整内容才会进入上下文。

装载工具

```
import { tool } from "ai";
import { z } from "zod";
import { readFileSync } from "node:fs";
import type { Skill } from "../skills";

export function createLoadSkillTool(skills: Skill[]) {
  const MAX_SKILL_CHARS = 4000;
  const byName = new Map(skills.map((s) => [s.name, s]));

  return tool({
    description: `Load the full content of a skill.
    WHEN TO USE: the task touches a domain you have a skill for (auth, db, testing,
    deployment, etc.). Check the # Skills section in your instructions.
    WHEN NOT TO USE: tasks unrelated to any available skill.
    DO NOT USE FOR: tasks where the skill name is not in the listed skills.`,
    inputSchema: z.object({
      name: z.string().describe("Skill name as listed in the Skills section"),
    }),
    execute: async ({ name }) => {
      const skill = byName.get(name);
      if (!skill) return `Unknown skill: ${name}`;
      const content = readFileSync(skill.path, "utf-8");
      return content.length > MAX_SKILL_CHARS
        ? content.slice(0, MAX_SKILL_CHARS) + `\n... (truncated at $
↪ {MAX_SKILL_CHARS} chars)`
        : content;
    },
  });
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14

15
16
17
18
19
20
21
22
23
24
25
26
27
28

上限很重要。一项已经写到一万五千字的技能，不应该在一 Agent 调用 `loadSkill` 就毁掉上下文窗口。截断消息会让模型知道如果需要重新加载并加偏移（这里没实现，但很容易添加）。

接入主流程

```
import { discoverSkills } from "./src/skills";

const skillDirs = [
  join(cwd, "skills"),
  join(process.env.HOME ?? "", ".harness", "skills"),
];
const skills = discoverSkills(skillDirs);

const tools = {
  // ...everything else
  loadSkill: createLoadSkillTool(skills),
};

const instructions = buildSystemPrompt({
  // ...existing context
  skills: skills.map((s) => ({ name: s.name, description: s.description })),
});
```

1
2
3
4
5
6

7
8
9
10
11
12
13
14
15
16
17

先规划技能，再做全局规划。姓名和描述见提示词。内容通过工具按需加载。

为什么要“名字进，内容出”

数据说明了一切。五个技能，每个一千字，大约相当于五千个技能 token 永远加到每个调用上。五个技能加上一句话描述，总共一百 token 。模型仍然知道这些技能的存在。Agent 根据任务决定加载哪些。只有有原因，完整内容才会进入上下文。

这与第五模块中的预防优先清理学科相同，应用于知识层而非工具输出层。

**** 注意：模型必须提出问题 ****

模型不会自动加载技能。系统提示词给它们命名，工具也在表面上。模型还需要决定是否调用 `loadSkill` 。把这当作取回路径，而不是保证。注意你的游戏过程：如果模型从未加载明显有用的技能，你的技能描述需要更犀利的钩子。

动手试试

把一个示例技能放进 `skills/auth-patterns/SKILL.md`：

```
---  
description: Patterns and pitfalls for adding authentication to this project  
---  
# Auth Patterns  
  
This project uses NextAuth with JWT sessions. Key files:  
  
- `lib/auth.ts`: NextAuth config  
- `middleware.ts`: route protection  
...
```

1
2
3

4
5
6
7
8
9
10

执行一个应该能触发该技能的任务：

```
bun run index.ts . "Add OAuth login to this project. Check the auth-patterns skill  
↪ first."
```

1

你应该看到 **name: "auth-patterns"** 的 Agent 调用 **loadSkill**，然后继续使用加载的内容。

运行一个无关的任务，确认 Agent 没有加载任何内容：

```
bun run index.ts . "What's the syntax for a TypeScript const assertion?"
```

1

没有技能负担。系统提示词提到技能存在；模型知道当它们不适用时，不要去追求它们。

```
npx tsc --noEmit
```

1

提交

```
git add src/skills.ts src/tools.ts src/system.ts index.ts skills/  
git commit -m "feat(skills): progressive-disclosure skill loading"
```

1

2

完成标准

- ☐ **discoverSkills** 扫描目录并解析前置文件

- ☐ 系统提示词列出技能并用一句话描述
- ☐ `loadSkill` 工具按需返回完整内容，但有上限
- ☐ 当名称冲突时，项目-本地技能覆盖全局技能
- ☐ 命名技能的任务触发 `loadSkill`；无关任务则不行
- ☐ `npx tsc --noEmit`

**** 注意：分段目标载荷 ****

现在 `loadSkill` 还原整个文件。对于五千字的技能来说，这很浪费，因为 `Agent` 只需要一个章节。通过添加一个可选的 `section: string` 参数，只返回请求的标题及其内容。现在模型可以加载 `loadSkill({ name: "auth-patterns", section: "OAuth flow" })` 了。你在哪里划清“技能作为文档”和“技能作为一个小型可搜索语料库”之间的界限？

参考实现

请参见上方完整代码块。练习方案就是应用到你的文件上的同一套代码。

自定义工具

模块 11 · 可扩展性 · 第 38/39 页 · 原文: <https://agentharnesszh.vercel.app/custom-tools>

自定义工具

你现在 `index.ts` 是手工制作工具列表。 `read`、`grep`、`bash`、`task`、`askUser`、`todo`、`loadSkill`。这对课程里的七个人来说是可以的。当有人想添加自己的 `deploy` 工具，或者用项目特定的安全命令包裹 `bash` 而不分叉 Harness 时，这种方法就不起作用。

一个工具注册表是接缝。新工具通过它注册。现有工具可以组合成新的工具。核心 Harness 保持不变。

学习成果

`src/registry.ts` 暴露了一个工具注册表，包含 `register`、`get`、`list` 和一个小型 `wrapTool` 辅助。Agent 的工具集是基于注册表构建的，而不是硬编码在 `index.ts` 中。

快速路径

1. 定义一个包含 `register(name, tool)`、`get(name)`、`list()` 的 `ToolRegistry`
2. 把内置工具线路移到 `registerBuiltins(registry, sandbox)` 辅助器里
3. 添加 `wrapTool(base, { beforeExecute, afterExecute })` 用于合成
4. 用 `registry.list()` 而不是内嵌对象来构建 Agent 的工具集

动手练习 11.2

构建注册表并迁移现有工具到那里。

要求：

1. `src/ registry.ts` 导出时 `createRegistry()` 返回一个对象，包含 `register`、`get`、`list` 和 `entries`
2. 内置工具通过 `registerBuiltins(registry, sandbox)` 注册。和之前一样的套装
3. `wrapTool(base, hooks)` 还回来了一个新的工具，可以 `beforeExecute` 运行，`afterExecute`base.execute`
4. `index.ts` 中，Agent 的工具来自 `Object.fromEntries(registry.entries())` 而非内联对象
5. 演示一个定制套位（`deploy` 工具）以证明接缝有效

实现提示：

- 注册表是一个 `Map<string, Tool>` 加上三到四个方法名称。别建得太过头
- `wrapTool` 是一个薄函数，返回一个新工具。基础工具保持不变
- `entries()` 退回 `[name, tool][]` 让 `Object.fromEntries` 干净利落地工作

注册表

```
import type { Tool } from "ai";

export interface ToolRegistry {
  register(name: string, tool: Tool): void;
  get(name: string): Tool | undefined;
  list(): string[];
  entries(): [string, Tool][];
}

export function createRegistry(): ToolRegistry {
  const tools = new Map<string, Tool>();
  return {
    register: (name, tool) => {
      tools.set(name, tool);
    },
    get: (name) => tools.get(name),
    list: () => [...tools.keys()],
    entries: () => [...tools.entries()],
  };
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20

注册表不拥有任何策略。它是一个带有打字界面的映射。谁调用 `register` 谁决定什么能进去。

内置辅助器

```
import type { Sandbox } from "./sandbox";
import { createReadTool, createGrepTool, createBashTool, createTaskTool,
  ↵ createAskUserTool, createTodoTool, createLoadSkillTool } from "./tools";
import { createApproval } from "./approval";
import type { Skill } from "./skills";

export function registerBuiltins(
  registry: ToolRegistry,
  sandbox: Sandbox,
  skills: Skill[],
) {
  registry.register("read", createReadTool(sandbox));
  registry.register("grep", createGrepTool(sandbox));
  registry.register(
    "bash",
    createBashTool(sandbox, createApproval({ mode: "interactive" })),
  );
  registry.register(
    "task",
    createTaskTool(sandbox, {
      read: registry.get("read")!,
      grep: registry.get("grep")!,
    }),
  );
  registry.register("askUser", createAskUserTool());
  registry.register("todo", createTodoTool());
  registry.register("loadSkill", createLoadSkillTool(skills));
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13

14
15
16
17
18
19
20
21
22
23
24
25
26
27

顺序在这里很重要。task 需要 read 和 grep 已经存在于注册表中，因为它会生成使用它们的子 Agent。如果顺序错误，你会注册 task 引用未明确。

包装器

```
import { tool, type Tool } from "ai";

interface WrapHooks {
  beforeExecute?: (input: any) => any | Promise<any>;
  afterExecute?: (result: any) => any | Promise<any>;
}

export function wrapTool(base: Tool, hooks: WrapHooks): Tool {
  return tool({
    description: base.description,
    inputSchema: base.inputSchema,
    execute: async (input) => {
      const transformed = hooks.beforeExecute ? await hooks.beforeExecute(input) :
      ↪ input;
      const result = await base.execute(transformed);
      return hooks.afterExecute ? await hooks.afterExecute(result) : result;
    },
  });
}
```

1
2
3
4

5
6
7
8
9
10
11
12
13
14
15
16
17
18

`beforeExecute` 可以重写输入。`afterExecute` 可以对输出进行后处理。这两种方式都是可选的。基础工具保持不变，因此项目可以包裹内置功能而不破坏其他用户的内置功能。

接入 CLI

```
import { createRegistry, registerBuiltins } from "./src/registry";

const registry = createRegistry();
registerBuiltins(registry, sandbox, skills);

registry.register("deploy", tool({
  description: `Deploy the project to a target environment.
  WHEN TO USE: pushing changes to staging or production.
  WHEN NOT TO USE: testing changes (use bash with the test runner instead).`,
  inputSchema: z.object({
    environment: z.enum(["staging", "production"]),
  }),
  execute: async ({ environment }) => {
    const { stdout } = await sandbox.exec(`vercel deploy --${environment}`);
    return stdout;
  },
}));

const agent = new ToolLoopAgent({
  // ...
  tools: Object.fromEntries(registry.entries()),
  instructions: buildSystemPrompt({
    // ...
    toolNames: registry.list(),
  }),
});
```

```
});
```

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
```

Agent 的 `tools` 场现在由注册表推导出来。添加新工具只需一个 `registry.register(...)` 调用，Agent 就建好了。移除工具只是一条线。

为项目包装 `bash`

这就是 `wrapTool` 赢得其地位的原因：

```
const baseBash = registry.get("bash")!;
registry.register("bash", wrapTool(baseBash, {
  beforeExecute: (input) => {
    if (input.command.startsWith("bun test")) {
      return { ...input, command: input.command + " --reporter=spec" };
    }
    return input;
  }
});
```

```
    },  
  }));
```

1
2
3
4
5
6
7
8
9

基础 `bash` 仍在记忆中; 注册表现在收录了一个包装版本, 增加了针对项目的调整。Agent 从未见过未包装的那张。Harness 核心从未改变。

**** 注意: 延伸曲面表 ****

表面	你可以自定义的内容	怎么会这样
工具	添加、移除、包裹	注册表加 <code>wrapTool</code>
技能	增加专业知识	<code>skills/</code> 目录加 <code>loadSkill</code>
沙箱	自定义后端	<code>createSandbox</code> 工厂
审批	自定义策略	配置加事件 (下一课)
系统提示词	自定义部分	<code>PromptContext</code> 加 <code>buildSystemPrompt</code>
模型	每个榜样	子 Agent 定义

每一行都映射到你已经写好的代码。注册表是第一排的入口。其余的也是伸展曲面, 每个都有自己的约定。

动手试试

添加一个专门针对项目的小工具来确认接缝:

```
registry.register("now", tool({  
  description: "Return the current timestamp",  
  inputSchema: z.object({}),  
  execute: async () => new Date().toISOString(),  
}));
```

1
2
3

4

5

运行一个使用它（Nits）的任务：

```
bun run index.ts . "What's the current timestamp? Use the now tool."
```

1

Agent 应该调用 `now` 并返回时间戳。移除 `register` 线，运行同一个提示词，Agent 应该会报告 `now` 不存在。

```
npx tsc --noEmit
```

1

提交

```
git add src/registry.ts index.ts
git commit -m "feat(registry): tool registry with wrapTool composition"
```

1

2

完成标准

- ☐ `createRegistry` 退还了一台工作用注册表
- ☐ `registerBuiltins` 线连接七个内置工具
- ☐ `wrapTool` 在基础工具周围构建钩子
- ☐ Agent 的 `tools` 场地建于 `registry.entries()`
- ☐ `registerBuiltins` 之后注册的自定义工具可被 Agent 呼叫
- ☐ `npx tsc --noEmit`

** 注意：替换，不要包裹 **

包裹工具可以保持底座不变。更换一个后，它会完全消失。添加 `registry.unregister(name)` 和一个模式，项目可以将内置 `bash` 替换为带有更小安全前缀列表的项目专用版本。替换需要在自助顺序中发生在哪里？如果 Agent 建好后才更换，什么会坏？

参考实现

请参见上方完整代码块。

扩展点

模块 11 · 可扩展性 · 第 39/39 页 · 原文: <https://agentharnesszh.vercel.app/extension-points>

扩展点

上一课的注册表句柄“有哪些工具”。它不句柄“工具调用周围发生了什么”。记录每一个调用。阻断写入特定文件。操作系统级沙箱中对命令进行封装。关机时自动承诺。这些都是跨字段的问题，不适合放在任何单一工具里。

事件是正确的原始。该束带会发出生命周期事件。扩展订阅。每个用户都可以在 Harness 继续前通过、屏蔽或修改事件。多个订阅者连锁。

这节课就是建筑草图。把事件总线装进工作 Harness 很简单，但价值在于在接线前先了解契约。

学习成果

你可以描述一个在关键生命周期事件触发的事件总线，可以阻塞或修改的处理程序，以及使多处理程序链可预测的顺序规则。

事件表面

```
type LifecycleEvent =
  | "session_start"
  | "tool_call"
  | "tool_result"
  | "session_before_compact"
  | "session_shutdown";

type EventResult = { block?: boolean; reason?: string; modify?: any } | void;

interface EventBus {
  on(event: LifecycleEvent, handler: (data: any) => Promise<EventResult>): void;
  emit(event: LifecycleEvent, data: any): Promise<EventResult[]>;
}
```

1
2
3
4
5
6
7
8
9

10
11
12
13

这些活动本身故意设计得很小。五个名字涵盖了分机通常需要插入的时刻。之后再加更多内容没问题；以五开頭是保持契约清晰可辨的关键。

四个例子

一旦你看到延伸的实际用途，形状就变得明显了。

每当记工具调用

```
bus.on("tool_call", async ({ toolName, input }) => {  
  console.error(`[${new Date().toISOString()}] ${toolName}: $  
  ↪ {JSON.stringify(input)}`);  
});
```

1
2
3

没有回报价值。操作者通过。Harness 继续和调用一起。

块写入受保护文件

```
const PROTECTED = [".env", "package-lock.json"];  
  
bus.on("tool_call", async ({ toolName, input }) => {  
  if (toolName === "write" && PROTECTED.some((p) => input.path.endsWith(p))) {  
    return { block: true, reason: `${input.path} is protected by policy.` };  
  }  
});
```

1
2
3
4
5
6
7

处理函数回 `block: true`。Harness 会停止调用，并将理由反馈给模型作为工具结果。模型以明文形式看到策略，并向用户报告。

在压实前注入安全防提示词

```
bus.on("session_before_compact", async () => {
  return {
    modify: {
      customInstructions:
        "Preserve all safety constraints and approval rules across compaction.",
    },
  };
});
```

1
2
3
4
5
6
7
8

处理函数回 `modify`。Harness 在继续前施加修改（此处为额外的指令线）。压缩是指指令可能泄露的时刻；这是防止泄漏的一种方法。

关机时自动提交

```
bus.on("session_shutdown", async ({ sandbox }) => {
  const { stdout } = await sandbox.exec("git status --porcelain");
  if (stdout.trim()) {
    await sandbox.exec(`git add -A && git commit -m "WIP: auto-save"`);
  }
});
```

1
2
3
4
5
6

这是模块 4 中的云-沙箱 `beforeStop` 钩子，推广版。任何因任何原因结束的会话都有机会检查其工作。

链式规则

多个处理器可以订阅同一个事件。它们按注册顺序运行。如果有处理器返回 `block: true`，调用停止，原因会回到模型。如果有返回 `modify`，后续处理程序会看到修改后的数据。

```
Tool call requested
-> emit "tool_call"
    handler 1: log (pass through)
    handler 2: check protected files (may block)
    handler 3: project safety policy (may block)
-> if any blocked: return reason to model, do not execute
-> if all passed: execute tool
-> emit "tool_result"
    handler 1: log result
    handler 2: telemetry
```

1
2
3
4
5
6
7
8
9
10

秩序很重要。在安全检查前记录，即使被阻止，也能捕捉到调用尝试。遥测结果后，记录了实际运行的部分。顺序正确，链条就能产生有用的痕迹。如果记错了，你就只记下了一半的故事。

事件如何与你已有的构建结合

这些片段开始以一种积极的方式重叠：

- 模块 2 的审批配置设定了操作模式（交互式、后台、委派）。它仍然在工具层面运行
- 事件总线绕着工具层运行。`tool_call` 处理器即使审批已通过，也可以进行封锁
- 模块 4（`afterStart`、`beforeStop`）的生命周期钩子与 `session_start` 和 `session_shutdown` 重叠。处理器签名更为通用；生命周期钩子是最常见的情况的方便名称
- 11.1 的技能系统是一个独立的检索界面。它不会经历事件，因为是模型决定是否加载技能，而不是 Harness

这个工具最终是分层的：工具在底部，围绕工具的事件，在会话边界有生命周期钩子，技能作为发现的知识，注册作为所有事物的入口。每一层都有自己的职责。

**** 注意：为什么这是最后一课 ****

事件总线是最柔软且最危险的延伸表面。不良处理者可能导致 Agent 死锁，通过日志泄露机密，或阻断合法工具调用。最后构建（在工具、工具、沙箱、提示词、上下文、子 Agent 和生命周期钩子之后）意味着你在插入前已经清楚自己在接入什么。

如果你早点接线，可能会有诱惑去用事件钩子来句柄所有问题。先学会其他层次的纪律，正是防止 Harness 变成一个庞大的 `on('tool_call', ...)` 操作者的关键。

动手试试

这是一个概念课。自我检视：

1. 不要回头，说出五个生命周期事件
2. 对于每个扩展，描述一个现实可行的扩展
3. 追踪当两名 `tool_call` 员同时返回 `block: true` 时会发生什么。谁的理智会赢？
4. 请解释事件总线与模块 2 的审批配置之间的关系。它们在哪些地方重叠，哪些地方没有？

如果你想真正构建这个，实现很简单：一个 `Map<string, Handler[]>` 和一个 `emit`，按顺序运行处理器，提前退出块上。在工具执行前和工具结果后插入 Agent 循环。模块 4 的生命周期钩子成为前两个订阅者。

提交

这节课没有代码，除非你接线总线。如果有，建议在另一个分支提交，并用日志扩展练习，再添加任何阻塞扩展。

完成标准

- ☐ 你可以说出五个生命周期事件
- ☐ 你可以描述直通、阻塞和修改返回值
- ☐ 你可以追踪多处理链并预测结果
- ☐ 你可以解释总线如何补充（而不是替代）审批配置

**** 注意：建一个遥测扩展 ****

订阅 `tool_call`、`tool_result` 和每步活动。记录时间戳、持续时间和 token 计数。在事件发生时将事件附加到 JSONL 文件上，这样崩溃时不会丢失追踪。会话结束时，生成一个单屏报告：总时间、工具调用计时、最慢工具、总 token。现在用不同的系统提示词做两次任务，然后把遥测差异化。哪个提示词工具的调用更少？减少浪费？这就是你如何进行 A/B 测试 Agent 行为而不猜测。